

РЕФЕРАТ

Курсовая работа посвящена разработке интерактивной онлайн-доски для совместной работы с поддержкой искусственного интеллекта. Целью работы является создание программного средства, позволяющего нескольким пользователям одновременно создавать, редактировать и совместно использовать визуальные доски, организовывать их в рабочие пространства, а также применять встроенного AI-ассистента для автоматического построения схем, диаграмм и текстовых блоков на холсте.

В ходе выполнения работы были рассмотрены подходы к построению совместно-редактируемых веб-систем, а также обосновано применение микросервисной архитектуры. Разработано клиент-серверное приложение: серверная часть реализована на платформе Node.js с использованием фреймворка NestJS, межсервисного взаимодействия по протоколу gRPC, СУБД PostgreSQL, кэша Redis и объектного хранилища MinIO; клиентская часть выполнена на базе Next.js 16 и React 19 с применением TypeScript, Tailwind CSS, библиотеки TanStack React Query и собственного движка отрисовки на HTML5 Canvas. Для обеспечения совместного редактирования в реальном времени применена технология WebSocket (Socket.IO).

Ключевым результатом работы стало создание функционирующего веб-приложения, обеспечивающего совместное создание интерактивных досок, организацию рабочих пространств с разграничением прав доступа, общение через AI-чат с возможностью прямого построения элементов на холсте, а также администрирование платформы. Дополнительно реализованы аутентификация по электронной почте с одноразовыми кодами, OAuth-вход через Google и GitHub, аудит действий пользователей, экспорт досок и контейнеризованное развертывание на основе Docker Compose. Полученное программное средство может использоваться образовательными учреждениями, проектными командами и компаниями для проведения совещаний, мозговых штурмов и совместного проектирования.

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

БД – база данных.

ИИ – искусственный интеллект.

ПО – программное обеспечение.

СУБД – система управления базами данных.

AI – Artificial Intelligence, искусственный интеллект.

API – Application Programming Interface, программный интерфейс приложения.

CRUD – Create, Read, Update, Delete, базовые операции с данными.

CSRF – Cross-Site Request Forgery, межсайтовая подделка запроса.

CSS – Cascading Style Sheets, каскадные таблицы стилей.

DTO – Data Transfer Object, объект передачи данных.

ER – Entity-Relationship, модель «сущность — связь».

gRPC – Google Remote Procedure Call, бинарный протокол удалённого вызова процедур.

GUI – Graphical User Interface, графический пользовательский интерфейс.

HTML – HyperText Markup Language, язык гипертекстовой разметки.

HTTP – HyperText Transfer Protocol, протокол передачи гипертекста.

HTTPS – HyperText Transfer Protocol Secure, защищённая версия HTTP.

IP – Internet Protocol, сетевой протокол передачи данных.

JSON – JavaScript Object Notation, текстовый формат обмена данными.

JWT – JSON Web Token, формат подписанного токена для аутентификации.

LLM – Large Language Model, большая языковая модель.

MIME – Multipurpose Internet Mail Extensions, стандарт описания типа содержимого.

MVC – Model–View–Controller, модель–представление–контроллер.

OAuth – Open Authorization, открытый протокол авторизации через сторонних провайдеров.

ORM – Object-Relational Mapping, объектно-реляционное отображение.

OTP – One-Time Password, одноразовый пароль.

REST – Representational State Transfer, архитектурный стиль построения распределённых систем.

RPC – Remote Procedure Call, удалённый вызов процедуры.

S3 – Simple Storage Service, объектное хранилище файлов.

SDK – Software Development Kit, набор инструментов разработчика.

SHA – Secure Hash Algorithm, алгоритм криптографического хеширования.

SMTP – Simple Mail Transfer Protocol, протокол передачи электронной почты.

SQL – Structured Query Language, язык структурированных запросов.

SSR – Server-Side Rendering, рендеринг на стороне сервера.

TCP – Transmission Control Protocol, протокол управления передачей данных.

TTL – Time To Live, время жизни значения в кэш-хранилище.

UI/UX – User Interface / User Experience, интерфейс пользователя и пользовательский опыт.

UML – Unified Modeling Language, унифицированный язык моделирования.

URL – Uniform Resource Locator, адрес ресурса в сети.

WS – WebSocket, протокол двунаправленной связи поверх TCP.

XML – eXtensible Markup Language, расширяемый язык разметки.

XSS – Cross-Site Scripting, межсайтовое выполнение скриптов.

YAML – YAML Ain't Markup Language, формат записи конфигурационных данных.

СОДЕРЖАНИЕ

Введение	8
1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ	11
1.1 Цель разработки	11
1.2 Назначение системы.....	11
1.3 Профиль предметной области	12
1.4 Методология разработки.....	14
1.5 Требования к UI/UX и эргономике.....	16
1.6 Архитектура и технологические требования.....	19
1.7 Функциональные требования	21
1.8 Функциональные требования к модулю искусственного интеллекта	22
1.9 Требования к данным и базе данных	24
1.10 Требования к проекту по стандартам	27
1.11 Тестирование (требования)	29
1.12 Документация и ввод в эксплуатацию (требования)	31
1.13 Оценивание (критерии успешности)	32
2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ.....	35
2.1 Диаграмма вариантов использования	35
2.2 Диаграммы активности	38
2.3 Диаграмма процесса обработки данных	45
2.4 Диаграмма классов	49
2.5 Диаграмма развертывания	56
3. РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ	60
3.1 Структура проекта и стек технологий.....	60
3.2 Реализация серверной части	64
3.3 Реализация пользовательского интерфейса	74
3.4 Реализация модуля искусственного интеллекта.....	83
3.5 Контейнеризация и совместный запуск	87
4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ.....	99
5. ДОКУМЕНТАЦИЯ И ВВОД В ЭКСПЛУАТАЦИЮ.....	113
5.1 Текстовое описание системы	114
5.2 Руководство пользователя	116
5.3 Руководство администратора	118
5.4 Описание API.....	122

5.5 Инструкция запуска	126
ПРОВЕРКА СООТВЕТСТВИЯ ТРЕБОВАНИЯМ.....	132
ЗАКЛЮЧЕНИЕ	139
Список использованных источников	141

Введение

Современные проектные команды, образовательные учреждения и распределенные коллективы все активнее используют визуальные средства совместной работы — интерактивные доски, карты идей и диаграммы [1, 2]. С распространением удаленного формата работы потребность в облачных интерактивных досках значительно возросла, а сама задача организации совместного редактирования визуального контента стала самостоятельной инженерной проблемой.

Наиболее известными представителями данного класса систем являются Miro, FigJam, Unidraw и Microsoft Whiteboard [3, 4, 5, 6]. В то же время большинство существующих систем являются проприетарными, имеют ограничения по числу досок и участников в бесплатных тарифах, а также не предоставляют гибких средств интеграции с собственными корпоративными процессами. Развитие больших языковых моделей открывает принципиально новый класс возможностей — автоматическую генерацию визуальных структур (mind-map, flow-диаграмм, организационных схем) непосредственно на холсте по текстовому описанию пользователя [7, 8].

Объектом исследования является процесс совместной работы пользователей с интерактивной графической доской в режиме реального времени.

Предметом исследования являются программные решения построения веб-приложений для совместного редактирования визуального контента и интеграции с моделями искусственного интеллекта.

Таким образом, **целью** проекта является решение задачи организации совместной визуальной работы пользователей путем разработки веб-приложения, обеспечивающего создание и редактирование интерактивных досок в режиме реального времени, организацию рабочих пространств, разграничение прав доступа и взаимодействие с встроенным AI-ассистентом, формирующим элементы доски по текстовому запросу пользователя.

Для достижения поставленной цели необходимо решить следующие **задачи**:

- разработка клиент-серверного веб-приложения, предназначенного для совместного создания и редактирования интерактивных досок;
- создание серверной части приложения на платформе Node.js с использованием фреймворка NestJS и микросервисной архитектуры с межсервисной коммуникацией по протоколу gRPC;
- разработка клиентской части на базе Next.js 16 и React 19 с типизацией TypeScript и стилизацией Tailwind CSS, обеспечивающей удобный пользовательский интерфейс и собственный движок отрисовки холста;
- реализация совместного редактирования в реальном времени посредством технологии WebSocket с применением библиотеки Socket.IO;
- интеграция модуля искусственного интеллекта на основе модели DeepSeek с поддержкой инструментов (tool-calling) для автоматического построения текстовых блоков и стрелок на холсте по текстовому запросу пользователя;
- обеспечение хранения данных в СУБД PostgreSQL, файлов — в объектном хранилище MinIO, а сессионных и кэшируемых данных — в Redis;
- реализация механизмов аутентификации (электронная почта + одноразовый код) и социального входа через Google и GitHub с использованием стандарта OAuth 2.0 и токенов JWT;
- разработка административной панели с учетом аудита действий, аналитики и управления пользователями и пространствами;
- обеспечение контейнеризованного развертывания всей системы с помощью Docker Compose.

Для решения поставленных задач используются современные веб-технологии и инструментальные средства: TypeScript, Node.js, NestJS, gRPC, Next.js, React, PostgreSQL, Redis, MinIO, Socket.IO, Docker, а также модель искусственного интеллекта DeepSeek в режиме tool-calling.

Таким образом, разработка этого приложения является актуальной задачей, поскольку существующие коммерческие платформы не покрывают всех потребностей образовательных и проектных коллективов, а интеграция AI-

ассистента непосредственно с холстом представляет собой новое и востребованное направление развития систем совместной визуальной работы.

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

1.1 Цель разработки

Целью разработки является создание программной системы, предназначенной для совместного создания и редактирования интерактивных досок (whiteboard) в режиме реального времени, организации этих досок в рабочие пространства (Spaces), разграничения прав доступа участников и предоставления интегрированного AI-ассистента, способного по запросу пользователя автоматически добавлять на доску текстовые блоки и связи между ними.

Конечным результатом разработки должна стать работоспособная веб-информационная система, позволяющая пользователю проходить аутентификацию по электронной почте и одноразовому коду либо через провайдеров Google и GitHub, создавать рабочие пространства, приглашать в них других участников по инвайт-коду, создавать в пространствах доски, совместно с другими пользователями редактировать содержимое доски (текст, стикеры, фигуры, свободное рисование, стрелки и изображения), наблюдать в реальном времени курсоры и действия других участников, а также отправлять команды AI-ассистенту, который автоматически встраивает в доску готовые элементы и связи.

Таким образом, в качестве конечного результата разработки рассматривается создание веб-системы, объединяющей инструменты совместной визуальной работы, организации проектных пространств, разграничения прав доступа и интеллектуальной генерации визуальных конструкций средствами искусственного интеллекта.

1.2 Назначение системы

Разрабатываемая система предназначена для автоматизации процессов, связанных с совместной визуальной работой проектных команд, образовательных групп и распределенных коллективов. Основное назначение системы заключается в том, чтобы предоставить пользователям единое онлайн-пространство, в котором они могут совместно создавать визуальные артефакты — диаграммы, схемы, карты

идей, mind-map, наброски архитектуры и заметки — без необходимости использовать локальные графические редакторы и пересылать друг другу файлы.

С точки зрения выполняемых функций система должна поддерживать несколько ролей пользователей. Неавторизованный пользователь получает доступ к ознакомительной странице приложения и функциям регистрации и авторизации. Зарегистрированный пользователь получает доступ к личному кабинету, где он может создавать рабочие пространства и доски, приглашать других участников по инвайт-коду, редактировать собственный профиль, привязывать OAuth-провайдеров и обмениваться сообщениями с AI-ассистентом. Дополнительно в системе предусмотрены три уровня административных ролей с нарастающим набором прав. Менеджер имеет доступ к административной панели в режиме чтения и работает с общей аналитикой, списками пользователей, досок и пространств, мониторингом активности в реальном времени и проверкой состояния микросервисов. Администратор дополнительно может изменять роли пользователей, блокировать и удалять учетные записи, удалять любые доски и пространства без проверки владельца, принудительно отключать пользователей от WebSocket-соединений и просматривать журнал административных действий. Супер-администратор обладает максимальными правами.

Особое место в системе занимает интеллектуальная составляющая, поскольку именно AI-ассистент, основанный на большой языковой модели DeepSeek, выступает дополнительным каналом взаимодействия пользователя с холстом. Он принимает текстовые сообщения пользователя в чате доски, анализирует текущее состояние холста (расположение и содержимое элементов) и через механизм tool-calling вызывает функции создания текстовых блоков и соединительных стрелок в виде кривых Безье. Это позволяет автоматически формировать на доске структурированные конструкции по короткому описанию пользователя.

1.3 Профиль предметной области

Предметная область разрабатываемой системы связана с организацией совместной визуальной работы пользователей в режиме реального времени на интерактивной графической доске. Данная область характеризуется высокой практической значимостью, поскольку именно визуальное представление идей, схем и связей между понятиями обеспечивает эффективную коммуникацию в проектных командах, учебных группах и распределенных коллективах. При этом сам процесс совместной работы является сложным, так как на качество взаимодействия влияют различные факторы: одновременное редактирование одного и того же визуального полотна несколькими участниками, необходимость согласованного состояния доски у всех клиентов, поддержка разнородных типов содержимого (текстовых блоков, стикеров, графических фигур, соединительных стрелок в виде кривых Безье, свободного рисования и изображений), а также разграничение прав доступа на уровне рабочих пространств и отдельных досок. В связи с этим задача организации совместной работы требует применения сетевых технологий двунаправленного обмена данными и средств интеллектуальной обработки пользовательских запросов, в том числе методов на основе больших языковых моделей.

Если рассматривать текущую ситуацию в предметной области по принципу «как есть», можно отметить несколько характерных особенностей и проблем. Во-первых, существующие коммерческие платформы совместной визуальной работы являются проприетарными облачными сервисами с ограниченными бесплатными тарифами, что ограничивает их применение в образовательных и проектных коллективах с большим количеством участников. Во-вторых, традиционные решения предоставляют средства искусственного интеллекта в виде отдельного чата либо вовсе не интегрируют их с холстом, в результате чего пользователь вынужден вручную переносить полученные подсказки в виде текстовых блоков и связей. В-третьих, без специализированной информационной системы отдельные операции, такие как аутентификация пользователей, организация рабочих пространств, разграничение прав доступа, хранение загружаемых файлов и ведение журнала действий, выполняются разрозненно и требуют дополнительных

трудозатрат на интеграцию. Это создает узкие места в процессе совместной работы и снижает оперативность получения визуального результата.

Основными участниками процессов в данной предметной области выступают неавторизованные посетители, зарегистрированные пользователи, владельцы и участники рабочих пространств, а также администраторы различного уровня, обеспечивающие сопровождение системы и управление учетными записями. К основным операциям относятся регистрация и аутентификация по электронной почте с одноразовым кодом либо через провайдеров Google и GitHub, создание рабочих пространств и приглашение в них участников, создание и совместное редактирование интерактивных досок с поддержкой различных типов содержимого, взаимодействие с AI-ассистентом и автоматическое построение визуальных элементов на холсте по текстовому запросу, а также администрирование пользователей, досок и пространств с ведением журнала действий. Именно из такого профиля предметной области вытекают основные требования к системе: наличие удобного веб-интерфейса с собственным движком отрисовки холста, поддержка совместного редактирования в реальном времени, аутентификация и разграничение прав доступа, интеграция модели искусственного интеллекта с поддержкой вызова инструментов, объектное хранилище для загружаемых файлов и административная панель с инструментами аналитики и аудита.

1.4 Методология разработки

Для разработки данной системы целесообразно использовать каскадную модель жизненного цикла, так как проект имеет достаточно четко определенную цель, понятную предметную область и заранее выделенные основные этапы работ. В рамках курсовой работы структура разрабатываемого решения известна заранее: требуется создать клиент-серверное веб-приложение для совместной работы с интерактивными досками, реализовать пользовательский интерфейс с собственным движком отрисовки холста, серверную часть на основе микросервисной архитектуры, базу данных, механизмы аутентификации и

социального входа, модуль совместного редактирования в реальном времени, а также модуль искусственного интеллекта с поддержкой вызова инструментов. При таких условиях каскадная модель является удобной, поскольку позволяет последовательно переходить от анализа требований к проектированию, затем к программной реализации, тестированию и итоговому вводу в эксплуатацию.

Выбор каскадной модели также обоснован тем, что результаты каждого этапа непосредственно используются на следующем этапе. Сначала определяется назначение системы, состав функций, роли пользователей и основные требования. Затем на этой основе проектируется архитектура решения, состав микросервисов и протоколы их взаимодействия, структура классов, схема базы данных, интерфейсы пользователя и логика взаимодействия компонентов. После этого выполняется программная реализация серверной части на платформе Node.js с использованием фреймворка NestJS и протокола gRPC, клиентской части на основе Next.js и React, а также модуля искусственного интеллекта на базе модели DeepSeek. Далее проводится проверка работоспособности системы, включая корректность регистрации и аутентификации, совместного редактирования досок в реальном времени, работы AI-ассистента и сохранения загружаемых файлов в объектном хранилище. Такой подход позволяет сохранить логичность разработки, упростить документирование и показать завершенность проекта, при этом разбиение на задачи представлено на рисунке 1.1.

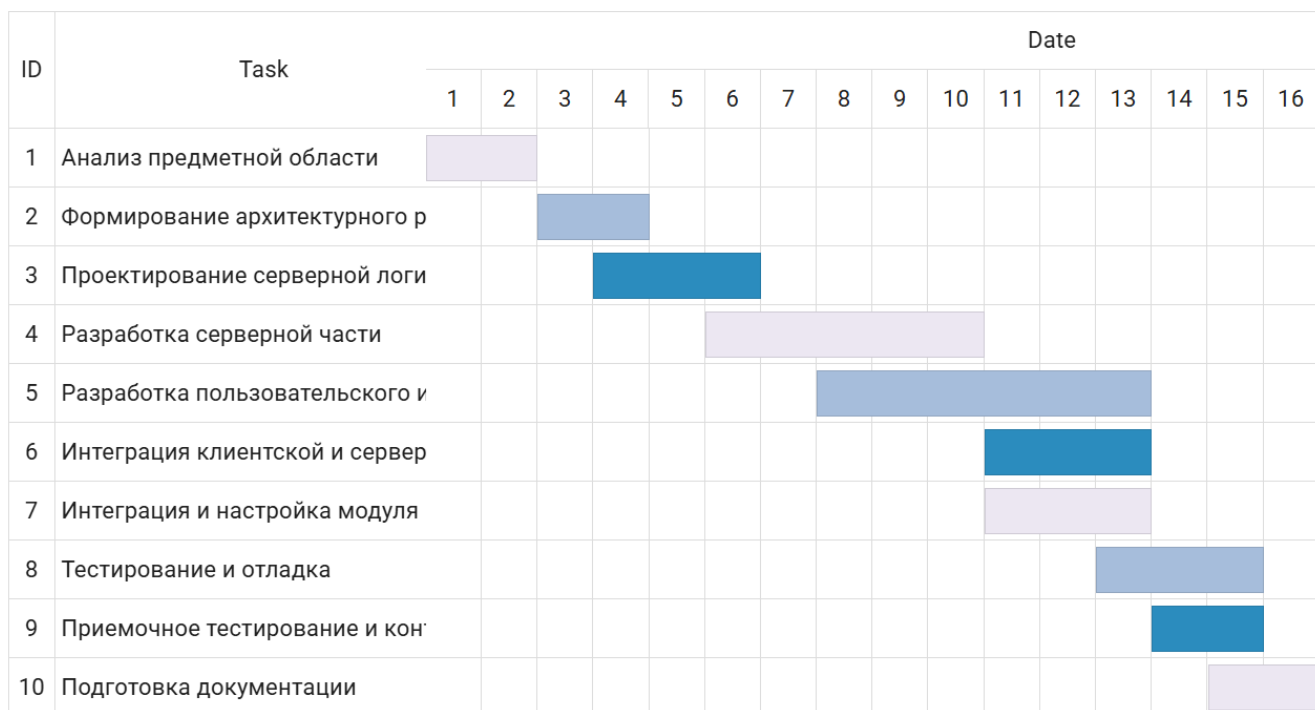


Рисунок 1.1 – Диаграмма Ганта

1.5 Требования к UI/UX и эргономике

Пользовательский интерфейс разрабатываемой системы должен быть понятным, логичным и удобным для повседневного использования. Поскольку основными действиями пользователя являются регистрация и вход в систему, создание рабочих пространств и интерактивных досок, использование инструментов холста (карандаша, ластика, выделения, текста, стикеров, графических фигур, соединительных стрелок в виде кривых Безье и изображений), совместное редактирование в режиме реального времени и взаимодействие с AI-ассистентом, интерфейс должен обеспечивать быстрое выполнение этих операций без лишних переходов и перегрузки экрана элементами управления. Особое значение имеет визуальная ясность: пользователь должен сразу понимать, где находятся основные функции, как создать новую доску, выбрать нужный инструмент и обратиться к AI-ассистенту.

С точки зрения эргономики в интерфейсе должны соблюдаться базовые принципы удобства взаимодействия: единообразие оформления страниц, читаемость текста, логичное расположение кнопок и полей ввода, понятные подписи и иконки инструментов холста, а также минимизация вероятности

пользовательских ошибок. Для этого формы регистрации, авторизации и создания рабочих пространств должны содержать только необходимые поля, а при неправильном вводе система должна выводить понятные сообщения об ошибках. Дополнительно интерфейс должен обеспечивать обратную связь с пользователем, например, через индикатор подключения к серверу совместного редактирования, отображение активных участников доски и положения их курсоров, а также сообщения об успешной отправке одноразового кода, сохранении изменений или получении ответа AI-ассистента. Такой подход соответствует общим принципам эргономического проектирования интерфейсов, применяемым в серии стандартов ISO 9241.

Для повышения удобства использования в системе целесообразно предусмотреть ограниченный набор пользовательских настроек. Например, можно добавить выбор цвета для стикеров, фигур и линий, регулировку масштаба холста, переключение между светлым и темным оформлением, а также средство отмены и повтора последних действий. Также важным требованием является адаптивность интерфейса, поскольку в работе предусмотрены варианты отображения навигации и боковой панели инструментов как для веб-версии, так и для устройств с меньшим разрешением экрана. Это делает систему более доступной и удобной для разных категорий пользователей.

В качестве ключевых экранов системы выступают: главная страница (рисунок 1.2), страница авторизации (рисунок 1.3), страница рабочих пространств (рисунок 1.4), страница интерактивной доски (рисунок 1.5). Именно эти экраны реализуют основные сценарии взаимодействия пользователя с системой.

Место для креативности и сотрудничества

Интерактивная онлайн-доска для совместной работы, где команды творят, планируют и воплощают идеи в жизнь.

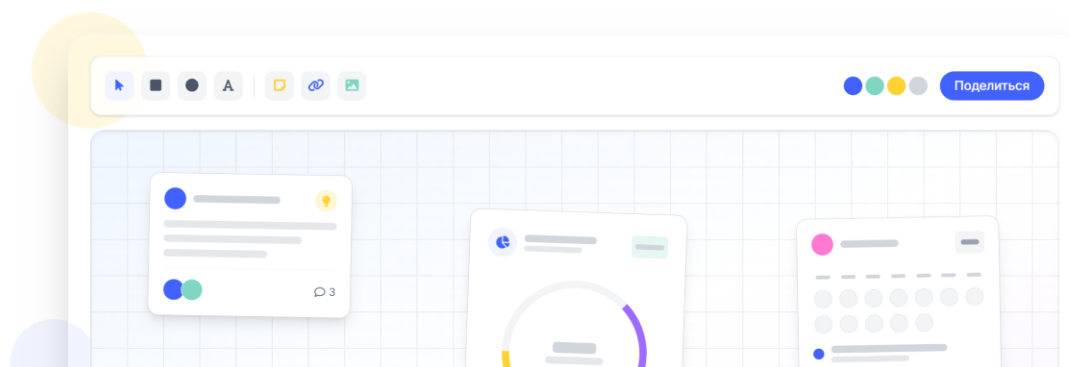
[Начать бесплатно →](#)[Узнать больше](#)

Рисунок 1.2 – Главная страница системы

Введите почту для входа или регистрации

EMAIL АДРЕС

name@company.com

[Продолжить](#)[Google](#)[GitHub](#)

Продолжая, вы соглашаетесь с [Условиями](#) и [Политикой](#)

Рисунок 1.3 – Форма авторизации

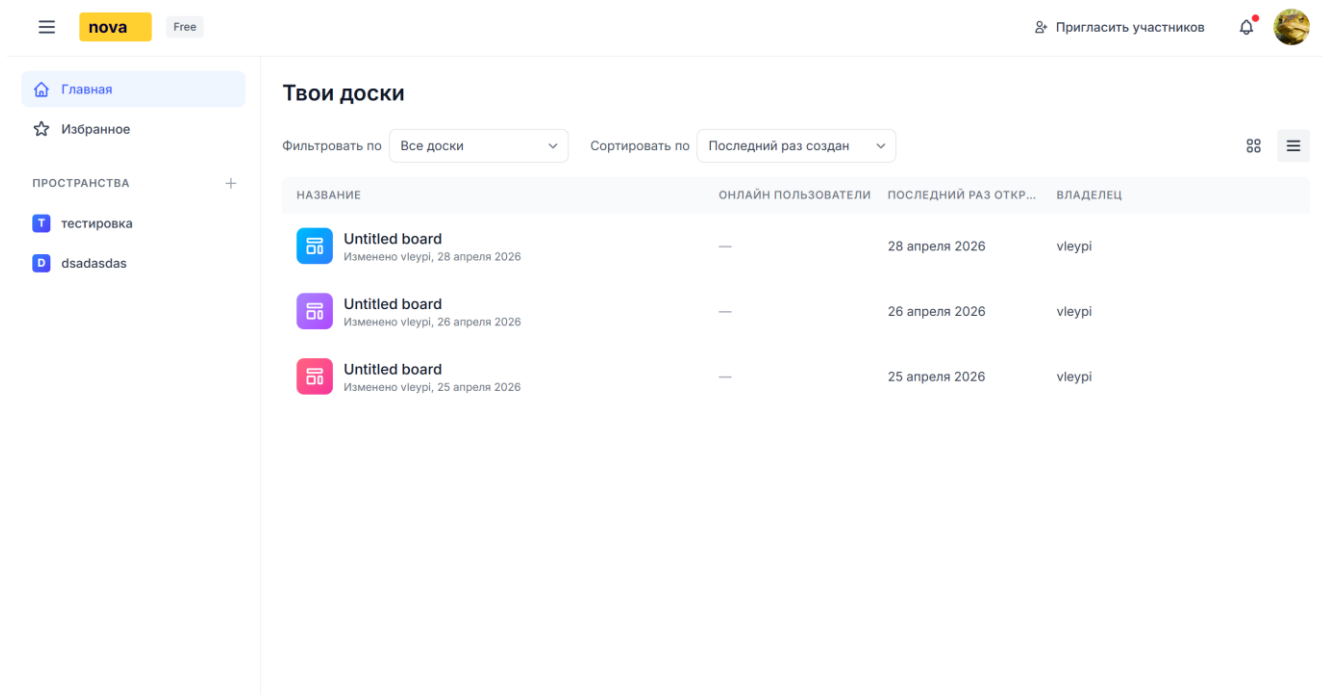


Рисунок 1.4 – Страница рабочих пространств

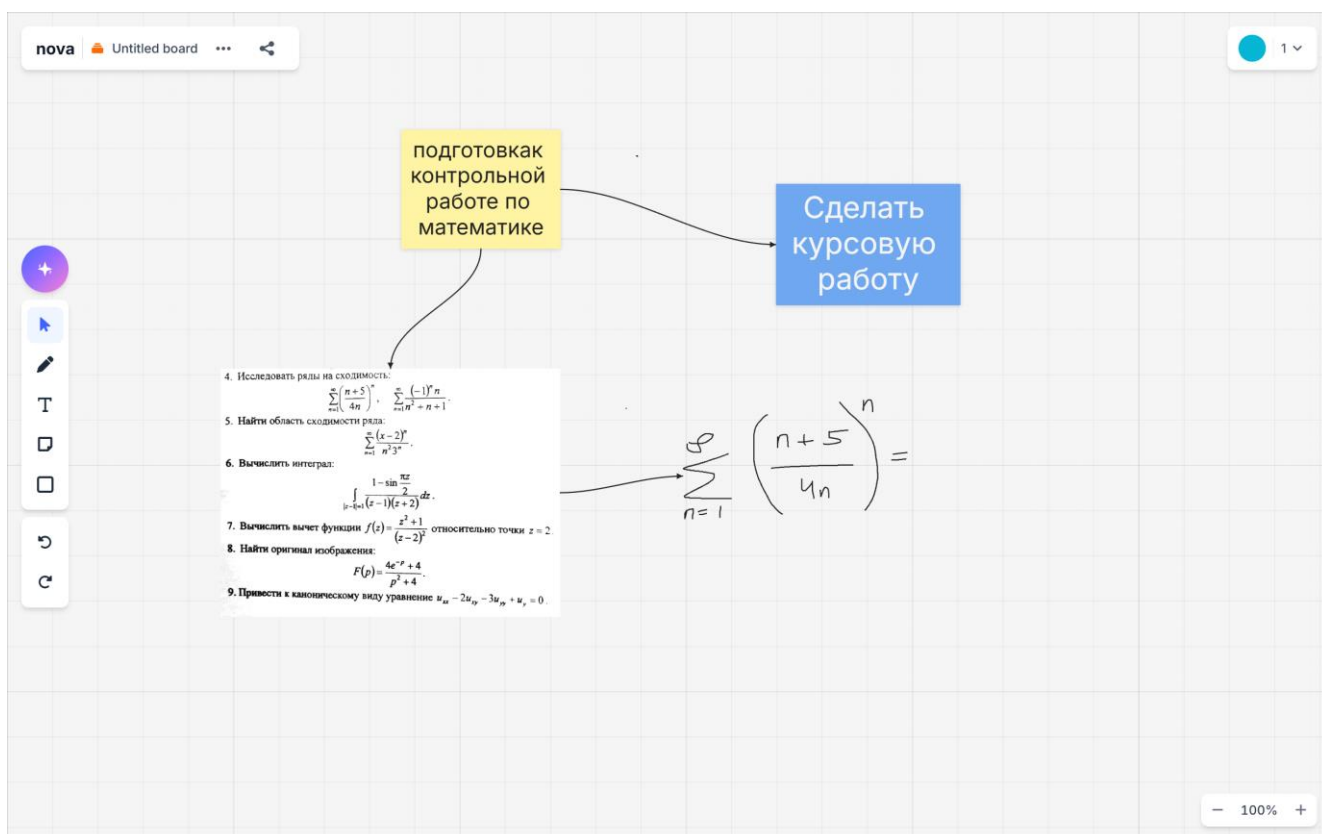


Рисунок 1.5 – Страница интерактивной доски

1.6 Архитектура и технологические требования

Разрабатываемая система построена на основе разделенной клиент-серверной архитектуры с микросервисной организацией серверной части, что

позволяет четко разделить пользовательский интерфейс от серверной логики, обработки данных, работы с базой данных, совместного редактирования в реальном времени и модуля искусственного интеллекта. Клиентская часть отвечает за отображение страниц, ввод данных пользователем, работу собственного движка отрисовки холста и представление результатов, тогда как серверная часть состоит из единой точки входа в виде шлюза API (API Gateway), а также независимых микросервисов работы с пользователями, аутентификации и социального входа, управления досками и рабочими пространствами и совместного редактирования. В качестве механизма обмена данными между клиентом и шлюзом API в проекте используется REST API поверх протокола HTTP, между клиентом и сервисом совместного редактирования – протокол WebSocket с использованием библиотеки Socket.IO, а между шлюзом API и внутренними микросервисами – бинарный протокол удаленного вызова процедур gRPC поверх Protobuf-схем. Подобное сочетание протоколов позволяет организовать взаимодействие компонентов системы по стандартным веб-технологиям, разграничить ответственность между внешним и внутренним контурами, обеспечить низкую задержку при обмене данными между микросервисами и упростить расширение и сопровождение приложения.

К технологическим требованиям проекта относятся надежность, безопасность, устойчивость работы и приемлемое время отклика. Система должна корректно обрабатывать пользовательские запросы, обеспечивать хранение основных данных в реляционной СУБД PostgreSQL, кэшировать сессионную информацию и состояние совместного редактирования в Redis, а также сохранять загружаемые пользователем файлы в объектном хранилище MinIO. Механизмы аутентификации должны быть реализованы на основе подписанных токенов JWT и одноразовых кодов, отправляемых на электронную почту, с поддержкой социального входа через провайдеров Google и GitHub по стандарту OAuth 2.0. Важным требованием является разграничение прав доступа на двух уровнях: на уровне глобальных ролей системы (обычный пользователь, менеджер, администратор, супер-администратор) и на уровне ролей в пределах конкретного

рабочего пространства (владелец и участник), поскольку часть функций, в том числе изменение ролей, блокировка учетных записей и удаление чужих досок, доступна только пользователям с расширенными правами. Кроме того, система должна обеспечивать корректное сохранение содержимого досок, истории операций совместного редактирования и журнала действий администраторов без нарушения целостности данных. Для повышения контролируемости эксплуатации целесообразно использовать логирование основных действий системы, например, операций аутентификации, обращений к AI-ассистенту и административных изменений.

1.7 Функциональные требования

Функциональные требования к системе определяются основными бизнес-процессами, реализуемыми в рамках веб-приложения для совместной работы с интерактивными досками. Система должна поддерживать регистрацию новых пользователей по электронной почте с подтверждением одноразовым кодом и социальный вход через провайдеров Google и GitHub, авторизацию существующих пользователей, разграничение доступа по ролям, создание рабочих пространств и приглашение в них участников, создание и редактирование интерактивных досок, совместное редактирование досок в режиме реального времени, использование инструментов холста (карандаша, ластика, выделения, текста, стикеров, графических фигур, соединительных стрелок в виде кривых Безье и изображений), а также взаимодействие с AI-ассистентом для автоматического построения визуальных элементов на холсте. Для администраторов дополнительно должны быть доступны функции просмотра общей аналитики и активности пользователей, изменения ролей, блокировки и удаления учетных записей, удаления любых досок и рабочих пространств без проверки владельца, принудительного отключения пользователей от сервиса совместного редактирования, просмотра журнала действий, а для супер-администратора – функция рассылки системных сообщений всем пользователям, находящимся в сети. Все указанные действия должны быть

доступны через пользовательский интерфейс и выполняться с опорой на серверную логику приложения.

Система должна обеспечивать выполнение базовых операций управления данными. К ним относятся добавление, просмотр, изменение и удаление записей, связанных с пользователями, рабочими пространствами, досками и размещенными на них элементами, то есть поддержка CRUD-операций в соответствующих модулях. Также должны быть предусмотрены механизмы валидации вводимых данных, в частности проверка корректности электронной почты, длины и сложности пароля, уникальности пользователя при регистрации, а также корректности параметров запроса при работе с досками. При ошибочных действиях пользователя система должна выдавать понятные уведомления и не допускать нарушения целостности данных.

В качестве сквозных бизнес-процессов в системе можно выделить как минимум три основных сценария. Первый сценарий – регистрация и последующая авторизация пользователя с переходом к работе в системе, включая социальный вход и выпуск подписанного токена JWT. Второй сценарий – совместная работа с доской, включающая создание рабочего пространства, приглашение участников, создание доски, редактирование ее содержимого несколькими пользователями одновременно и взаимодействие с AI-ассистентом для автоматической генерации текстовых блоков и связей между ними. Третий сценарий – административное сопровождение системы, включающее просмотр аналитики, изменение ролей пользователей, блокировку учетных записей, удаление сторонних досок и рабочих пространств, а также ведение журнала действий. Каждый из этих процессов должен выполняться как через пользовательский интерфейс, так и через внутренние программные механизмы обработки запросов на сервере.

1.8 Функциональные требования к модулю искусственного интеллекта

Модуль искусственного интеллекта в разрабатываемой системе предназначен для автоматического построения визуальных элементов на интерактивной доске по текстовому запросу пользователя. По своему типу это

генеративный модуль на основе большой языковой модели DeepSeek, доступной через совместимый с OpenAI программный интерфейс с поддержкой вызова инструментов. Входными данными для модуля выступают текстовое сообщение пользователя из чата конкретной доски, а также служебный контекст: идентификатор и текущее состояние доски, перечень уже существующих элементов и история предыдущих сообщений. В проекте предусмотрено, что используемая модель и адрес ее программного интерфейса задаются параметрами `AI_MODEL` и `AI_BASE_URL`, а перечень доступных инструментов жестко описан в коде сервиса и не может быть расширен пользователем без изменения серверной части. Это означает, что модуль ИИ должен быть параметризуемым на уровне конфигурации развертывания и управляемым со стороны разработчиков системы.

Система должна обеспечивать полный цикл работы с модулем: прием сообщения пользователя, формирование контекста на основе текущего состояния доски, обращение к языковой модели по протоколу потоковой передачи, последовательный разбор полученных вызовов инструментов и их выполнение в виде операций над доской. Модуль должен поддерживать три режима работы: режим работы с холстом, при котором модель вызывает инструменты для создания и изменения элементов, режим обычного текстового ответа без вызова инструментов и смешанный режим, сочетающий короткое текстовое сообщение и набор вызовов инструментов. Перечень инструментов включает создание текстового элемента, создание стикера, создание графической фигуры, построение соединительной стрелки в виде кривой Безье между двумя существующими элементами с выбором сторон присоединения, а также изменение, перемещение и удаление существующих элементов. Обязательным требованием является обработка ошибок выполнения инструментов и логирование неудачных вызовов, чтобы можно было оценивать корректность работы модели на конкретных пользовательских запросах.

С точки зрения места модуля ИИ в общем бизнес-процессе он располагается в центре сценария совместной работы с доской. Пользователь формулирует запрос в чате доски, серверная часть приложения принимает запрос, формирует контекст

и обращается к языковой модели, получает последовательность вызовов инструментов, выполняет их над состоянием доски и публикует возникающие при этом события всем участникам через сервис совместного редактирования. В результате новые элементы, построенные AI-ассистентом, отображаются в режиме реального времени у всех пользователей, открывших ту же доску. Таким образом, модуль искусственного интеллекта является не изолированным компонентом, а встроенной частью общей логики работы системы, непосредственно влияющей на ключевую ценность разрабатываемого программного продукта.

1.9 Требования к данным и базе данных

База данных системы должна обеспечивать хранение учетных записей пользователей, рабочих пространств, интерактивных досок и размещенных на них элементов холста, сведений о загруженных файлах изображений, истории действий администраторов, а также служебной информации, необходимой для сопровождения системы. Основным средством хранения выступает реляционная СУБД PostgreSQL, в которой данные физически разделены по двум базам в соответствии с микросервисной архитектурой: одна база обслуживает сервис пользователей и журнал действий, вторая – сервис досок, рабочих пространств, элементов холста и загруженных файлов. Сессионные и оперативные данные, в том числе одноразовые коды подтверждения и состояние совместного редактирования, хранятся в Redis, а сами файлы изображений – в объектном хранилище MinIO.

С точки зрения логической структуры базы данных целесообразно выделить основные сущности: пользователь, рабочее пространство, член рабочего пространства, доска, элемент холста, избранная доска, загруженный файл и запись журнала действий. Кроме того, в схеме используются перечисляемые типы (enum), хранящиеся в качестве справочных значений непосредственно в столбцах соответствующих сущностей: роль пользователя в системе, роль пользователя в пределах рабочего пространства, тип элемента холста и статус загруженного файла. Связи между сущностями организованы по ссылочным идентификаторам и

обеспечивают целостность данных при удалении доски, рабочего пространства или учетной записи пользователя.

Для пользовательских данных обязательными являются адрес электронной почты и ссылка на роль пользователя, при этом пароль в системе не хранится, поскольку аутентификация выполняется через одноразовый код или внешнего провайдера OAuth. Для элементов холста обязательными являются ссылка на доску, тип элемента, координаты и размеры, а также описание содержимого в формате JSON, что позволяет единообразно хранить разнородные типы визуальных элементов. Для загружаемых файлов обязательными являются ключ объекта в объектном хранилище, MIME-тип, размер и контрольная сумма, обеспечивающая целостность данных. Логическая модель базы данных представлена на рисунке 1.6.

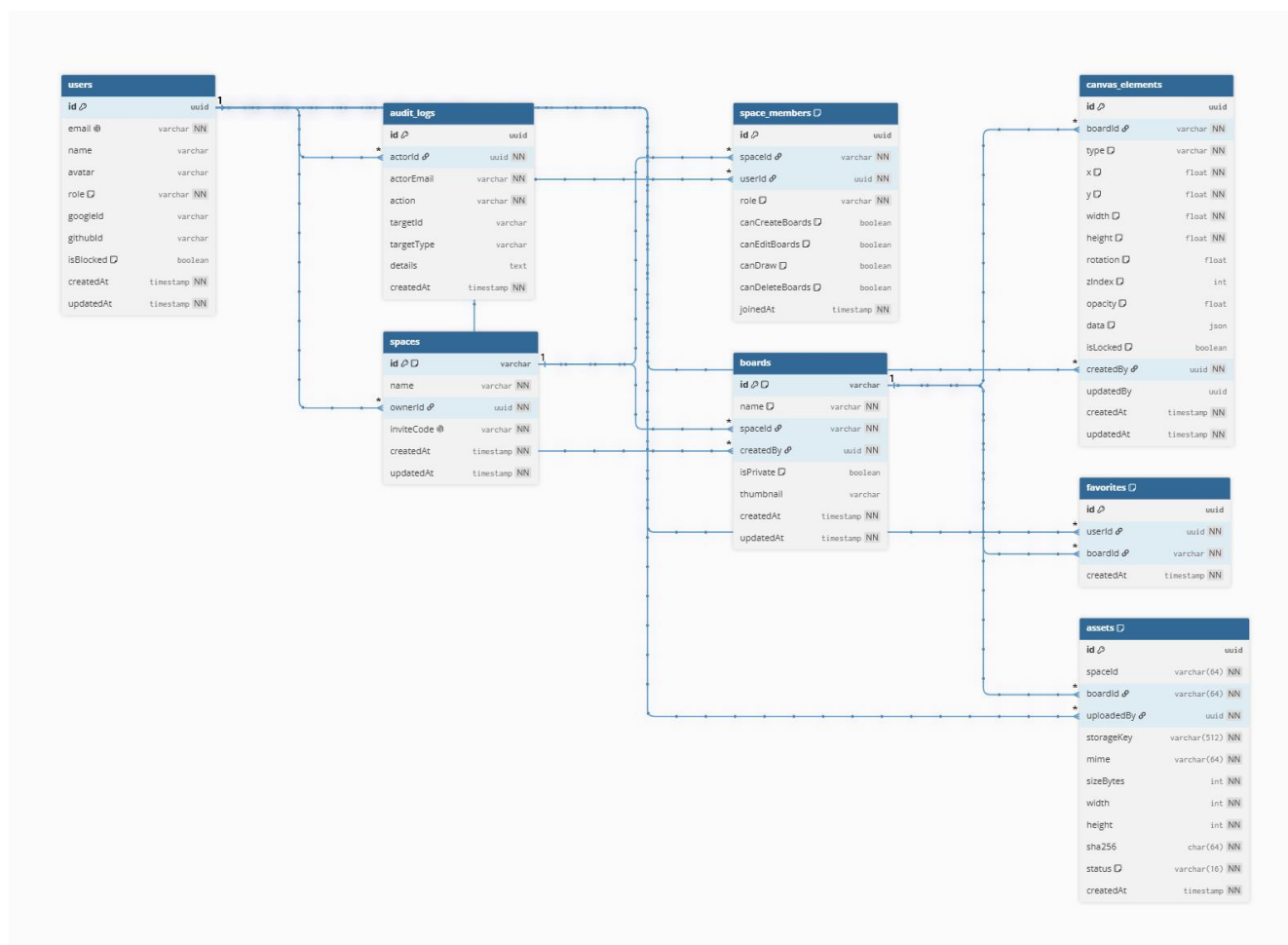


Рисунок 1.6 – ER-диаграмма базы данных

На рисунке 1.6:

1. **Users** – хранит учетные записи пользователей системы.

Содержит идентификатор пользователя, адрес электронной почты, имя, ссылку на аватар, роль в системе, идентификаторы внешних провайдеров Google и GitHub при наличии социального входа, признак блокировки учетной записи, а также даты создания и последнего обновления записи.

2. **Spaces** – хранит рабочие пространства пользователей.

Содержит идентификатор пространства, его название, ссылку на пользователя-владельца, уникальный код приглашения, по которому новые участники присоединяются к пространству, а также даты создания и последнего обновления записи.

3. **Space_members** – хранит сведения о членстве пользователей в рабочих пространствах.

Содержит ссылку на пространство, ссылку на пользователя, роль пользователя в пределах данного пространства (владелец или участник), флаги конкретных прав на создание досок, редактирование досок, рисование и удаление досок, а также дату присоединения.

4. **Boards** – хранит интерактивные доски, созданные пользователями.

Содержит идентификатор доски, ее название, ссылку на родительское рабочее пространство, ссылку на пользователя-создателя, признак закрытости доски, ссылку на миниатюру, а также даты создания и последнего обновления записи.

5. **Canvas_elements** – хранит элементы, размещенные на досках.

Содержит идентификатор элемента, ссылку на доску, тип элемента (текстовый блок, стикер, графическая фигура, соединительная стрелка в виде кривой Безье, штрих свободного рисования или изображение), координаты, ширину, высоту, угол поворота, порядок в стопке (z-индекс), прозрачность, описание содержимого в формате JSON, признак блокировки, ссылки на пользователей, создавших и последними обновивших элемент, а также даты создания и последнего обновления записи.

6. **Favorites** – хранит сведения об избранных досках пользователей.

Содержит ссылку на пользователя, ссылку на доску и дату добавления записи. Эта

сущность обеспечивает реализацию связи «многие ко многим» между пользователями и досками.

7. **Assets** – хранит сведения о файлах изображений, загружаемых на доски. Содержит ссылки на рабочее пространство и доску, ссылку на пользователя, выполнившего загрузку, ключ объекта в хранилище MinIO, MIME-тип, размер в байтах, ширину и высоту изображения, контрольную сумму SHA-256, статус обработки файла и дату создания записи.

8. **Audit_logs** – хранит журнал действий администраторов и системы. Содержит идентификатор записи, ссылку на пользователя-инициатора действия, его адрес электронной почты на момент действия, тип выполненного действия, тип и идентификатор сущности, над которой выполнялось действие, дополнительные детали в текстовом виде и дату совершения действия.

1.10 Требования к проекту по стандартам

Система должна:

- обеспечивать создание, просмотр, изменение и удаление пользовательских данных, рабочих пространств, интерактивных досок и размещенных на них элементов холста с автоматической проверкой корректности формата, типа и обязательности заполнения полей, в том числе валидацией адреса электронной почты, длины и сложности пароля и уникальности учетной записи, в соответствии с требованиями к качеству и корректности функционирования программных средств, установленными ГОСТ 19.301-79;

- обеспечивать хранение данных в структурированной форме в реляционной СУБД PostgreSQL, кэширование сессионной информации и состояния совместного редактирования в Redis, а также хранение файлов изображений в объектном хранилище MinIO, включая сведения об учетных записях, рабочих пространствах, досках, элементах холста, загруженных файлах и журнале действий, с учетом требований к составу и организации информационного обеспечения, установленных ГОСТ 34.201-89;

– обеспечивать разграничение прав доступа пользователей на двух уровнях: на уровне глобальных ролей системы (обычный пользователь, менеджер, администратор, супер-администратор) и на уровне ролей в пределах конкретного рабочего пространства (владелец или участник), при этом функции изменения ролей, блокировки учетных записей, удаления чужих досок и пространств, а также рассылки системных сообщений должны быть доступны только уполномоченным лицам в соответствии с требованиями управления доступом по ГОСТ Р ИСО/МЭК 27001-2018;

– вести журнал действий администраторов и системы, фиксируя вход в систему, изменение ролей, блокировку и удаление учетных записей, удаление досок и рабочих пространств, принудительное отключение пользователей от сервиса совместного редактирования, а также рассылку системных сообщений, для целей аудита и контроля безопасности в соответствии с требованиями ГОСТ Р ИСО/МЭК 27001-2018;

– при возникновении ошибок ввода, аутентификации, обращения к доске или выполнения операций AI-ассистента информировать пользователя понятными и недвусмысленными сообщениями, не раскрывающими служебную информацию, и сохранять устойчивость работы приложения в соответствии с требованиями к программной документации и корректности функционирования по ГОСТ 19.301-79;

– иметь удобный и эргономичный пользовательский интерфейс, обеспечивающий понятную навигацию, доступность основных функций системы, наглядное отображение состояния доски и активных участников совместного редактирования, а также удобство работы как в веб-версии, так и на устройствах с меньшим разрешением экрана, в соответствии с принципами эргономики, установленными стандартом ISO 9241;

– функционировать устойчиво при кратковременных разрывах соединения с сервером совместного редактирования и обеспечивать сохранение состояния доски с последующим автоматическим восстановлением подключения, а также

сохранение содержимого досок и загруженных файлов, в соответствии с требованиями к автоматизированным системам, установленными ГОСТ 34.602-89;

- обрабатывать запросы пользователя, связанные с аутентификацией, созданием и редактированием досок, совместным редактированием в реальном времени и взаимодействием с AI-ассистентом, в пределах допустимого времени отклика, обеспечивающего нормальную эксплуатацию системы, согласно требованиям ГОСТ 34.602-89;

- поддерживать интеграцию и совместимость клиентской и серверной частей приложения на основе клиент-серверной архитектуры с микросервисной организацией серверной части, обеспечивая корректный обмен данными между интерфейсом, шлюзом API, внутренними микросервисами по протоколу gRPC и сервисом совместного редактирования по протоколу WebSocket, в соответствии с требованиями ГОСТ 34.601-90;

- обеспечивать защиту конфиденциальной информации, включая учетные данные пользователей, одноразовые коды подтверждения и подписанные токены доступа, с использованием механизмов аутентификации по протоколу OAuth 2.0 и одноразовому коду, авторизации, выпуска подписанных токенов JWT, а также ограничения доступа к административным функциям, в соответствии с требованиями ГОСТ Р ИСО/МЭК 27001-2018.

1.11 Тестирование (требования)

Тестирование разрабатываемой системы должно подтверждать корректность работы основных функций, реализованных в клиентской и серверной частях приложения. Для данной системы обязательной является проверка пользовательских сценариев, связанных с регистрацией по электронной почте с подтверждением одноразовым кодом и социальным входом через Google и GitHub, авторизацией существующих пользователей, разграничением прав доступа, созданием рабочих пространств и интерактивных досок, совместным редактированием досок в режиме реального времени, взаимодействием с AI-ассистентом и выполнением административных операций. Поскольку в проекте

предусмотрены отдельные роли пользователей, микросервисная организация серверной части, а также механизмы взаимодействия с базой данных, объектным хранилищем и модулем искусственного интеллекта, тестирование должно охватывать не только интерфейсную часть, но и корректность бизнес-логики.

В рамках проверки работоспособности системы целесообразно выделить несколько основных видов тестирования. Во-первых, это функциональное тестирование, в ходе которого проверяется выполнение ключевых операций: регистрация нового пользователя и подтверждение электронной почты одноразовым кодом, вход в систему по протоколу OAuth 2.0, создание рабочего пространства и приглашение участников по коду, создание доски, размещение и редактирование элементов холста, а также взаимодействие с AI-ассистентом и автоматическое построение текстовых блоков, стикеров и соединительных стрелок в виде кривых Безье. Во-вторых, это тестирование прав доступа, при котором проверяется, что обычный пользователь не может получить доступ к административным функциям, связанным с изменением ролей, блокировкой учетных записей, удалением чужих досок и пространств, а участник рабочего пространства не может изменять параметры пространства, доступные только владельцу. В-третьих, необходимо тестирование корректности обработки ошибок, когда система должна выдавать понятные сообщения при неправильном вводе данных, неверном одноразовом коде, неудачной авторизации через внешнего провайдера или попытке обращения к закрытой доске. Дополнительно следует предусмотреть интеграционное тестирование, подтверждающее корректное взаимодействие клиентской части, шлюза API, внутренних микросервисов по протоколу gRPC, сервиса совместного редактирования по протоколу WebSocket и модуля искусственного интеллекта.

Критерием того, что тест считается «пройдено», является полное совпадение фактического результата с ожидаемым. Например, при корректной регистрации в системе должен создаваться новый пользователь и выпускаться подписанный токен JWT; при правильном вводе одноразового кода либо корректном прохождении сценария OAuth пользователь должен успешно входить в систему;

при создании или изменении элемента доски одним пользователем соответствующее изменение должно мгновенно отображаться у всех остальных участников, открывших ту же доску; при запросе к AI-ассистенту должны корректно вызываться предусмотренные инструменты и создаваться соответствующие элементы холста. Для тестов разграничения доступа успешным считается результат, при котором запрещенные конечные точки возвращают ответ с кодом отказа в доступе для пользователей без соответствующей роли. Для тестов устойчивости и обработки ошибок система считается прошедшей проверку, если при кратковременном разрыве соединения с сервером совместного редактирования она автоматически восстанавливает подключение и состояние доски, а при возникновении некорректных ситуаций сохраняет работоспособность и выводит пользователю понятное сообщение об ошибке. Таким образом, успешным тестированием считается подтверждение корректной работы всех обязательных функций системы и их соответствия ранее сформулированным требованиям.

1.12 Документация и ввод в эксплуатацию (требования)

Для разработанной системы должен быть подготовлен комплект документации, необходимый для ее использования, сопровождения и первичного развертывания. С учетом особенностей проекта в состав документации целесообразно включить как минимум руководство пользователя, руководство администратора, краткое описание REST API и инструкцию по запуску системы. Это обусловлено тем, что в приложении реализованы различные пользовательские роли, предусмотрены функции совместной работы с интерактивными досками, взаимодействия с AI-ассистентом и административного управления, а также используется клиент-серверная архитектура с микросервисной организацией серверной части. Кроме того, в задании на курсовую работу отдельно указана необходимость подготовки инструкции пользователя, что делает этот раздел обязательным.

Руководство пользователя должно содержать описание основных действий обычного пользователя: порядок регистрации по электронной почте с

подтверждением одноразовым кодом и социального входа через Google и GitHub, авторизацию, создание рабочих пространств и приглашение участников по коду, создание интерактивных досок, использование инструментов холста и обращение к AI-ассистенту для автоматического построения визуальных элементов. Руководство администратора должно включать описание административных функций: просмотр общей аналитики и активности пользователей, изменение ролей, блокировку и удаление учетных записей, удаление любых досок и пространств, принудительное отключение пользователей от сервиса совместного редактирования, просмотр журнала действий, а для супер-администратора – рассылку системных сообщений. Описание API должно содержать перечень основных конечных точек шлюза, назначение запросов, параметры передачи данных и ожидаемые результаты обработки. Инструкция по запуску должна включать порядок установки Docker Compose, настройки переменных окружения, подготовки баз данных PostgreSQL, запуска Redis и объектного хранилища MinIO, последовательного запуска микросервисов и шлюза API, а также проверки работоспособности после развертывания.

С точки зрения ввода в эксплуатацию система должна быть подготовлена к установке в рабочую среду с возможностью подключения к базе данных, использования механизмов безопасности и сохранения загружаемых файлов в объектном хранилище. В процессе ввода в эксплуатацию должны быть выполнены настройка параметров подключения, создание учетной записи первого пользователя, которому система автоматически назначает роль супер-администратора, проверка регистрации и совместного редактирования доски с участием нескольких пользователей, а также тестовое обращение к AI-ассистенту. Система может считаться подготовленной к эксплуатации в том случае, если она успешно запускается, принимает пользовательские запросы, обеспечивает совместное редактирование досок в реальном времени, корректно обрабатывает ошибки и обеспечивает разграничение доступа в соответствии с ролями.

1.13 Оценивание (критерии успешности)

Критерии успешности проекта определяются степенью соответствия разработанной системы целям и требованиям, сформулированным в техническом задании и пояснительной записке. В первую очередь успешным считается такой результат, при котором реализованы все обязательные функции системы: регистрация и авторизация пользователей по электронной почте и через провайдеров Google и GitHub, разграничение прав доступа, создание рабочих пространств и интерактивных досок, совместное редактирование досок в режиме реального времени с использованием всех инструментов холста, взаимодействие с AI-ассистентом и автоматическое построение визуальных элементов на холсте, а также административные функции, связанные с управлением пользователями, досками и журналом действий. Кроме того, система должна быть подготовлена к демонстрации на защите и сопровождаться необходимой документацией.

Оценивание должно учитывать не только факт наличия программного кода, но и полноту реализации проектных решений. К числу основных критериев относятся: соответствие разработанной микросервисной архитектуры поставленной задаче, корректная работа клиентской и серверной частей, наличие реляционной базы данных и продуманной структуры хранения данных, использование Redis для оперативных данных и MinIO для хранения файлов, реализация механизмов безопасности на основе токенов JWT, одноразовых кодов и провайдеров OAuth, использование модуля искусственного интеллекта на основе DeepSeek с поддержкой вызова инструментов для построения элементов на холсте, наличие тестирования и оформление результатов разработки в соответствии с требованиями курсовой работы. Отдельно оценивается качество пользовательского интерфейса, логичность навигации, удобство выполнения основных действий и наглядность совместного редактирования.

Итого проект можно считать успешным, если выполнены обязательные пункты технического задания, система работает без критических ошибок, демонстрирует заявленные функции в ходе защиты и позволяет показать завершённый пользовательский сценарий от входа в систему до совместного редактирования доски с участием AI-ассистента. При этом каждый пропущенный

обязательный пункт, например, отсутствие части функций, документации, тестирования или демонстрации работоспособности, снижает итоговую оценку проекта. Следовательно, критерием успешности является не только наличие реализованного программного продукта, но и его полнота, устойчивость, документированность и готовность к практической демонстрации.

2. ПРОЕКТИРОВАНИЕ ИНФОРМАЦИОННОЙ СИСТЕМЫ

2.1 Диаграмма вариантов использования

В целях формализации требований к разрабатываемой системе и наглядного представления взаимодействия пользователей с ее основными функциями целесообразно использовать диаграмму вариантов использования. Такая диаграмма позволяет определить состав ключевых сценариев работы системы, выделить основных акторов и показать распределение доступных им операций. Для системы совместной работы с интерактивными досками это особенно важно, поскольку в ней предусмотрены разные роли пользователей и разграничение прав доступа к отдельным функциям. Диаграмма вариантов использования разрабатываемой системы представлена на рисунке 2.1.

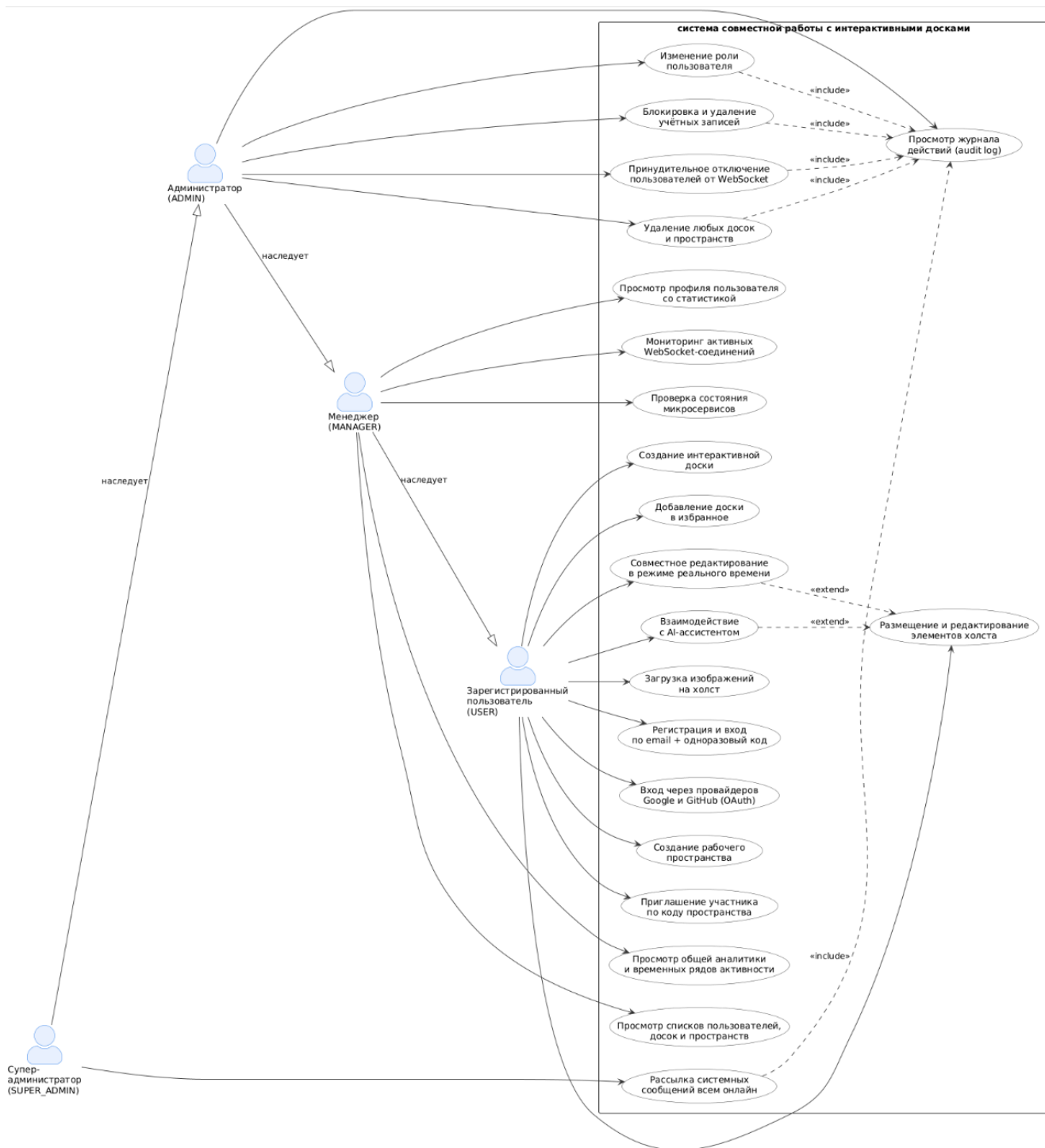


Рисунок 2.1 – Диаграмма вариантов использования

На диаграмме вариантов использования (рисунок 2.1) представлена разрабатываемая система совместной работы с интерактивными досками и основные сценарии взаимодействия пользователей с ее функциональностью. В качестве акторов выделены зарегистрированный пользователь, администратор и супер-администратор. При этом администратор рассматривается как расширенная роль пользователя, а супер-администратор – как расширенная роль

администратора, поэтому помимо базовых пользовательских возможностей соответствующие акторы имеют доступ и к дополнительным функциям. Внутри границ системы показаны основные варианты использования: регистрация и вход в систему по электронной почте с одноразовым кодом, вход через провайдеров OAuth, создание рабочих пространств и приглашение участников по коду, создание и редактирование интерактивных досок, размещение и редактирование элементов холста (текстовых блоков, стикеров, графических фигур, соединительных стрелок в виде кривых Безье, штрихов свободного рисования и изображений), совместное редактирование досок в режиме реального времени, взаимодействие с AI-ассистентом, просмотр аналитики, изменение ролей и блокировка учетных записей, удаление досок и рабочих пространств, принудительное отключение пользователей от сервиса совместного редактирования, просмотр журнала действий и рассылка системных сообщений.

Зарегистрированный пользователь взаимодействует с системой в рамках базового бизнес-процесса: проходит регистрацию и аутентификацию, создает рабочие пространства, приглашает в них участников по коду, создает интерактивные доски и совместно редактирует их с другими участниками, использует инструменты холста и обращается к AI-ассистенту. Администратор, в свою очередь, помимо указанных действий может просматривать общую аналитику и активность пользователей, изменять роли, блокировать и удалять учетные записи, удалять любые доски и рабочие пространства без проверки владельца, принудительно отключать пользователей от сервиса совместного редактирования и просматривать журнал действий. Супер-администратор дополнительно имеет эксклюзивную возможность рассылать системные сообщения всем пользователям, находящимся в сети. Таким образом, диаграмма показывает разграничение прав доступа по ролям и отражает, что часть функций системы доступна всем авторизованным пользователям, а часть – только администраторам соответствующего уровня.

Пунктирные связи на диаграмме отражают логическую зависимость между отдельными вариантами использования. В частности, варианты использования

совместного редактирования доски и взаимодействия с AI-ассистентом расширяют сценарий редактирования доски, поскольку оба они выполняются в контексте уже открытой доски, а изменение ролей и блокировка пользователя включают подвариант записи в журнал действий, что обеспечивает сквозную фиксацию административных операций. В целом диаграмма демонстрирует состав ключевых функций системы и распределение ответственности между ролями пользователей.

2.2 Диаграммы активности

Для более детального описания логики работы системы и последовательности выполнения ключевого бизнес-процесса целесообразно использовать диаграмму активности. В отличие от диаграммы вариантов использования, она позволяет показать не только состав функций, но и порядок выполнения действий, условия переходов между этапами, а также распределение операций между участниками процесса. Для разрабатываемой системы совместной работы с интерактивными досками такая диаграмма особенно важна, поскольку процесс совместного редактирования включает взаимодействие пользователя, клиентского приложения, шлюза API, сервиса совместного редактирования и сервиса досок. Диаграмма активности, отражающая процесс совместного редактирования интерактивной доски, представлена на рисунке 2.2.

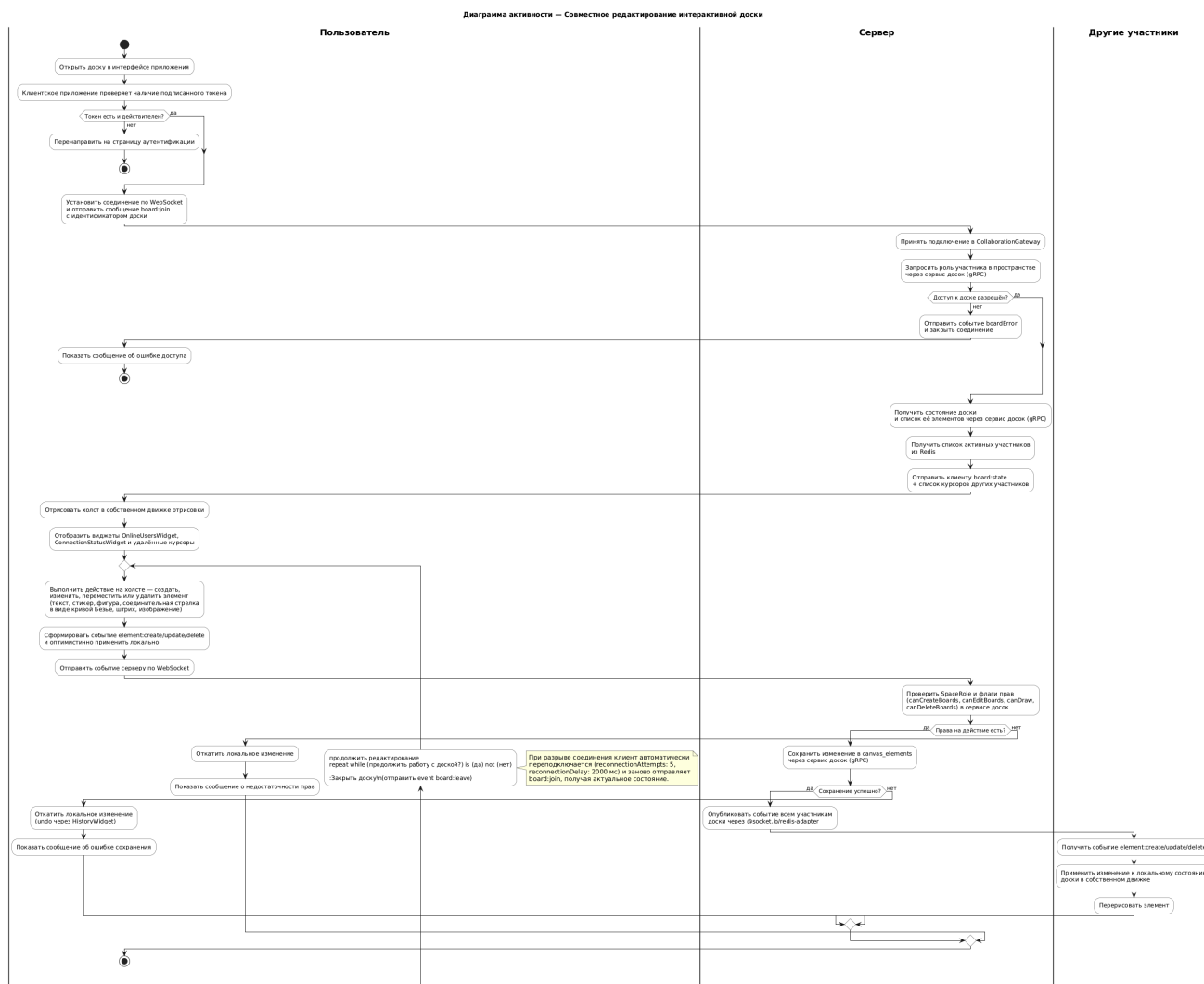


Рисунок 2.2 – Совместное редактирование интерактивной доски

Диаграмма начинается с действий пользователя, который открывает доску в интерфейсе приложения. После этого управление переходит к клиентской части, которая выполняет проверку авторизации и устанавливает соединение с сервисом совместного редактирования по протоколу WebSocket, передавая идентификатор доски в сообщении присоединения. В ответ сервер передает клиенту текущее состояние доски, перечень размещенных на ней элементов и информацию об активных участниках совместного редактирования. Если пользователь не авторизован, выполняется перенаправление на страницу авторизации, а если у него нет прав доступа к данной доске, выводится соответствующее сообщение об ошибке.

После успешного подключения пользователь выполняет действие на холсте: создает текстовый блок, стикер, графическую фигуру, соединительную стрелку в

виде кривой Безье, штрих свободного рисования или загружает изображение, либо изменяет, перемещает и удаляет уже размещенные элементы. Клиентская часть формирует соответствующее событие и отправляет его серверу совместного редактирования. Сервер выполняет проверку прав пользователя в пределах рабочего пространства, после чего сохраняет изменение элемента через сервис досок и публикует событие всем участникам, открывшим ту же доску. В случае ошибки сохранения система откатывает локальное изменение у инициатора и выводит ему сообщение об ошибке.

На завершающем этапе клиентские приложения всех участников применяют полученное событие к своему состоянию холста, после чего новое или измененное содержимое отображается у всех пользователей. При кратковременном разрыве соединения с сервером совместного редактирования клиент автоматически выполняет повторное подключение и синхронизирует свое состояние с актуальным состоянием доски. Таким образом, диаграмма отражает полный цикл совместного редактирования – от открытия доски и установки соединения до согласованного отображения изменений у всех участников – и показывает разграничение действий между клиентом, шлюзом API, сервисом совместного редактирования и сервисом досок.

Для описания логики взаимодействия пользователя с системой на этапе получения доступа к ее функциям целесообразно использовать диаграмму активности, отражающую процесс регистрации и авторизации. Такой вид диаграммы позволяет последовательно показать действия пользователя и системы, а также условия перехода между этапами, связанные с проверкой корректности данных, наличия учетной записи и прав доступа. Для разрабатываемой системы данный процесс является одним из базовых, поскольку именно через него обеспечивается выпуск подписанного токена доступа и разграничение прав между обычным пользователем и администратором. Диаграмма активности процесса регистрации и авторизации пользователя представлена на рисунке 2.3.

Диаграмма активности — Регистрация и авторизация пользователя

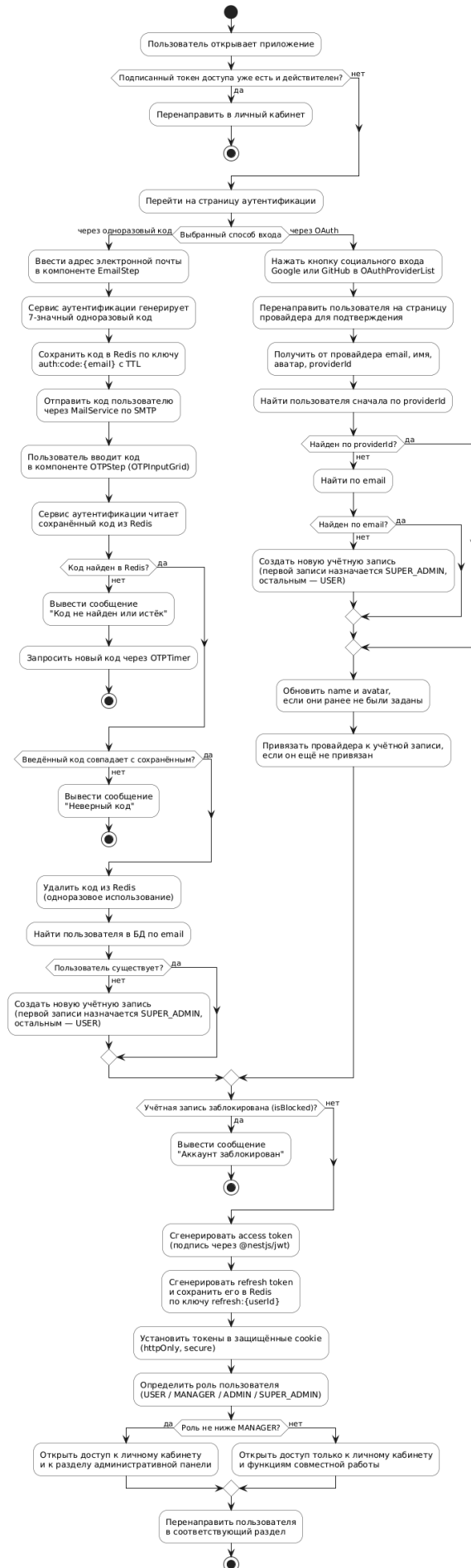


Рисунок 2.3 – Регистрация и авторизация пользователя

Диаграмма начинается с открытия приложения пользователем, после чего система определяет, имеется ли у него учетная запись и выбранный способ входа. Если пользователь выбирает регистрацию или вход по электронной почте, выполняется переход на соответствующую страницу, где он вводит адрес электронной почты. Затем сервис аутентификации проверяет корректность формата адреса и при необходимости создает новую учетную запись, после чего генерирует и отправляет одноразовый код подтверждения на указанную электронную почту. При неправильно введенном адресе или превышении количества попыток пользователю выводится соответствующее сообщение об ошибке.

После получения одноразового кода пользователь вводит его в форме подтверждения, и система выполняет сверку введенного значения с сохраненным в Redis. В случае несовпадения или истечения срока действия кода выводится сообщение об ошибке. Если же код корректен, сервис аутентификации формирует подписанный токен JWT и возвращает его клиенту. При выборе альтернативного способа входа пользователь проходит сценарий OAuth 2.0 через провайдеров Google или GitHub, после чего сервис аутентификации получает профиль пользователя у провайдера, привязывает его к существующей учетной записи либо создает новую и так же выпускает токен доступа. При первой регистрации пользователя в системе ему автоматически назначается роль супер-администратора, остальным – роль обычного пользователя.

При успешной проверке учетных данных клиентская часть сохраняет полученный токен и выполняет последующие запросы к шлюзу API с его использованием. Шлюз API при каждом запросе проверяет подпись и срок действия токена, определяет роль пользователя и предоставляет доступ к соответствующим функциям. Если пользователь имеет административную роль, ему становятся доступны как пользовательские, так и административные возможности системы; в противном случае открываются только стандартные пользовательские функции. Таким образом, диаграмма отражает полный цикл

предоставления доступа к системе: от выбора способа входа до успешной авторизации и разграничения прав в зависимости от роли пользователя.

Для описания внутренней логики работы интеллектуального модуля системы целесообразно использовать диаграмму активности, отражающую процесс взаимодействия пользователя с AI-ассистентом. Такая диаграмма позволяет последовательно показать действия пользователя и системы, а также основные условия принятия решений, связанные с проверкой прав доступа, формированием контекста доски и разбором вызовов инструментов, возвращаемых языковой моделью. Для разрабатываемой системы данный процесс имеет особое значение, поскольку именно от корректности взаимодействия с AI-ассистентом зависит автоматическое построение визуальных элементов на холсте по текстовому запросу. Диаграмма активности процесса взаимодействия с AI-ассистентом представлена на рисунке 2.4.

Диаграмма начинается с того, что пользователь открывает чат AI-ассистента на интерактивной доске и отправляет текстовый запрос. Клиентская часть передает сообщение серверу совместного редактирования вместе с идентификатором доски и идентификатором пользователя. Сервер выполняет проверку прав пользователя на работу с указанной доской и в случае отказа возвращает соответствующее сообщение об ошибке. Если права подтверждены, сервис AI-ассистента получает текущее состояние доски, перечень уже размещенных элементов и историю предыдущих сообщений, после чего формирует контекст запроса.

После подготовки контекста сервис AI-ассистента обращается к большой языковой модели по протоколу потоковой передачи с указанием перечня доступных инструментов. Модель возвращает либо обычное текстовое сообщение, либо последовательность вызовов инструментов, либо их сочетание. Сервис последовательно разбирает полученные вызовы и для каждого инструмента из списка – создания текстового элемента, стикера, графической фигуры, соединительной стрелки в виде кривой Безье, изменения, перемещения или удаления элемента – выполняет соответствующую операцию над состоянием доски через сервис досок. В случае ошибки выполнения инструмента сервис фиксирует неудачный вызов и продолжает обработку оставшихся вызовов.

На завершающем этапе сервер совместного редактирования публикует события об изменениях доски всем участникам, открывшим ее в данный момент, и они отображаются у каждого пользователя в режиме реального времени. Текстовый ответ AI-ассистента, при его наличии, выводится в чате доски у инициатора запроса. Таким образом, диаграмма отражает полный цикл взаимодействия с AI-ассистентом – от ввода текстового запроса и проверки прав до выполнения вызовов инструментов и согласованного отображения построенных AI-ассистентом визуальных элементов у всех участников совместного редактирования.

2.3 Диаграмма процесса обработки данных

Для наглядного представления логики преобразования, проверки и последующей обработки входной информации в системе целесообразно использовать диаграмму активности процесса обработки данных. Такая диаграмма позволяет отразить последовательность действий пользователя и системы, а также показать ключевые точки принятия решений, связанные с проверкой корректности данных, прав доступа и выбором дальнейшего сценария обработки. Для разрабатываемой системы данный процесс является важным, поскольку через него реализуются как операции сохранения содержимого досок и загружаемых файлов, так и взаимодействие с сервисом совместного редактирования и модулем искусственного интеллекта. Диаграмма активности процесса обработки данных представлена на рисунке 2.5.

Рисунок 2.5 – Диаграмма процесса обработки данных

Диаграмма начинается с действий пользователя, который выполняет операцию в клиентском приложении: создает или редактирует элемент на холсте, загружает изображение, изменяет параметры рабочего пространства, отправляет запрос AI-ассистенту или выполняет административное действие. Клиентская часть формирует соответствующий запрос и отправляет его на шлюз API. После этого шлюз принимает запрос, проверяет формат и полноту переданных параметров, а также корректность и срок действия подписанного токена JWT. Если запрос не соответствует установленным требованиям или токен отсутствует или недействителен, пользователю возвращается сообщение об ошибке, и дальнейшая обработка не выполняется.

Если запрос корректен, шлюз API переходит к проверке прав доступа, учитывая как глобальную роль пользователя в системе, так и его роль в пределах конкретного рабочего пространства. При отсутствии необходимых прав пользователю возвращается сообщение о недостаточности прав доступа. В случае успешного прохождения этой проверки шлюз определяет тип запроса и направляет его соответствующему внутреннему микросервису по протоколу gRPC: запросы, связанные с учетными записями и журналом действий, передаются сервису пользователей, запросы, связанные с рабочими пространствами, досками и элементами холста, – сервису досок, а запросы, связанные с загрузкой файлов, обрабатываются через выдачу подписанной ссылки на объектное хранилище MinIO. Если запрос относится к работе с AI-ассистентом, он передается сервису совместного редактирования, который дополнительно формирует контекст доски и обращается к большой языковой модели для разбора пользовательского сообщения.

После выполнения операции выбранный микросервис сохраняет результат в реляционной базе данных PostgreSQL, при необходимости обновляет оперативные данные в Redis, а файлы изображений сохраняются пользователем непосредственно в объектное хранилище MinIO по выданной подписанной ссылке. После успешного завершения операции сервер совместного редактирования

публикует события об изменениях всем участникам, открывшим затронутую доску, а шлюз API возвращает пользователю либо сообщение об успешной обработке запроса, либо сформированный результат для дальнейшего отображения в интерфейсе. Таким образом, диаграмма отражает полный процесс обработки входной информации – от ее формирования на клиенте и проверки на шлюзе API до выполнения операции внутренним микросервисом, сохранения в базе данных или объектном хранилище и согласованного отображения у всех пользователей системы.

2.4 Диаграмма классов

В разработанной диаграмме классов (рисунок 2.6) отражена структура системы совместной работы с интерактивными досками на уровне основных предметных сущностей и программных компонентов. В модели выделены классы учетной записи пользователя с разграничением по системной роли и роли в пределах рабочего пространства, классы рабочего пространства и его участника, классы доски, элемента холста, избранной доски, загружаемого файла и записи журнала действий. Кроме доменной части, на диаграмме показаны контроллеры REST API в составе шлюза, WebSocket-шлюз совместного редактирования, классы передачи данных, сервисный слой в составе отдельных микросервисов и репозитории доступа к данным, что позволяет представить взаимодействие между пользовательскими запросами, бизнес-логикой и механизмами хранения. Также в структуре системы учтено использование шаблонов Strategy и Adapter для разбора вызовов инструментов AI-ассистента и обращения шлюза API к внутренним микросервисам по протоколу gRPC.

Класс «Участник пространства (SpaceMember)» связывает пользователей с пространствами и хранит роль пользователя в пределах данного пространства, флаги конкретных прав на создание досок, их редактирование, рисование и удаление, а также дату присоединения. Между классами User и Space установлена связь типа «один ко многим», поскольку один пользователь может владеть несколькими пространствами, а связь между Space и SpaceMember отражает состав участников в пределах одного пространства.

Класс «Доска (Board)» описывает отдельную интерактивную доску и содержит идентификатор доски, ее название, ссылку на родительское рабочее пространство, ссылку на пользователя-создателя, признак закрытости, ссылку на миниатюру и даты создания и обновления записи. Между классами Space и Board установлена связь «один ко многим», поскольку в одном пространстве может располагаться несколько досок. Класс «Избранное (Favorite)» представляет связь «многие ко многим» между пользователями и досками и фиксирует факт добавления конкретной доски в избранное конкретным пользователем, что соответствует реальной логике работы системы, в которой одна и та же доска может быть отмечена несколькими пользователями.

Один из ключевых классов предметной области – «Элемент холста (CanvasElement)». Он содержит идентификатор элемента, ссылку на доску, тип элемента, координаты, ширину, высоту, угол поворота, порядок в стопке, прозрачность, описание содержимого в формате JSON, признак блокировки, ссылки на пользователей, создавших и последними обновивших элемент, а также даты создания и обновления записи. Между классами Board и CanvasElement установлена связь композиции, поскольку элементы холста существуют только в контексте конкретной доски. Тип элемента задается перечислением «Тип элемента (ElementType)», которое включает значения «text» для текстовых блоков, «sticky» для стикеров, «shape» для графических фигур, «connector» для соединительных стрелок в виде кривых Безье, «stroke» для штрихов свободного рисования и «image» для изображений. Такое решение позволяет хранить разнородные

визуальные элементы в единой таблице и обрабатывать их через единый интерфейс работы с холстом.

Класс «Загруженный файл (Asset)» предназначен для хранения сведений о файлах изображений, размещаемых на досках. В нем содержатся ссылки на рабочее пространство и доску, ссылка на пользователя, выполнившего загрузку, ключ объекта в объектном хранилище, MIME-тип, размер в байтах, ширина и высота изображения, контрольная сумма SHA-256, статус обработки файла и дата создания записи. Статус задается перечислением «Статус файла (AssetStatus)» и позволяет отслеживать жизненный цикл загрузки от создания подписанной ссылки до подтверждения сохранения объекта. Связь между классами Board и Asset показывает, что один и тот же объектный файл привязан к конкретной доске, на которой он используется.

Для обеспечения возможности контроля за административными действиями на диаграмме введен класс «Запись журнала действий (AuditLog)». Он содержит идентификатор записи, ссылку на пользователя-инициатора действия, его адрес электронной почты на момент действия, тип выполненного действия, тип и идентификатор сущности, над которой выполнялось действие, дополнительные детали в текстовом виде и дату совершения. Связь между классами User и AuditLog показывает, что записи журнала формируются по результатам действий администраторов и супер-администратора. Это соответствует логике эксплуатации системы, в которой операции изменения ролей, блокировки, удаления досок и пространств и принудительного отключения пользователей подлежат обязательному протоколированию.

Кроме классов предметной области, на диаграмме представлены перечисления «Системная роль (Role)», «Роль в пространстве (SpaceRole)», «Тип элемента (ElementType)» и «Статус файла (AssetStatus)». Перечисление системных ролей задает допустимые типы учетных записей: обычный пользователь, менеджер, администратор и супер-администратор. Перечисление ролей в пространстве описывает позицию пользователя относительно конкретного рабочего пространства: владелец или участник. Использование перечислений

позволяет формально ограничить набор допустимых значений и сделать модель более строгой и понятной.

Отдельный блок диаграммы образует фрагмент REST API, реализованный в составе шлюза API. Здесь представлены контроллеры аутентификации (AuthController), пользователей (UsersController), досок и рабочих пространств вместе с операциями избранного (BoardsController), загружаемых файлов (AssetsController), а также семейство административных контроллеров: AdminUsersController, AdminBoardsController, AdminSpacesController, AdminAnalyticsController, AdminAuditController, AdminRealtimeController и AdminSystemController. Дополнительно показан WebSocket-шлюз CollaborationGateway, отвечающий за обработку событий совместного редактирования, и WebSocket-шлюз AiGateway, через который пользователи взаимодействуют с AI-ассистентом. Таким образом, контроллеры и шлюзы представляют внешний интерфейс взаимодействия клиента с серверной частью приложения.

Для передачи данных между клиентом и сервером на диаграмме предусмотрены объекты типа DTO. К ним относятся SendAuthCodeDto и VerifyAuthCodeDto для процедуры аутентификации, CreateSpaceDto и UpdateSpaceDto для рабочих пространств, CreateBoardDto, UpdateBoardDto и MoveBoardDto для досок, UpdateMemberPermissionsDto для участников пространства, PresignDto и ConfirmDto для двухэтапной загрузки файлов в объектное хранилище, а также UpdateProfileDto, UpdateRoleDto и BlockUserDto для пользовательских и административных операций. Эти классы содержат только те данные, которые необходимы для передачи в запросе или ответе, а их использование позволяет отделить внутренние объекты предметной области от объектов, применяемых во внешнем API.

Следующий крупный блок диаграммы составляет сервисный слой, распределенный между микросервисами. В сервисе аутентификации выделены классы AuthService и MailService, реализующие отправку одноразовых кодов и обработку сценариев OAuth. В сервисе пользователей выделены классы

UserService и AuditLogService, отвечающие за работу с учетными записями и журналом действий. В сервисе досок выделены классы SpacesService, BoardsService и AssetsService, реализующие основную бизнес-логику работы с пространствами, досками и загружаемыми файлами. В сервисе совместного редактирования выделены классы CollaborationService и AiService, отвечающие соответственно за публикацию событий совместного редактирования и за взаимодействие с большой языковой моделью. Связи между контроллерами и сервисами показывают, что контроллер не реализует сам бизнес-логику, а только передает выполнение соответствующему сервису по протоколу gRPC.

Ниже на диаграмме расположен слой доступа к данным, представленный репозиториями TypeORM. К ним относятся UserRepository, AuditLogRepository, SpaceRepository, SpaceMemberRepository, BoardRepository, CanvasElementRepository, FavoriteRepository и AssetRepository. Репозитории используются для сохранения и получения объектов из реляционной базы данных PostgreSQL и соответствуют отдельным сущностям предметной области. Связи сервисов с репозиториями показывают, что сервисный слой взаимодействует с данными не напрямую, а через отдельные абстракции доступа. Это соответствует многослойной архитектуре и упрощает модификацию системы в дальнейшем.

На диаграмме также отражен архитектурный принцип разделения ответственности между уровнями Controller – Service – Repository, который в работе соотнесен с подходом MVC. Контроллеры принимают запросы и возвращают ответы, сервисы реализуют бизнес-логику, а репозитории обеспечивают доступ к данным. Такое построение делает приложение более структурированным, уменьшает связанность компонентов и облегчает тестирование, развитие и сопровождение системы.

Помимо основной архитектуры, на диаграмме показано использование двух шаблонов проектирования. Первый из них – Strategy. В рамках этого шаблона определен интерфейс «Инструмент AI-ассистента (AiTool)», который задает общий метод выполнения операции над состоянием доски. Его конкретными реализациями являются CreateTextElementTool, CreateStickyTool, CreateShapeTool,

CreateConnectorTool, UpdateElementTool, MoveElementTool и DeleteElementTool. Это означает, что AI-ассистент может вызывать разные инструменты в зависимости от запроса пользователя и интерпретации языковой модели. Класс AiService хранит реестр доступных инструментов и выполняет каждый вызов через единый интерфейс. Такое решение позволяет гибко расширять набор инструментов без изменения остальной части системы – при необходимости можно добавить новый тип операции, не переписывая логику разбора ответов модели.

Второй шаблон – Adapter. Для его реализации в шлюзе API используются инъектируемые gRPC-клиенты ClientGrpc, у которых при инициализации соответствующего модуля методом getService запрашивается реализация интерфейса нужного сервиса – IAuthService, IUsersService или IBoardsService, описанных в библиотеке общих типов в соответствии с proto-схемами. Получаемые таким образом объекты выступают в роли динамически создаваемых адаптеров и инкапсулируют детали сериализации Protobuf, обработки таймаутов и ошибок транспорта, предоставляя контроллерам единообразный объектно-ориентированный интерфейс. Использование данного подхода позволяет вынести логику межсервисной коммуникации из контроллерного слоя и облегчает добавление новых микросервисов в систему.

Если рассматривать диаграмму в целом, то она показывает полный путь обработки пользовательского запроса внутри системы. Зарегистрированный пользователь формирует запрос, например, на создание элемента на холсте, который передается через контроллер досок в шлюзе API в gRPC-адаптер сервиса досок и далее в сервис досок. Сервис взаимодействует с соответствующим репозиторием и сохраняет элемент в базе данных, после чего сервис совместного редактирования публикует событие об изменении всем участникам доски через WebSocket-шлюз. Параллельно администратор может управлять пользователями и пространствами через семейство административных контроллеров, а AI-ассистент по запросу пользователя через сервис AI вызывает соответствующие инструменты и формирует элементы на холсте.

Таким образом, представленная диаграмма классов отражает не только структуру данных предметной области, но и архитектурную организацию программного решения. В ней последовательно показаны учетные записи пользователей и их роли, основные сущности совместной работы с досками, элементы REST API и WebSocket-шлюзы, сервисы, репозитории и используемые шаблоны проектирования. За счет этого диаграмма дает целостное представление о построении системы и логике взаимодействия ее основных компонентов.

2.5 Диаграмма развертывания

Для наглядного представления физической структуры системы и распределения ее компонентов по вычислительным узлам целесообразно использовать диаграмму развертывания. Такая диаграмма позволяет показать, где именно размещаются клиентская часть, шлюз API, внутренние микросервисы, базы данных и вспомогательные хранилища, а также каким образом между ними осуществляется взаимодействие по сети. Для разрабатываемой системы это особенно важно, поскольку она реализована как распределенное клиент-серверное решение с микросервисной организацией серверной части, балансировкой нагрузки и выделением отдельных узлов для пользовательского интерфейса, серверной обработки и хранения данных. Диаграмма развертывания системы представлена на рисунке 2.7.

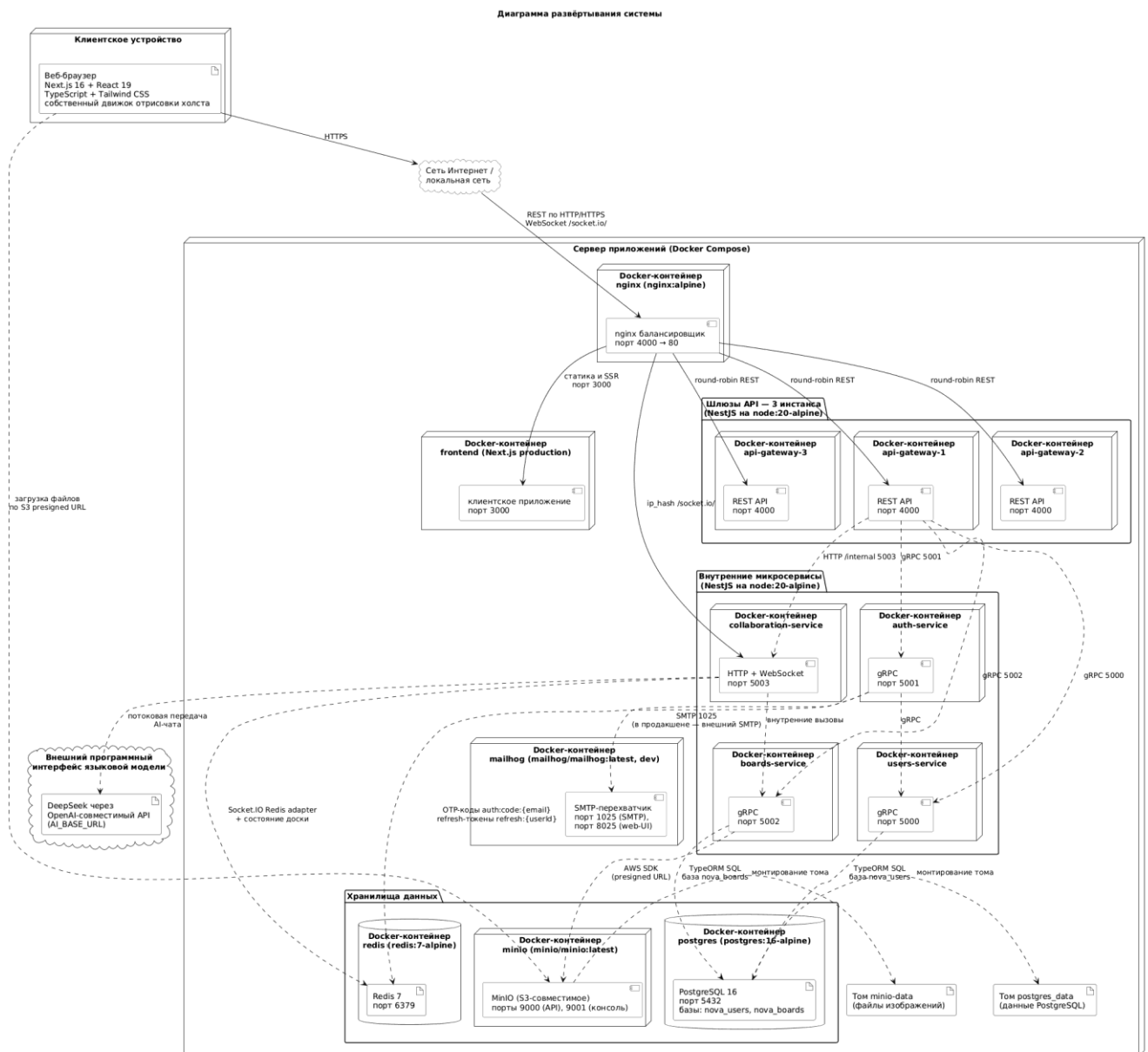


Рисунок 2.7 – Диаграмма развертывания

На рисунке 2.7 представлена диаграмма развертывания разрабатываемой системы совместной работы с интерактивными досками. В верхней части схемы показано клиентское устройство, на котором работает веб-браузер, реализующий пользовательский интерфейс системы. Клиентское приложение построено на основе фреймворка Next.js 16, библиотеки React 19 и языка TypeScript, использует стилизацию Tailwind CSS и собственный движок отрисовки холста. Развертывание клиентской части также выполняется в Docker-контейнере, который размещает оптимизированную сборку Next.js и обслуживает статические ресурсы и серверный рендеринг страниц. Взаимодействие с серверной частью осуществляется по протоколам HTTP и HTTPS для REST-запросов и по протоколу WebSocket поверх

HTTP/HTTPS для совместного редактирования и взаимодействия с AI-ассистентом. Между клиентским устройством и сервером приложений показана сеть передачи данных, которая может быть представлена как Интернетом, так и локальной сетью.

Центральным узлом схемы является сервер приложений, внутри которого с помощью Docker Compose развернут набор контейнеров, объединенных в единую виртуальную сеть. Внешний трафик принимает контейнер балансировщика нагрузки nginx, который прослушивает порт 4000 и по алгоритму round-robin распределяет REST-запросы между тремя независимыми контейнерами шлюза API (api-gateway-1, api-gateway-2 и api-gateway-3), а соединения по протоколу WebSocket направляет в контейнер сервиса совместного редактирования с привязкой клиента к одному и тому же экземпляру по схеме ip_hash. Шлюз API реализован на платформе Node.js с использованием фреймворка NestJS и при обработке запросов обращается к внутренним микросервисам по протоколу gRPC: контейнеру сервиса пользователей (users-service на порту 5000), контейнеру сервиса аутентификации (auth-service на порту 5001), контейнеру сервиса досок (boards-service на порту 5002), а также к контейнеру сервиса совместного редактирования (collaboration-service на внутреннем порту 5003) по внутреннему HTTP-протоколу. Для тестирования отправки одноразовых кодов в составе развертывания предусмотрен контейнер mailhog, перехватывающий исходящие письма и предоставляющий веб-интерфейс на порту 8025; в продуктивной среде он заменяется внешним SMTP-сервером.

В нижней части диаграммы представлены контейнеры хранения данных. Контейнер базы данных PostgreSQL версии 16 (postgres-контейнер на порту 5432) хранит две логически разделенные базы – nova_users и nova_boards – и связан с сервисом пользователей и сервисом досок по протоколу TCP с использованием SQL-запросов через драйвер TypeORM. Контейнер Redis версии 7 (redis-контейнер на порту 6379) используется для хранения одноразовых кодов подтверждения и состояния совместного редактирования и связан с сервисом аутентификации и сервисом совместного редактирования. Контейнер объектного хранилища MinIO

(minio-контейнер на порту 9000 для прикладного интерфейса и 9001 для административной консоли) обеспечивает хранение файлов изображений, загружаемых пользователями на доски, и взаимодействует с сервисом досок по S3-совместимому протоколу, при этом клиентское приложение выполняет загрузку файлов в хранилище напрямую по подписанным ссылкам. Хранение постоянных данных PostgreSQL и MinIO обеспечивается через именованные Docker-тома `postgres_data` и `minio-data`.

Таким образом, диаграмма отражает многоуровневую архитектуру развертывания системы: уровень клиентского устройства, уровень балансировщика нагрузки и шлюза API, уровень внутренних микросервисов и уровень хранения данных. Такое развертывание обеспечивает логическое разделение компонентов, упрощает сопровождение системы, повышает управляемость и делает архитектуру пригодной для дальнейшего горизонтального масштабирования – в частности, за счет добавления новых инстансов шлюза API и сервиса совместного редактирования в конфигурацию балансировщика `nginx`.

3. РЕАЛИЗАЦИЯ ИНФОРМАЦИОННОЙ СИСТЕМЫ

3.1 Структура проекта и стек технологий

Структура (рисунок 3.1) разрабатываемой системы совместной работы с интерактивными досками построена по многоуровневому принципу с микросервисной организацией серверной части, что обеспечивает логическое разделение компонентов приложения и упрощает их сопровождение. В проекте выделяются клиентский уровень, уровень балансировщика нагрузки и шлюза API, уровень внутренних микросервисов, уровень сервиса совместного редактирования и модуля искусственного интеллекта, а также уровень хранения данных. Такое построение позволяет разделить пользовательский интерфейс, обработку запросов, реализацию бизнес-логики и работу с хранилищами. Наряду со структурой приложения на рисунке также представлен используемый стек технологий, включающий средства frontend- и backend-разработки, технологии совместного редактирования в реальном времени, интеграции с большой языковой моделью, инструменты хранения информации и средства разработки и развертывания.

Структура проекта и стек технологий

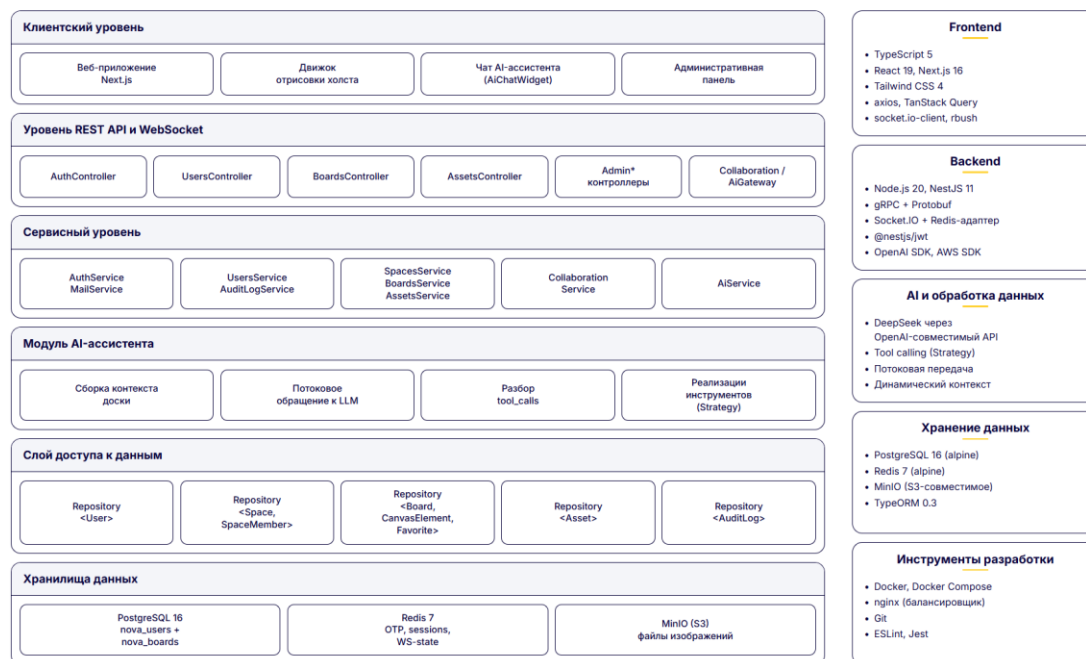


Рисунок 3.1 – Структура проекта и стек технологий разрабатываемой системы

На рисунке 3.1 представлена общая структура разработанной системы и используемый стек технологий. Архитектура проекта построена по многоуровневому принципу и включает клиентский уровень, уровень балансировщика нагрузки nginx и шлюза API, уровень внутренних микросервисов пользователей, аутентификации и досок, уровень сервиса совместного редактирования с модулем искусственного интеллекта, слой доступа к данным на основе TypeORM-репозитория и уровень хранилищ данных.

Клиентский уровень реализован в виде Next.js-приложения на базе библиотеки React 19 с типизацией на TypeScript и стилизацией Tailwind CSS. Он включает разделы для регистрации и авторизации, рабочих пространств, интерактивных досок, профиля пользователя, административной панели и публичной посадочной страницы. Особое место в составе клиентского приложения занимает собственный движок отрисовки холста, реализованный без использования внешних библиотек двумерной графики; в его состав входят модули отрисовки сцены, обработки ввода с клавиатуры и указательных устройств, выделения и трансформации элементов, ведения локальной истории действий, синхронизации состояния с сервером, работы с вьюпортом, изображениями, инструментами и якорями соединительных стрелок. Для обращения к серверной части используется библиотека axios в связке с TanStack Query, а для двунаправленного обмена сообщениями в реальном времени – клиент socket.io-client.

Уровень серверной части реализован на платформе Node.js с использованием фреймворка NestJS и организован в виде монорепозитория из пяти приложений. Шлюз API выступает единой точкой входа для REST-запросов клиента и направляет их соответствующим внутренним микросервисам по протоколу gRPC поверх Protobuf-схем, описанных в общих proto-файлах. Сервис пользователей отвечает за работу с учетными записями и журналом действий, сервис аутентификации – за процедуру подтверждения адреса электронной почты одноразовым кодом и вход через провайдеров OAuth с выпуском подписанных токенов JWT, сервис досок – за работу с рабочими пространствами, досками,

элементами холста и загружаемыми файлами. Использование микросервисной архитектуры с межсервисной коммуникацией по gRPC обеспечивает четкое разграничение ответственности, удобство независимой разработки и горизонтального масштабирования.

Сервис совместного редактирования реализует двунаправленное взаимодействие с клиентами по протоколу WebSocket на основе библиотеки Socket.IO и использует Redis в качестве адаптера для обмена сообщениями между несколькими инстансами сервиса. Внутри того же сервиса располагается модуль искусственного интеллекта, обращающийся к большой языковой модели DeepSeek через совместимый с OpenAI программный интерфейс по протоколу потоковой передачи и выполняющий разбор вызовов инструментов, возвращаемых моделью. Такое размещение позволяет AI-ассистенту работать в общем контексте доски и публиковать изменения всем участникам совместного редактирования через единый канал сообщений.

Уровень хранения данных представлен реляционной СУБД PostgreSQL версии 16, выделенной для хранения двух логически разделенных баз – учетных записей с журналом действий и рабочих пространств с досками, – а также Redis версии 7 для хранения одноразовых кодов подтверждения и состояния совместного редактирования и объектным хранилищем MinIO для хранения файлов изображений, загружаемых пользователями на доски. Взаимодействие сервисов с реляционной базой данных выполняется через слой репозитория TypeORM, что позволяет изолировать логику работы с хранилищем от бизнес-логики приложения, а с объектным хранилищем – через подписанные ссылки, получаемые с использованием AWS SDK.

В правой части рисунка показан стек технологий системы. Для клиентской части используются TypeScript, React, Next.js, Tailwind CSS, библиотека TanStack Query для работы с серверным состоянием, axios для REST-запросов, socket.io-client для совместного редактирования и rbush для пространственного индексирования элементов холста по R-дереву. Применение Next.js обеспечивает рендеринг страниц на стороне сервера и упрощает структурирование маршрутов

приложения, а React и TypeScript обеспечивают надежную типизацию и декларативное описание интерфейса.

В качестве серверного стека используются Node.js, NestJS, gRPC и Protobuf, Socket.IO, библиотека @nestjs/jwt для работы с подписанными токенами доступа и официальный SDK OpenAI для обращения к большой языковой модели. Фреймворк NestJS упрощает построение приложения, настройку модулей, валидацию входных данных и интеграцию слоев системы, а использование gRPC обеспечивает эффективную межсервисную коммуникацию с низкой задержкой. Для взаимодействия с объектным хранилищем применяется AWS SDK с поддержкой S3-совместимого протокола и формированием подписанных ссылок.

Для хранения данных используются PostgreSQL, Redis и MinIO. Выбор PostgreSQL обусловлен ее надежностью, поддержкой сложных запросов, высокой производительностью и удобством интеграции с серверными приложениями через TypeORM. Redis обеспечивает быстрое хранение одноразовых кодов с истечением срока действия и согласование сообщений совместного редактирования между несколькими инстансами сервиса. Объектное хранилище MinIO позволяет эффективно сохранять загружаемые пользователями файлы изображений и предоставляет S3-совместимый прикладной интерфейс, что упрощает дальнейшую замену его на облачный сервис при необходимости.

В блоке инструментов разработки и развертывания указаны Git, Docker, Docker Compose, nginx, ESLint и Jest. Git применяется для контроля версий и совместной работы над проектом, Docker и Docker Compose – для контейнеризации всех компонентов системы и упрощения развертывания, nginx – для балансировки нагрузки между инстансами шлюза API и сервиса совместного редактирования, ESLint – для автоматической проверки качества исходного кода, а Jest – для написания и запуска модульных тестов серверной и клиентской частей. Применение данного набора инструментов делает процесс разработки и ввода системы в эксплуатацию более организованным и воспроизводимым.

Таким образом, представленная структура проекта показывает последовательную организацию системы по уровням – от пользовательского

интерфейса до хранилищ данных. Используемый стек технологий обеспечивает реализацию веб-интерфейса с собственным движком отрисовки холста, серверной логики на основе микросервисной архитектуры, механизмов совместного редактирования в реальном времени и взаимодействия с большой языковой моделью, а также надежного хранения и развертывания всех компонентов. В совокупности это формирует целостную архитектуру системы, ориентированную на модульность, расширяемость и удобство сопровождения.

3.2 Реализация серверной части

Серверная часть разрабатываемой системы реализует основную бизнес-логику приложения, обеспечивает обработку пользовательских запросов, взаимодействие с базой данных и объектным хранилищем, выполнение операций совместного редактирования досок и работу с AI-ассистентом, а также контроль прав доступа. В рамках проекта серверная часть построена по микросервисной архитектуре с разделением функциональности на пять приложений: шлюз API, сервис пользователей, сервис аутентификации, сервис досок и сервис совместного редактирования. Внутри каждого приложения соблюдается многоуровневая организация с разделением на контроллеры, сервисы и репозитории. Такой подход позволяет сделать систему более понятной, расширяемой и удобной для сопровождения.

В качестве основного языка разработки серверной части используется TypeScript, а в качестве базового фреймворка – NestJS на платформе Node.js. Данное решение позволяет упростить настройку приложения, организовать обработку HTTP-запросов через REST API, реализовать зависимость компонентов друг от друга через механизм внедрения зависимостей, а также обеспечить интеграцию с системой безопасности и базой данных. Межсервисная коммуникация выполняется по протоколу gRPC поверх Protobuf-схем, описанных в общих proto-файлах. Для работы с базой данных используется подход, основанный на репозиториях TypeORM, что позволяет отделить бизнес-логику от непосредственной работы с данными.

REST-контроллеры

С точки зрения архитектуры серверная часть включает несколько уровней. На внешнем уровне располагаются REST-контроллеры (рисунок 3.2) шлюза API, принимающие запросы от клиентского приложения. Ниже находится сервисный слой, реализованный в составе отдельных микросервисов и сосредоточивший основную логику обработки данных и выполнения операций. Далее следует слой репозитория, обеспечивающий доступ к хранимой информации. Такое разделение обязанностей соответствует распространенному подходу Controller – Service – Repository и позволяет избежать избыточной связанности между компонентами системы.

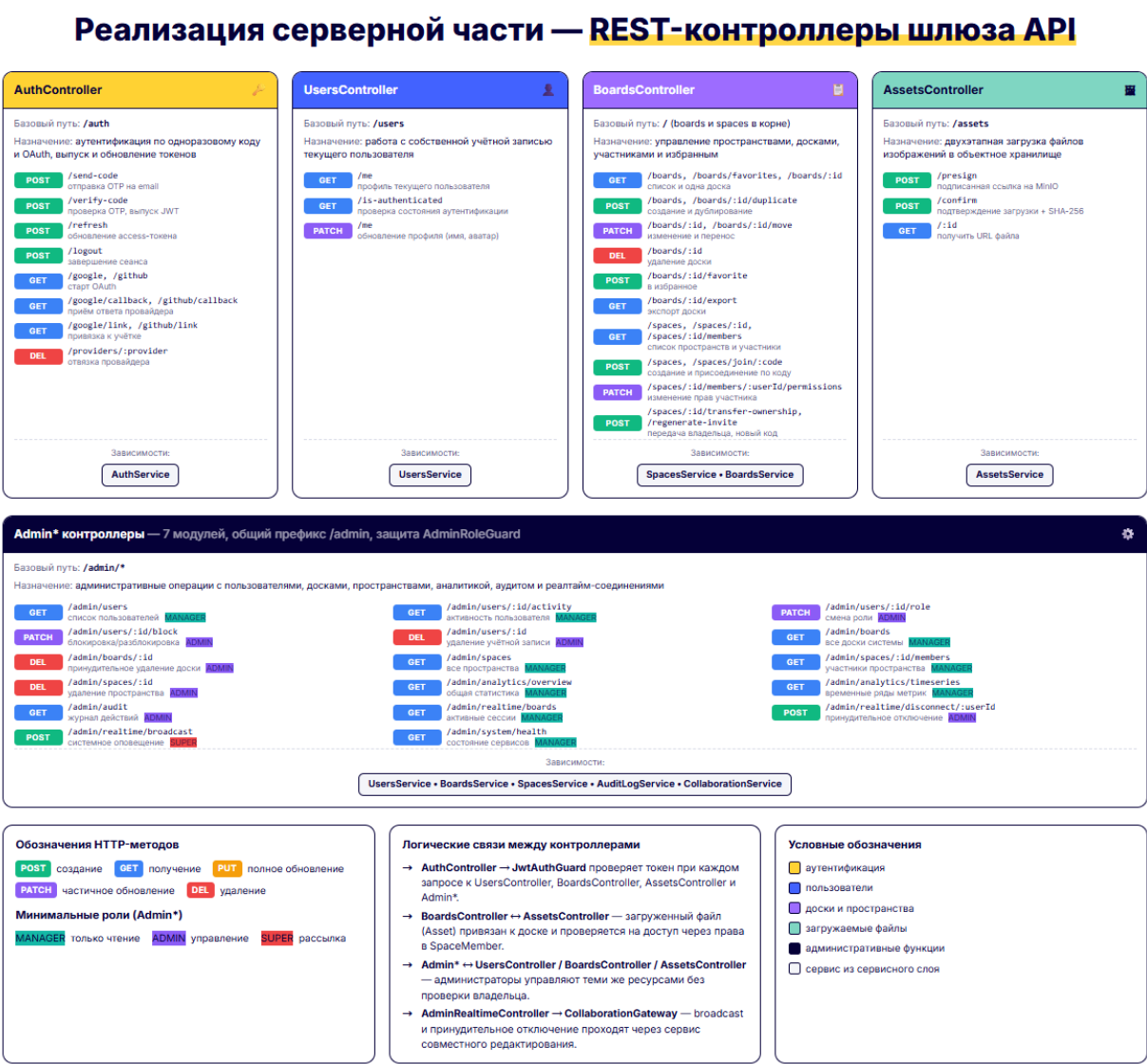


Рисунок 3.2 – REST-контроллеры серверной части

Контроллер `AuthController` отвечает за операции, связанные с аутентификацией и регистрацией пользователей. Через него выполняются отправка одноразового кода на электронную почту, его проверка, вход через провайдеров Google и GitHub по стандарту OAuth 2.0, обновление подписанных токенов и завершение сеанса работы. Контроллер принимает данные клиента, передает их сервису аутентификации и возвращает результат выполнения операции.

Контроллер `UserController` реализует функциональность, связанную с учетной записью текущего пользователя: получение профиля, обновление профиля и проверку состояния аутентификации.

Контроллер `BoardsController` объединяет в себе работу с интерактивными досками, рабочими пространствами, участниками пространств, приглашениями по коду, изменением прав участников, передачей владельца пространства и добавлением досок в избранное. Такое объединение отражает реальную организацию кода: эндпойнты пространств и избранного находятся в одном контроллере шлюза, поскольку обращаются к одному и тому же микросервису досок.

Контроллер `AssetsController` обеспечивает двухэтапную загрузку файлов изображений: формирование подписанной ссылки на объектное хранилище MinIO, подтверждение успешной загрузки и проксирование получения файла по идентификатору.

Отдельную группу образуют административные контроллеры, сосредоточенные в директории `admin` шлюза API. К ним относятся `AdminUserController`, отвечающий за списки пользователей, изменение ролей, блокировку и удаление учетных записей; `AdminBoardsController` – за управление любыми досками без проверки владельца; `AdminSpacesController` – за управление любыми рабочими пространствами и просмотр их участников; `AdminAnalyticsController` – за общую аналитику и временные ряды активности пользователей и досок; `AdminAuditController` – за выборки из журнала действий с фильтрами; `AdminRealtimeController` – за принудительное отключение пользователей от сервиса совместного редактирования и рассылку системных

сообщений; AdminSystemController – за проверку состояния микросервисов. Каждый из этих контроллеров защищен декоратором @Roles с указанием минимально допустимой роли пользователя.

Помимо REST-контроллеров, в сервисе совместного редактирования реализованы два WebSocket-шлюза. Шлюз CollaborationGateway обрабатывает сообщения присоединения и выхода из доски (board:join и board:leave), перемещения курсора (cursor:move), промежуточные события рисования (element:drawing) и операции с элементами холста (element:create, element:update и element:delete). Шлюз AiGateway обрабатывает сообщения взаимодействия с AI-ассистентом (ai:message) и публикует потоковые ответы языковой модели и события выполнения инструментов всем участникам доски.

Сервисный слой

Сервисный слой (рисунок 3.3) является основным местом реализации бизнес-логики. Именно здесь происходит проверка входных данных, вызов алгоритмов работы с досками, выполнение административных действий, координация работы с базой данных и объектным хранилищем, а также взаимодействие с большой языковой моделью. Сервисы распределены между четырьмя внутренними микросервисами и вызываются шлюзом API по протоколу gRPC.

Сервисный слой — бизнес-логика и связи с репозиториями

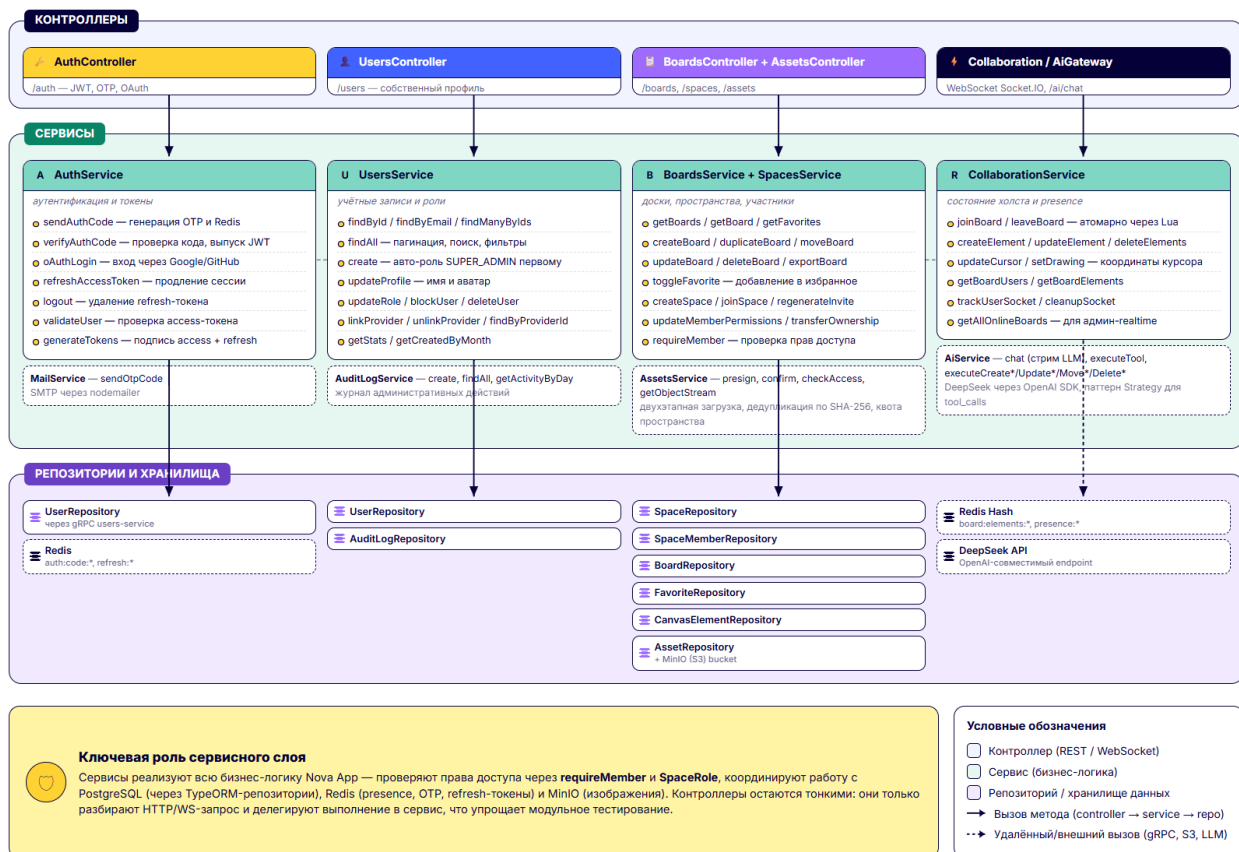


Рисунок 3.3 – Структура сервисного слоя серверной части

В составе сервиса аутентификации выделены два класса. Сервис AuthService отвечает за процедуру отправки и проверки одноразового кода, обработку сценариев OAuth для провайдеров Google и GitHub, выпуск и обновление подписанных токенов JWT, а также завершение сеанса работы. Сервис MailService обеспечивает доставку сформированных одноразовых кодов на электронную почту пользователя.

В составе сервиса пользователей выделены сервис UsersService, отвечающий за создание учетных записей, поиск пользователей по идентификатору, электронной почте и идентификаторам внешних провайдеров, обновление профиля, изменение роли, блокировку и удаление учетных записей, и сервис AuditLogService, отвечающий за запись и выборку событий журнала действий. При создании первой учетной записи в системе UsersService автоматически назначает ей роль супер-администратора, что обеспечивает первоначальное разворачивание системы.

В составе сервиса досок выделены сервис `SpacesService`, отвечающий за создание, изменение и удаление рабочих пространств, приглашение участников по коду, перевыпуск кода, передачу прав владельца и изменение прав отдельных участников; сервис `BoardsService`, отвечающий за создание, изменение и удаление досок, добавление досок в избранное, их дублирование и экспорт; и сервис `AssetsService`, отвечающий за формирование подписанных ссылок на объектное хранилище, подтверждение загрузки и проверку прав доступа к загруженным файлам, в том числе за дедупликацию одинаковых файлов по контрольной сумме SHA-256.

В составе сервиса совместного редактирования выделены сервис `CollaborationService`, реализующий поддержку состояния доски в Redis, фиксирующий онлайн-участников и публикующий события редактирования всем участникам доски через библиотеку `Socket.IO` с использованием Redis-адаптера, и сервис `AIService`, выполняющий потоковый запрос к большой языковой модели `DeepSeek` и последовательный разбор полученных вызовов инструментов с их последующим выполнением над состоянием доски.

Контроллеры шлюза API не реализуют бизнес-логику самостоятельно, а делегируют ее сервисам соответствующих микросервисов. Для этого в шлюзе используется инъектируемый клиент `ClientGrpc`, у которого при инициализации модуля вызывается метод `getService` с указанием интерфейса соответствующего сервиса (`IAuthService`, `IUsersService` или `IBoardsService`), описанного в библиотеке общих типов. Полученный объект выступает в роли динамически создаваемого клиента и инкапсулирует детали сериализации `Protobuf`, обработки таймаутов и ошибок транспорта. Такое разделение обязанностей делает архитектуру более понятной и поддерживаемой.

Таким образом, сервисный слой выступает центральным звеном архитектуры серверной части. Он объединяет внешние запросы, поступающие через контроллеры шлюза API, с механизмами доступа к данным и внутренней бизнес-логикой системы. За счет вынесения основной логики на сервисный уровень и распределения ее между независимыми микросервисами структура приложения

становится упорядоченной, удобной для сопровождения, тестирования и последующего расширения.

Репозитории и работа с данными

Для организации доступа к базе данных в серверной части системы используется слой репозиторий TypeORM. Как показано на рисунке 3.4, репозитории выступают промежуточным звеном между сервисным слоем и хранилищем данных, обеспечивая сохранение, обновление, поиск и удаление сущностей без привязки бизнес-логики к конкретной реализации базы данных. В отличие от подхода с отдельными классами-репозиториями для каждой сущности, в проекте используется стандартный механизм TypeORM, при котором сервис получает репозиторий конкретной сущности через декоратор `@InjectRepository`, а тип репозитория задается обобщенным типом `Repository<T>`.



Рисунок 3.4 – Репозитории и работа с данными в серверной части

Как видно из рисунка 3.4, взаимодействие с данными в системе организовано по многоуровневому принципу. Верхний уровень образует сервисный слой, представленный сервисами UsersService, AuditLogService, SpacesService, BoardsService, AssetsService, CollaborationService и AiService, которые используют соответствующие репозитории для работы с сущностями предметной области.

На уровне репозитория выделены следующие компоненты. Репозиторий Repository<User> используется в сервисе пользователей для хранения и поиска учетных записей, в том числе по идентификатору, электронной почте и идентификаторам внешних провайдеров. Репозиторий Repository<AuditLog> используется тем же сервисом для ведения журнала действий администраторов. Репозитории Repository<Space> и Repository<SpaceMember> используются сервисом досок для работы с рабочими пространствами и их участниками. Репозитории Repository<Board>, Repository<CanvasElement> и Repository<Favorite> – для работы с досками, элементами холста и избранным. Репозиторий Repository<Asset> используется сервисом досок для хранения сведений о загружаемых файлах изображений.

Нижний уровень схемы образуют хранилища данных. Реляционная база данных PostgreSQL версии 16 разделена на две логические базы: база nova_users обслуживает сервис пользователей и хранит учетные записи и журнал действий, а база nova_boards обслуживает сервис досок и хранит рабочие пространства, участников, доски, элементы холста, избранное и сведения о загруженных файлах. Кэш-хранилище Redis версии 7 используется сервисом аутентификации для хранения одноразовых кодов с истечением срока действия по ключу вида auth:code:{email} и сервисом совместного редактирования – для хранения состояния доски и согласования сообщений между несколькими инстансами. Объектное хранилище MinIO используется для хранения самих файлов изображений, доступ к которым организуется через подписанные ссылки, формируемые с использованием AWS SDK.

Таким образом, использование слоя репозитория TypeORM делает архитектуру более модульной, облегчает тестирование, а сочетание реляционного хранилища, кэш-хранилища и объектного хранилища позволяет каждому виду данных размещаться в наиболее подходящем для него средстве хранения.

Права доступа

В серверной части системы реализовано разграничение прав доступа на основе ролей, что позволяет ограничить выполнение операций в зависимости от

статуса пользователя. Как показано на рисунке 3.5, в системе выделены пять основных уровней: неавторизованный посетитель, обычный пользователь (USER), менеджер (MANAGER), администратор (ADMIN) и супер-администратор (SUPER_ADMIN), каждый из которых обладает собственным набором разрешенных действий. Дополнительно для каждой пары пользователь – рабочее пространство определена роль участника пространства (OWNER или MEMBER) с набором флагов разрешенных действий.

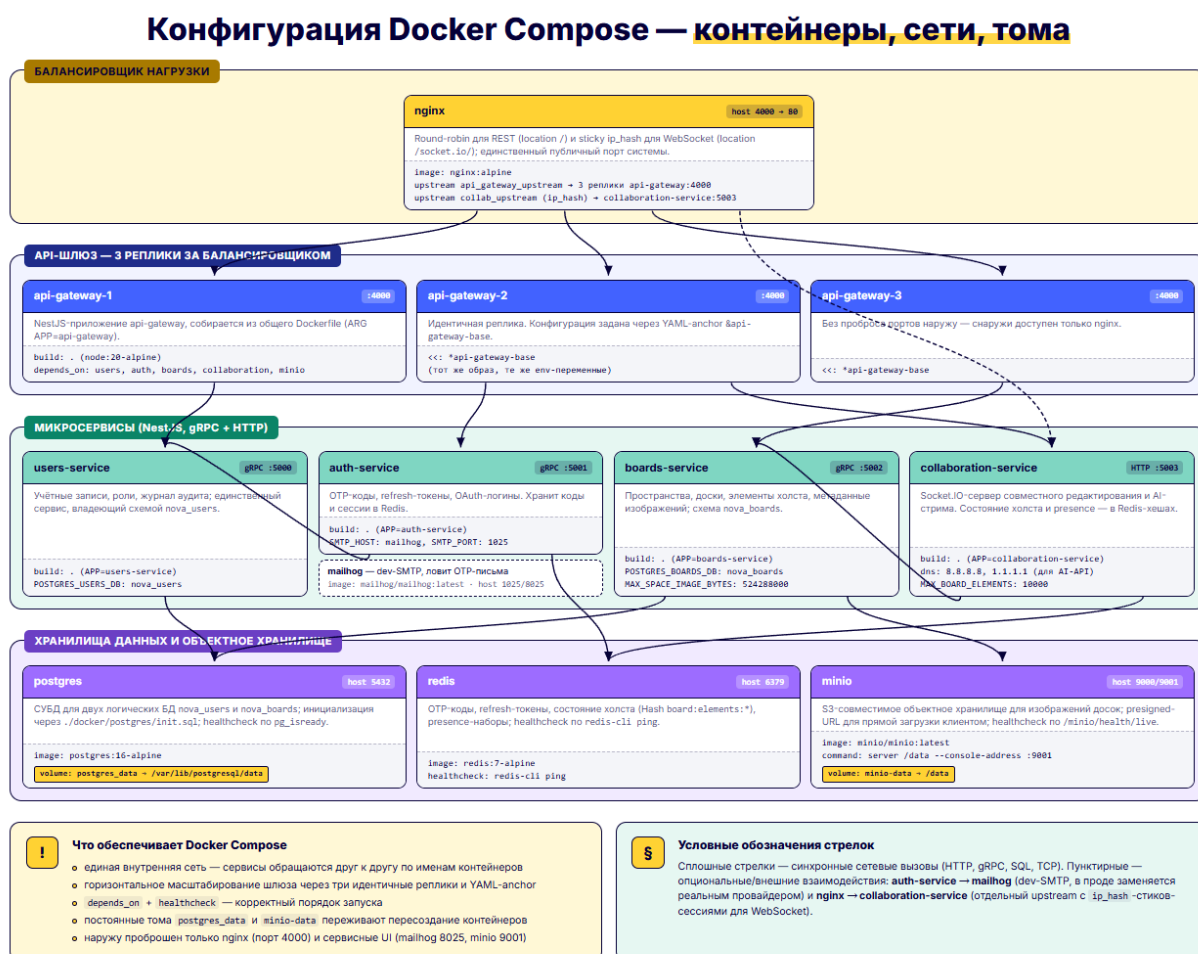


Рисунок 3.5 – Роли пользователей и права доступа в системе

Как видно из рисунка 3.5, минимальный уровень прав имеет неавторизованный посетитель. Для него доступны только базовые действия, не требующие выпуска токена доступа: просмотр посадочной страницы, регистрация в системе по электронной почте, вход в систему по одноразовому коду или через провайдеров Google и GitHub. При этом неавторизованный посетитель не имеет

доступа к функциям работы с досками, AI-ассистентом и административным операциям.

Следующий уровень представлен ролью обычного пользователя (USER). После успешной аутентификации пользователь получает доступ к основному функционалу системы. В частности, он может создавать собственные рабочие пространства, приглашать в них участников по коду, создавать и редактировать доски, использовать инструменты холста, обращаться к AI-ассистенту, загружать файлы изображений, а также работать со своей учетной записью в пределах разрешенных операций.

Роль менеджера (MANAGER) предназначена для сотрудников, выполняющих наблюдение за работой системы без права изменения данных. Менеджер наследует возможности обычного пользователя и дополнительно получает доступ к просмотру общей аналитики, списков пользователей, досок и рабочих пространств, мониторингу активности в реальном времени и проверке состояния микросервисов.

Максимальный уровень доступа в режиме управления системой имеет администратор (ADMIN). Как показано на рисунке 3.5, администратор наследует все возможности менеджера и дополнительно получает доступ к функциям изменения ролей пользователей, блокировки и удаления учетных записей, удаления любых досок и рабочих пространств без проверки владельца, принудительного отключения пользователей от сервиса совместного редактирования и просмотра журнала действий. Эксклюзивную возможность супер-администратора составляет рассылка системных сообщений всем пользователям, находящимся в сети, при этом первый зарегистрированный в системе пользователь автоматически получает роль супер-администратора.

Дополнительно на серверной стороне реализовано разграничение прав в пределах конкретного рабочего пространства. Каждой паре пользователь – пространство соответствует запись участника со значением роли OWNER или MEMBER и набором флагов разрешенных действий: создания досок, их редактирования, рисования и удаления. Часть операций над пространством, такие

как изменение его параметров, передача владельца и удаление, доступны только владельцу пространства. Все указанные проверки выполняются на уровне сервисного слоя.

Реализация разграничения прав построена на стандартных средствах NestJS. На уровне шлюза API подключен класс `JwtAuthGuard`, проверяющий подпись и срок действия подписанного токена доступа при каждом запросе. Дополнительно подключен класс `RolesGuard`, который считывает метаданные, выставленные декоратором `@Roles` на методах контроллеров, и сравнивает текущую роль пользователя с минимально допустимой в соответствии с заранее заданной иерархией. Внутри сервисного слоя выполняются проверки роли участника пространства и флагов разрешенных действий перед выполнением соответствующих операций.

Таким образом, представленная на рисунке 3.5 схема показывает иерархию ролей и соответствующее распределение прав доступа в системе. Разграничение полномочий между неавторизованным посетителем, обычным пользователем, менеджером, администратором и супер-администратором в сочетании с разграничением прав внутри рабочих пространств позволяет обеспечить безопасность приложения, корректное выполнение операций и защиту критически важных функций серверной части.

3.3 Реализация пользовательского интерфейса

Пользовательский интерфейс системы реализован в виде веб-приложения, обеспечивающего последовательный переход пользователя от ознакомления с системой к авторизации, организации рабочих пространств, созданию интерактивных досок, совместному редактированию в реальном времени, взаимодействию с AI-ассистентом и административному сопровождению системы. При проектировании интерфейса особое внимание уделялось простоте взаимодействия, логичности навигации и наглядному представлению состояния совместной работы.

Интерфейс построен по сценарному принципу: пользователь сначала получает доступ к ознакомительному разделу системы, затем проходит регистрацию и аутентификацию, после чего попадает в личный кабинет, где может создавать рабочие пространства, приглашать в них участников, открывать интерактивные доски и совместно редактировать их вместе с другими пользователями. Такой подход позволяет сделать работу с системой понятной даже для пользователя, не обладающего специальной технической подготовкой.

Главная страница и навигация по системе

Общая структура пользовательского интерфейса начинается с посадочной страницы, обеспечивающей ознакомление с возможностями веб-приложения и переход к процедуре аутентификации. Начальный экран пользовательского интерфейса представлен на рисунке 3.6.

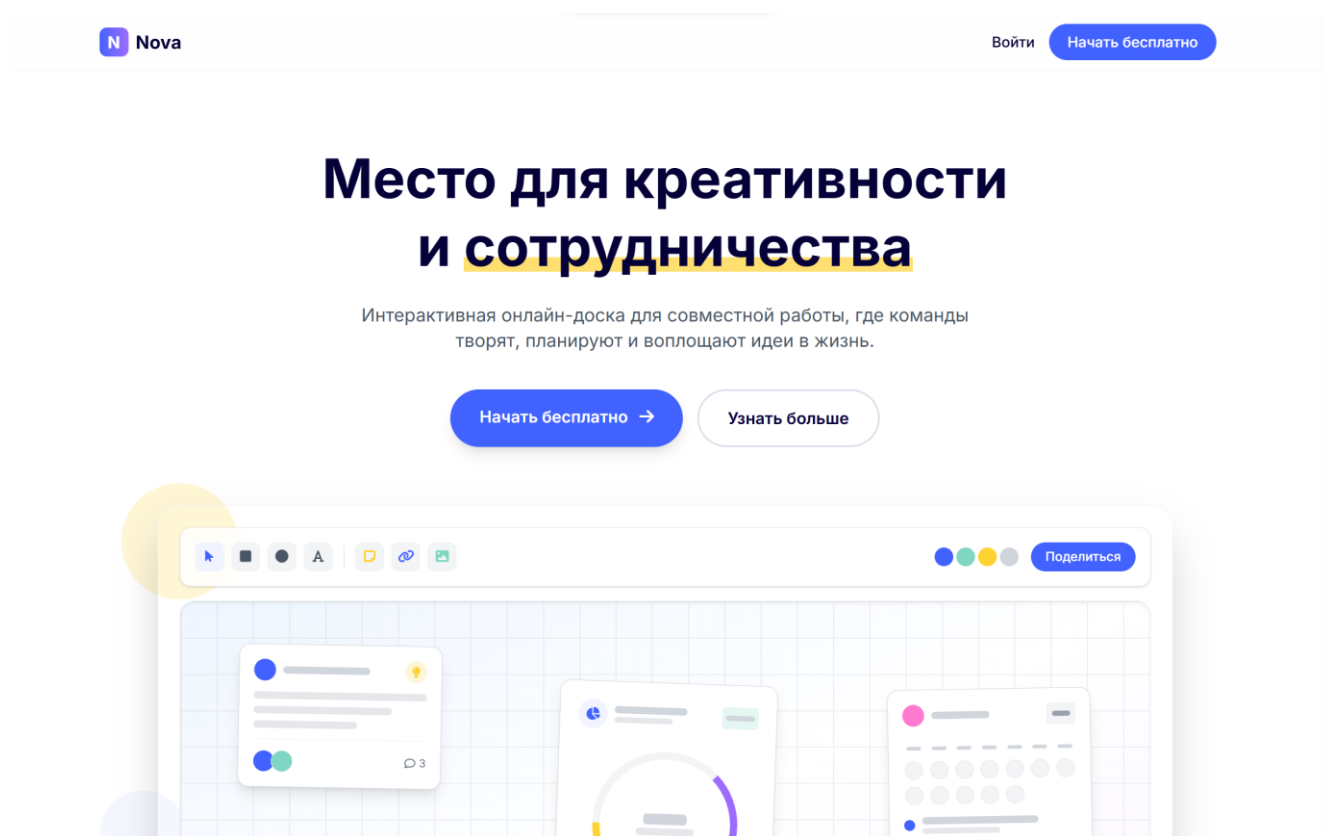


Рисунок 3.6 – Главная страница пользовательского интерфейса

Как показано на рисунке 3.6, в верхней части страницы размещен компонент Header, содержащий логотип системы, основные пункты меню навигации и кнопку перехода к аутентификации. Центральную часть страницы занимает блок Hero с заголовком системы, кратким описанием ее назначения и призывом к действию.

Ниже расположены блок Templates с примерами шаблонов готовых досок и блок SuccessStory с примером успешного применения системы. Завершает страницу компонент Footer с дополнительной навигацией и общей информацией о проекте. Такое решение делает стартовую страницу не только навигационной, но и информационной, позволяя пользователю сразу понять назначение системы и область ее применения.

Страница аутентификации пользователя

После перехода к защищенным функциям системы пользователь должен пройти процедуру входа. Интерфейс страницы аутентификации пользователя представлен на рисунке 3.7.

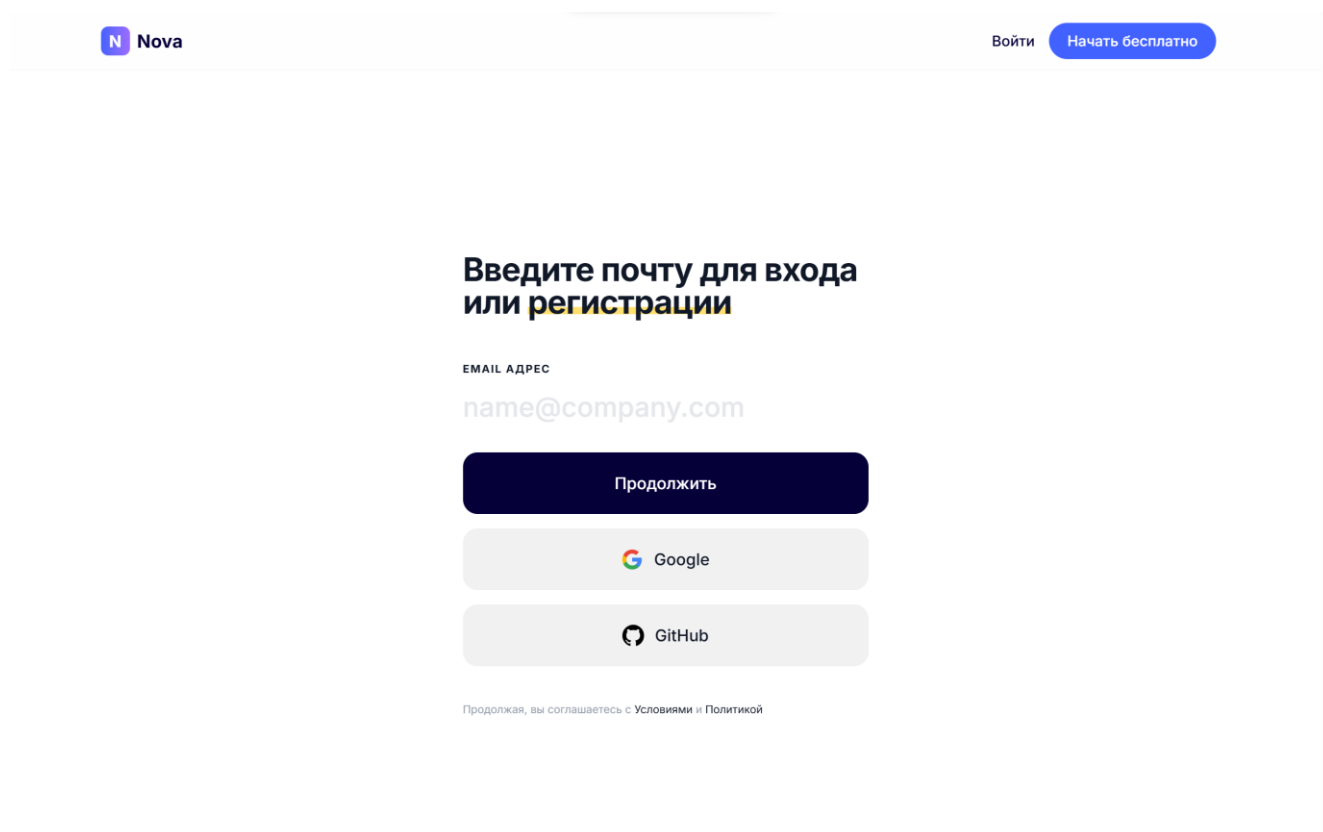
The image shows a web interface for user authentication. At the top left is the 'Nova' logo. At the top right are links for 'Войти' (Login) and 'Начать бесплатно' (Start free). The main heading is 'Введите почту для входа или регистрации' (Enter email for login or registration). Below this is an input field labeled 'EMAIL АДРЕС' with placeholder text 'name@company.com'. There are three buttons: a dark blue 'Продолжить' (Continue) button, a light gray button with the Google logo, and another light gray button with the GitHub logo. At the bottom, there is a small link: 'Продолжая, вы соглашаетесь с Условиями и Политикой'.

Рисунок 3.7 – Страница аутентификации пользователя

Как видно на рисунке 3.7, страница аутентификации реализована в виде двухэтапного сценария при помощи компонента AuthFlow, который переключает отображаемый шаг в зависимости от состояния процесса входа. На первом шаге, реализованном компонентом EmailStep, пользователю предлагается ввести адрес электронной почты, после чего система отправляет на него одноразовый код подтверждения. Рядом с формой ввода расположены кнопки социального входа

через провайдеров Google и GitHub, реализованные компонентом OAuthProviderList.

На втором шаге, реализованном компонентом OTPStep, отображается сетка для ввода одноразового кода OTPInputGrid и таймер повторной отправки OTPTimer, который позволяет запросить новый код по истечении заданного интервала времени. Такое решение соответствует современной практике беспарольной аутентификации и обеспечивает удобный для пользователя сценарий входа без необходимости запоминать пароль.

Личный кабинет и список рабочих пространств

После успешной аутентификации пользователь попадает в личный кабинет, в котором отображаются его рабочие пространства и доски. Интерфейс личного кабинета представлен на рисунке 3.8.

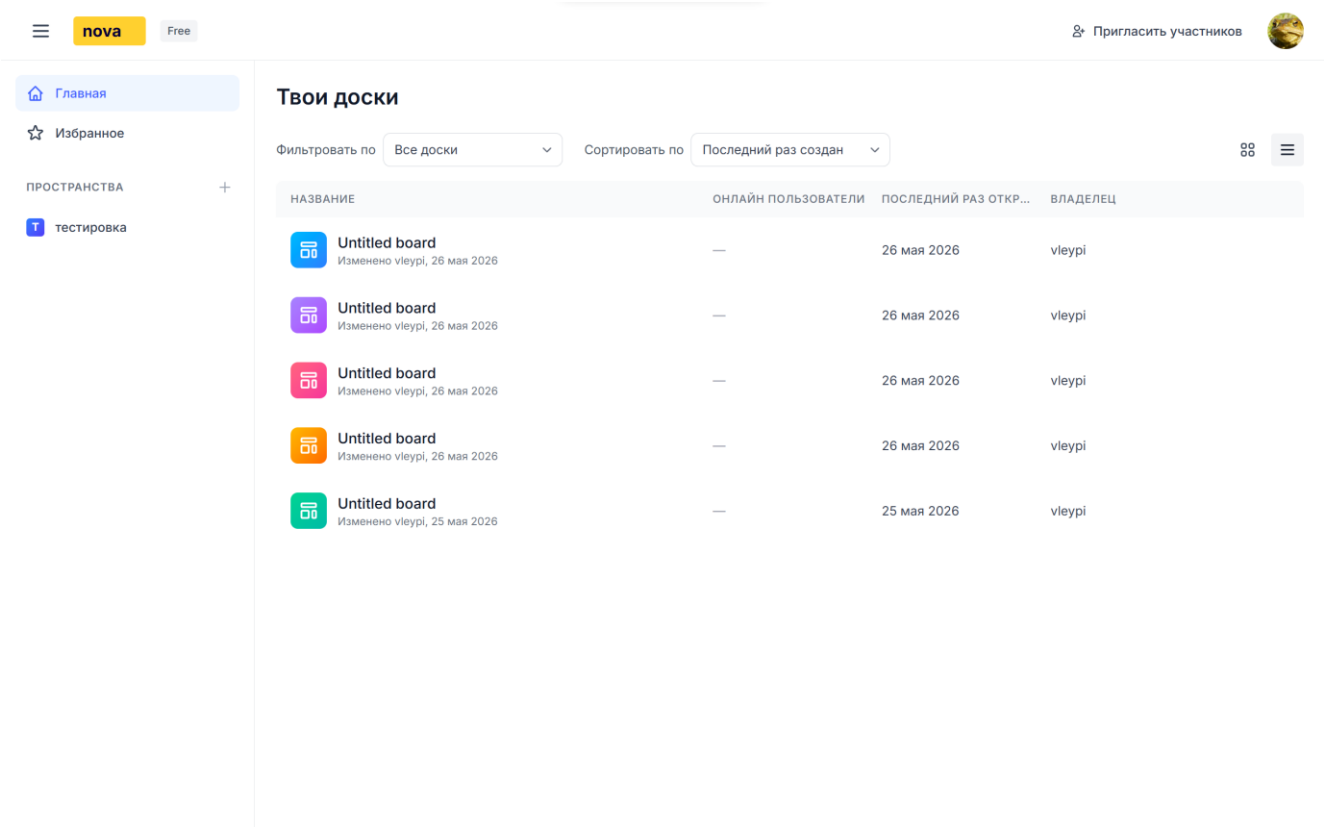


Рисунок 3.8 – Личный кабинет пользователя со списком рабочих пространств

Как показано на рисунке 3.8, страница личного кабинета построена по двухколоночной схеме. В левой части расположена боковая панель Aside, содержащая список рабочих пространств пользователя, пункты перехода к

избранным доскам и общему списку всех досок, а также кнопку создания нового пространства. В верхней части страницы расположен компонент AppBar с поисковой строкой, значком профиля и средствами выхода из системы.

Центральную часть страницы занимает список рабочих пространств SpacesList, отображающий созданные пользователем и присоединенные по коду пространства. При нажатии на кнопку создания пространства открывается модальная форма CreateSpaceForm с полем ввода названия. Для пустого состояния предусмотрен компонент SpacesEmptyState, предлагающий пользователю создать первое рабочее пространство. Внутри страницы конкретного пространства дополнительно доступны модальные окна управления участниками: InviteModal для приглашения новых участников по коду и SpaceMembersList для просмотра состава участников.

Страница рабочего пространства со списком досок

После выбора рабочего пространства пользователь переходит на страницу со списком досок, входящих в это пространство. Интерфейс соответствующей страницы представлен на рисунке 3.9.

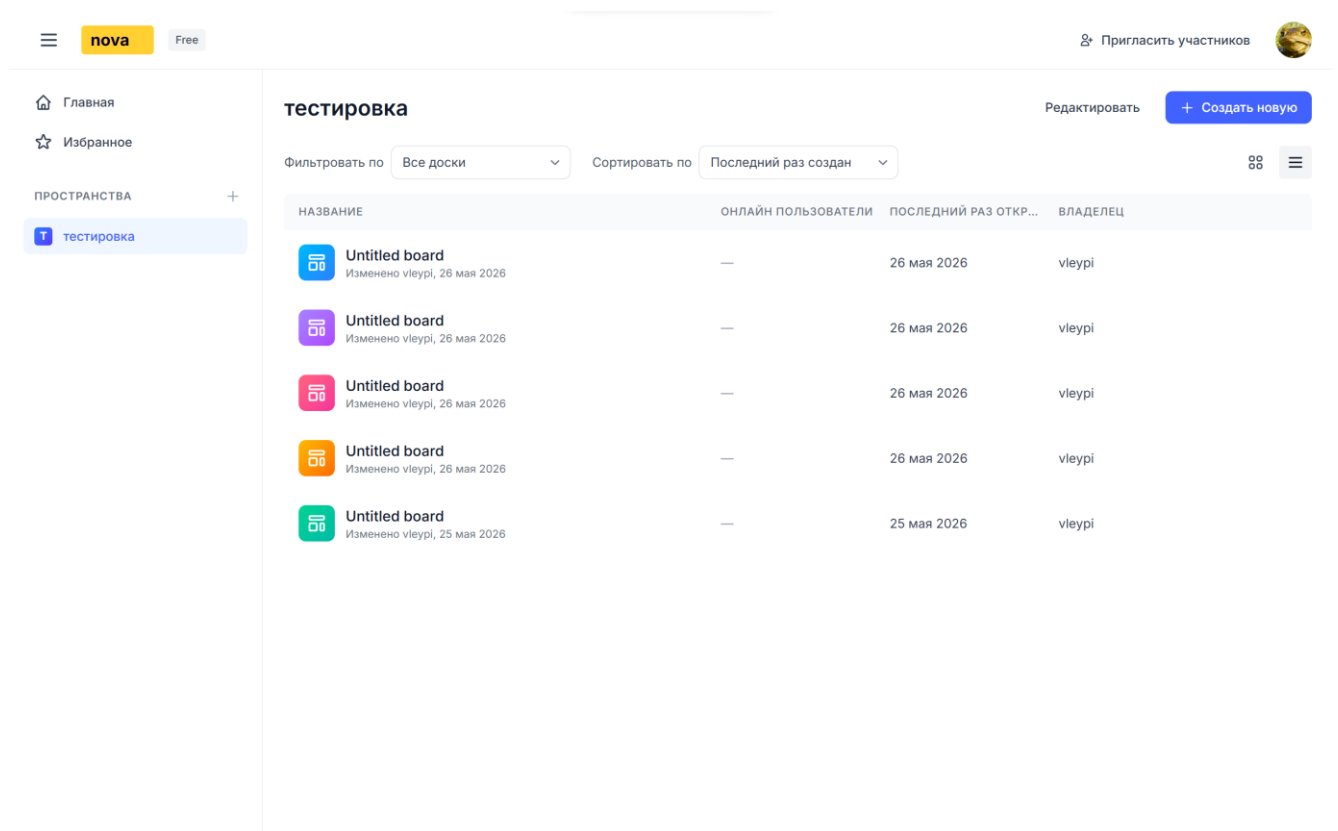


Рисунок 3.9 – Страница рабочего пространства со списком досок

Как видно из рисунка 3.9, в верхней части страницы расположен компонент PageHeader с названием пространства, кнопкой создания новой доски и средствами редактирования пространства. Ниже расположен компонент Filters, позволяющий выполнять фильтрацию, сортировку и переключение режима отображения списка между плиточным и табличным представлениями.

Основную часть страницы занимает список досок BoardList, отображающий каждую доску в виде карточки BoardCard в плиточном режиме или строки BoardItem в табличном режиме. С каждой доской связан контекстный компонент BoardContextMenu, открываемый правой кнопкой мыши и предоставляющий действия переименования, удаления, дублирования, перемещения между пространствами и изменения миниатюры. Для пустого состояния пространства предусмотрен компонент EmptyBoards. Все модальные окна, связанные с управлением досками, координируются компонентом BoardModalsManager.

Интерактивная доска и инструменты холста

Основным сценарием работы пользователя в системе является совместное редактирование интерактивной доски. Интерфейс соответствующего экрана представлен на рисунке 3.10.

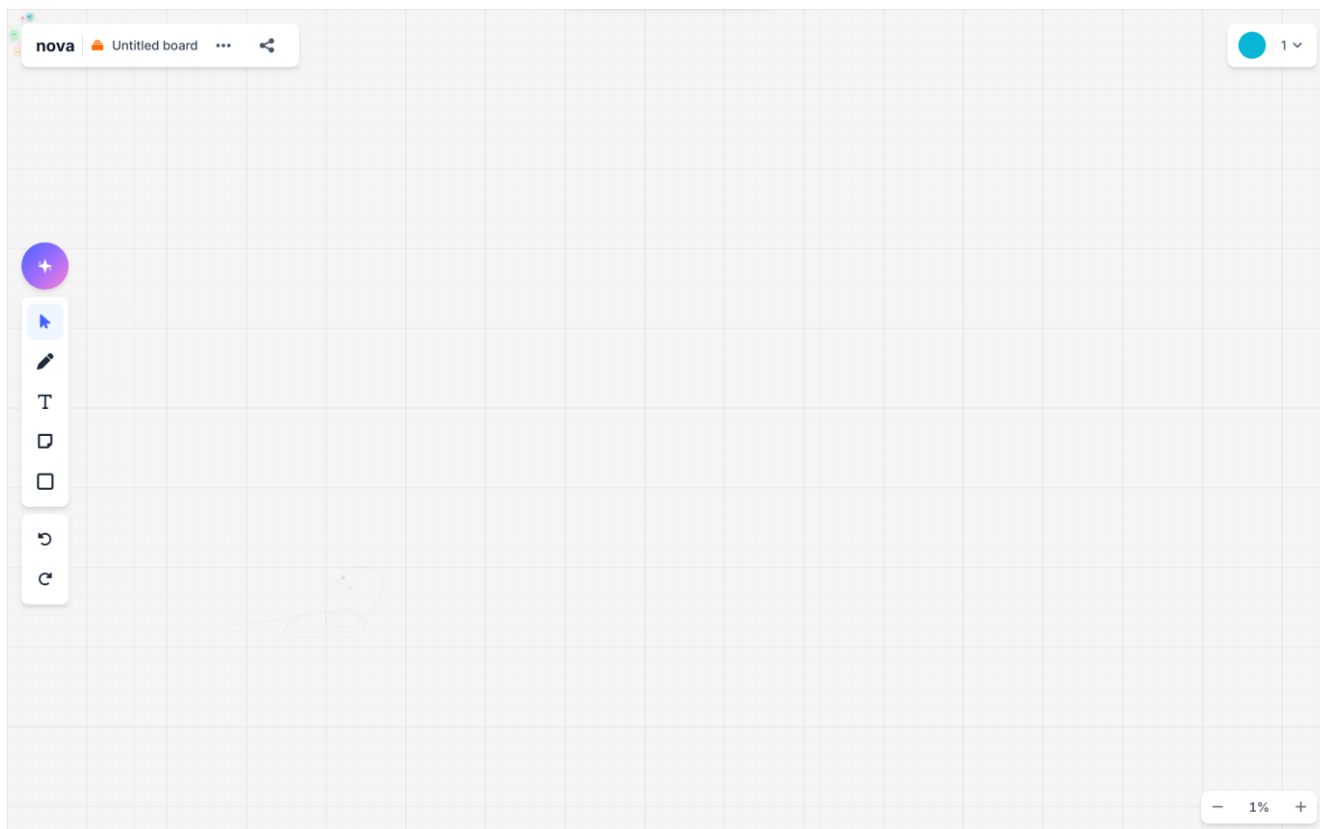


Рисунок 3.10 – Интерактивная доска и инструменты холста

Как показано на рисунке 3.10, центральную часть страницы занимает собственный движок отрисовки холста, отображающий все размещенные на доске элементы. В верхней части страницы расположен компонент BoardTopBar с названием доски и логотипом системы. В левой части страницы расположена вертикальная панель инструментов BoardLeftToolbar, содержащая основные инструменты холста: выделения, карандаша свободного рисования, добавления текстового блока, добавления стикера и добавления графической фигуры, а также кнопку открытия чата AI-ассистента и виджет управления историей действий HistoryWidget с кнопками отмены и повтора.

При активации отдельных инструментов рядом с основной панелью появляются дополнительные подпанели: PenSubToolbar – для выбора толщины и цвета линии при свободном рисовании, ShapeSubToolbar – для выбора типа графической фигуры (прямоугольник, эллипс, ромб или треугольник), StickyColorBar – для выбора цвета стикера и TextFormatToolbar – для форматирования текстового содержимого элементов. При выделении одного или нескольких элементов на холсте появляется визуальная рамка SelectionOverlay и контекстная панель SelectionToolbar с действиями, доступными для выбранных элементов.

В правой и нижней частях экрана размещены вспомогательные виджеты совместной работы: OnlineUsersWidget отображает список пользователей, открывших доску в данный момент, ConnectionStatusWidget показывает текущее состояние соединения с сервером совместного редактирования, ZoomWidget предоставляет управление масштабом холста, а компонент RemoteCursors отображает курсоры других участников в их актуальных позициях. Соединительные стрелки между элементами реализованы в виде кривых Безье, что обеспечивает наглядное представление логических связей между блоками.

Чат AI-ассистента

Особое место в интерфейсе занимает чат AI-ассистента, доступный непосредственно из интерактивной доски. Соответствующий экран представлен на рисунке 3.11.

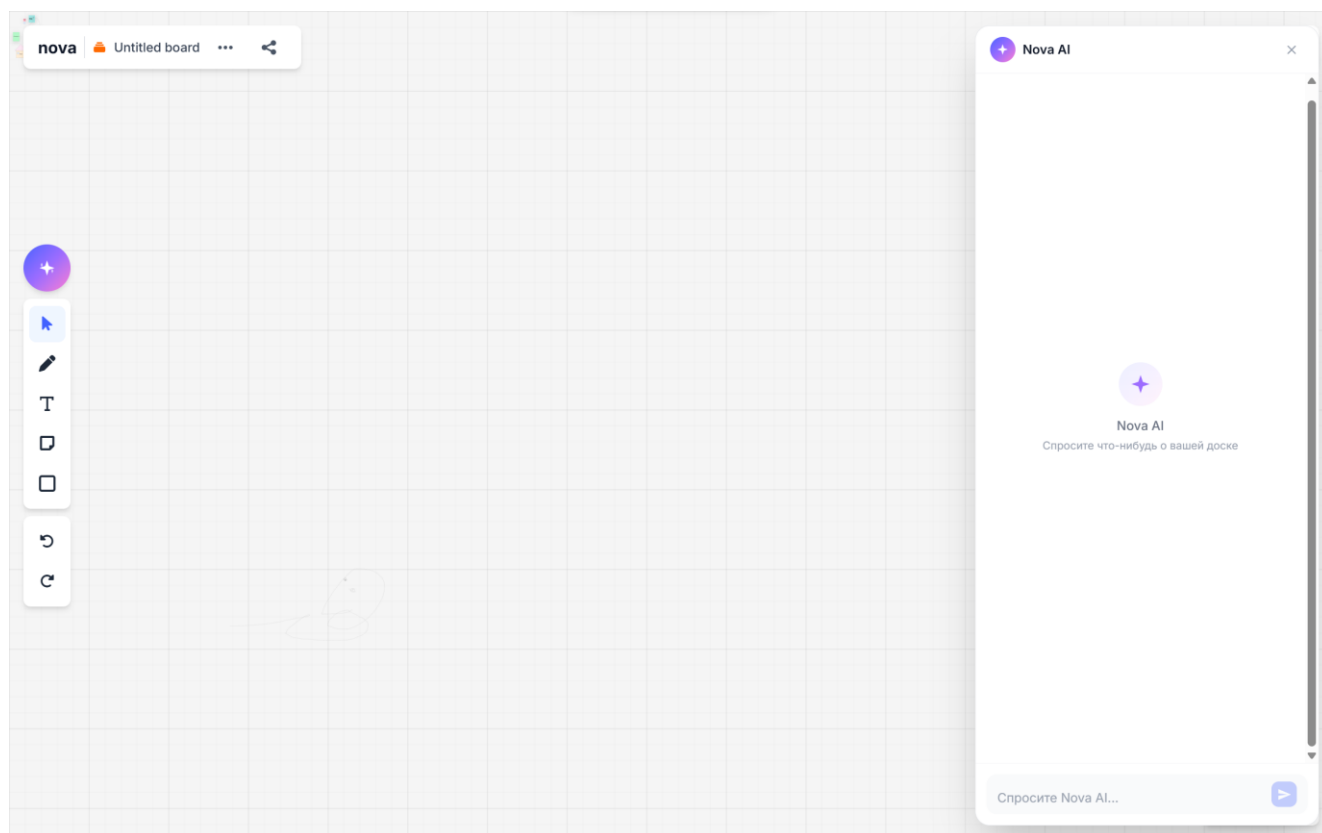


Рисунок 3.11 – Чат AI-ассистента на интерактивной доске

Как видно на рисунке 3.11, чат AI-ассистента реализован в виде плавающей панели AiChatWidget, открывающейся по нажатию на соответствующую кнопку в основной панели инструментов. Панель содержит историю сообщений с пользователем, поле ввода нового сообщения и кнопку отправки запроса. Сообщения пользователя и ответы языковой модели отображаются последовательно в виде ленты.

При отправке сообщения клиентское приложение через сервис AI-чата устанавливает потоковое соединение с сервером совместного редактирования и получает ответ языковой модели частями по мере его формирования. Параллельно с текстовой частью ответа клиент получает события создания новых элементов на холсте, выполненных AI-ассистентом через вызов инструментов, а также события о неудачных вызовах. Все построенные AI-ассистентом текстовые блоки, стикеры,

фигуры и соединительные стрелки в виде кривых Безье немедленно отображаются у всех участников совместного редактирования.

Административная панель

Для пользователей с административными ролями в системе предусмотрен отдельный раздел административной панели, доступный по выделенному маршруту. Интерфейс начального экрана административной панели представлен на рисунке 3.12.

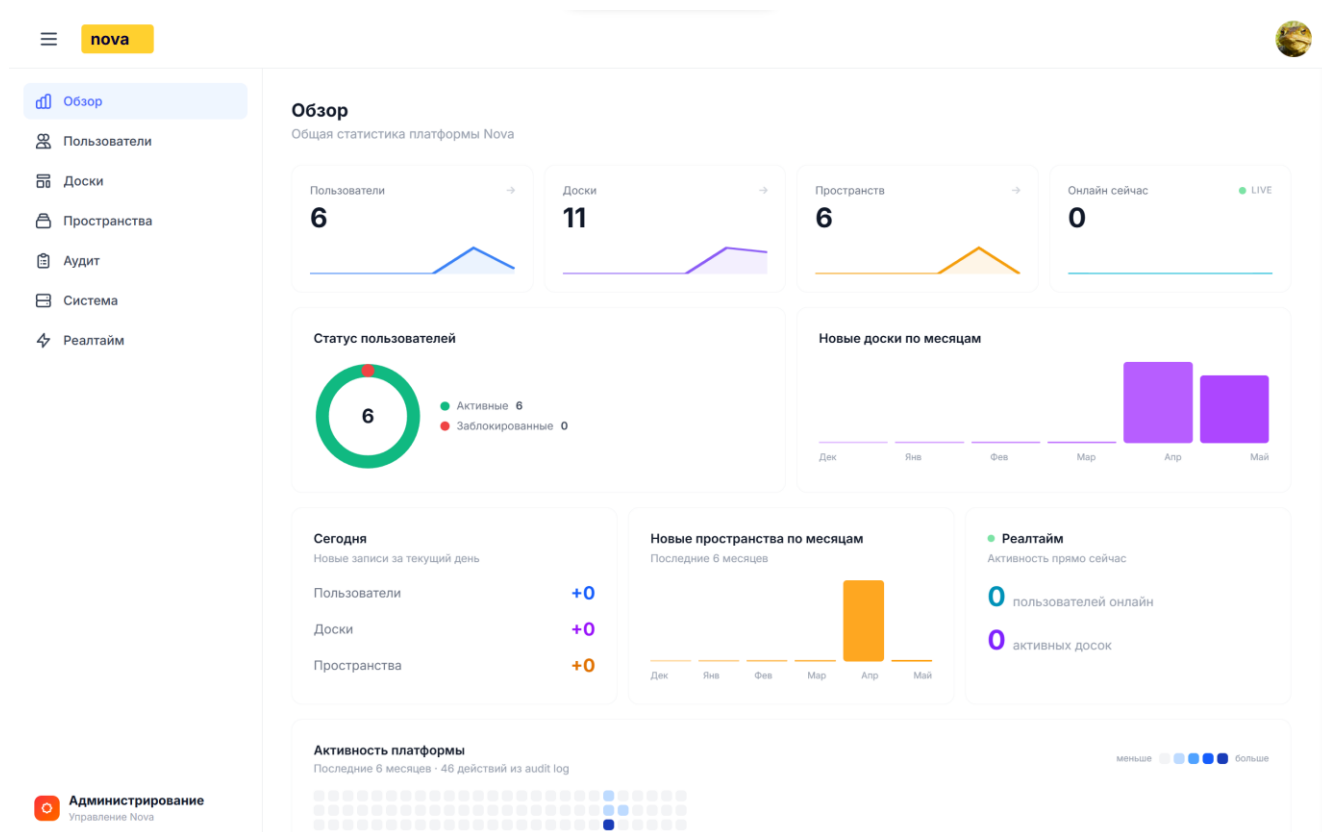


Рисунок 3.12 – Административная панель системы

Как показано на рисунке 3.12, страница административной панели построена по двухколоночной схеме. В левой части размещена боковая навигация AdminAside с пунктами «Обзор», «Пользователи», «Доски», «Пространства», «Журнал действий», «Реальное время» и «Система». Каждый из пунктов соответствует отдельной странице административной панели, доступ к которой регулируется ролью пользователя через компонент AdminGuard.

Начальный экран реализован страницей OverviewPage и содержит ключевые метрики системы: количество пользователей, рабочих пространств, досок и элементов холста, а также графики временных рядов активности и тепловую карту

активности по дням недели и часам. Страница `UsersPage` предоставляет средства управления учетными записями: списком пользователей, поиском, фильтрацией, формой `EditUserForm` с разделами назначения роли `UserRoleSection`, просмотра статистики пользователя `UserStatsTab` и удаления учетной записи `UserDangerZone`. Аналогичным образом построены страницы управления досками `BoardsPage` и пространствами `SpacesPage`, страницы `AuditPage` для просмотра журнала действий, `RealtimePage` для мониторинга текущих соединений и принудительного отключения пользователей и `SystemPage` для проверки состояния микросервисов системы.

Таким образом, реализация пользовательского интерфейса ориентирована на удобство, наглядность и последовательность взаимодействия. Интерфейс обеспечивает полный цикл работы пользователя с системой: от ознакомления и аутентификации до создания рабочих пространств, совместного редактирования интерактивных досок, взаимодействия с AI-ассистентом и административного сопровождения.

3.4 Реализация модуля искусственного интеллекта

Модуль искусственного интеллекта в разрабатываемой системе реализован в виде программной обвязки над большой языковой моделью `DeepSeek`, доступной через совместимый с `OpenAI` программный интерфейс. Ключевым программным элементом является сервис `AIService`, который выполняет подготовку контекста доски, обращение к модели, разбор возвращаемых вызовов инструментов и их выполнение над состоянием доски. Имя используемой модели и адрес ее программного интерфейса задаются переменными окружения `AI_MODEL` и `AI_BASE_URL`, что позволяет при необходимости заменять модель без изменения исходного кода сервиса.

Обращение к модели выполняется по протоколу потоковой передачи и сопровождается передачей следующих параметров. Параметр температуры со значением `0.5` регулирует степень детерминированности генерации и позволяет балансировать предсказуемость вызовов инструментов с разнообразием текстовых

ответов. Параметр максимально допустимой длины ответа со значением 4096 ограничивает объем формируемого текста. Параметр режима выбора инструментов со значением «auto» позволяет модели самостоятельно принимать решение о вызове того или иного инструмента в зависимости от смысла запроса пользователя. Дополнительно в запросе передается перечень доступных инструментов с описанием их аргументов. Для моделей с режимом рассуждений вместо температуры используются параметры коэффициента усилий и уровня подробности.

Поведение модели в системе настраивается с помощью собственной системной инструкции, состоящей из двух частей. Первая часть является статической и передается в каждом запросе. В ней заданы роль ассистента в составе коллаборативной доски, строгое правило ответа исключительно на русском языке независимо от языка пользовательского сообщения, три режима работы – «работа с холстом», «обычный ответ» и «смешанный режим», а также подробный каталог доступных инструментов с описанием их назначения, допустимых параметров, рекомендованных типов фигур, правил выбора цвета и формата текста. Вторая часть инструкции является динамической и пересобирается для каждого запроса: в нее входят текущее состояние доски, перечень уже размещенных элементов и история предыдущих сообщений с пользователем. Такое разделение позволяет точно направлять поведение модели и соотносить ее ответы с актуальным содержимым доски.

С точки зрения внутреннего устройства большая языковая модель представляет собой нейронную сеть архитектуры трансформер, обученную предсказывать следующий токен в последовательности на основе предыдущих. Несмотря на то, что в разрабатываемой системе сама модель не обучается, а используется в готовом виде, для понимания работы модуля целесообразно кратко рассмотреть ключевые математические соотношения, лежащие в ее основе.

На первом этапе входной текст разбивается на токены – подсловные единицы фиксированного словаря объемом V . Каждый токен сопоставляется числовому

вектору фиксированной размерности d с помощью обучаемой матрицы эмбедингов E размером $V \times d$ по соотношению (6):

$$e_i = E[x_i], \quad (1)$$

где x_i – индекс i -го токена в словаре, e_i – соответствующий ему вектор представления, а строка матрицы $E[x_i]$ выбирается по этому индексу. Полученные векторы суммируются с позиционными представлениями, что позволяет модели учитывать порядок токенов в последовательности.

На втором этапе вычисляются три набора векторных представлений – запрос, ключ и значение – для каждого токена входной последовательности по соотношению (7):

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V, \quad (2)$$

где X – матрица эмбедингов входной последовательности размером $n \times d$, W_Q , W_K и W_V – обучаемые матрицы проекций размером $d \times d_k$. Полученные матрицы Q , K и V содержат соответственно запросы, ключи и значения для всех n токенов последовательности.

На третьем этапе для каждого токена вычисляется механизм самовнимания, который определяет, насколько сильно текущий токен связан со всеми остальными токенами последовательности. Расчет выполняется по соотношению (8) – формуле масштабированного скалярного произведения внимания:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V, \quad (3)$$

где $Q \cdot K^T$ – матрица попарных скалярных произведений запросов и ключей размером $n \times n$, $\sqrt{d_k}$ – нормирующий множитель, предотвращающий насыщение функции $softmax$ при больших значениях размерности, а $softmax$ переводит каждую строку этой матрицы в распределение вероятностей «весов внимания». Умножение полученного распределения на матрицу V дает новое представление каждого токена как взвешенной комбинации значений всех остальных токенов последовательности.

В соотношении (8) ключевую роль играет функция *softmax*, которая преобразует произвольный числовой вектор в распределение вероятностей и определяется по соотношению (9):

$$\text{softmax}(z_i) = \frac{\exp(z_i)}{\sum_{j=1}^K \exp(z_j)}, \quad (4)$$

где z_i – i -я компонента входного вектора, K – размерность вектора, а выходное значение $\text{softmax}(z_i)$ лежит в интервале от 0 до 1, причем сумма всех выходных значений равна единице. Именно это свойство позволяет интерпретировать выход *softmax* как вероятностное распределение – как над весами внимания, так и над словарем токенов при формировании ответа модели.

Описанные выше преобразования выполняются параллельно в нескольких независимых группах – головах внимания – и многократно повторяются на последовательных слоях трансформера. На выходе модели для каждого момента времени t формируется вектор логитов o_t над всем словарем токенов. Вероятность очередного токена в ответе вычисляется по соотношению (10):

$$P(x_t | x_{<t}) = \text{softmax}\left(\frac{o_t}{T}\right), \quad (5)$$

где T – температура генерации, $x_{<t}$ – все предыдущие токены ответа, а x_t – очередной выбираемый токен. При значениях T , близких к нулю, распределение становится более резким и модель ведет себя более детерминированно, а при $T = 1$ сохраняется исходное распределение. В разрабатываемой системе используется значение $T = 0.5$, обеспечивающее устойчивый выбор инструментов и одновременно достаточное разнообразие текстовых ответов.

Описанная процедура порождения токенов применяется не только для формирования обычного текстового ответа в чате, но и для формирования вызовов инструментов. При работе в режиме работы с холстом модель порождает специальные служебные токены, соответствующие имени вызываемого инструмента и его аргументам в формате JSON. Сервис AiService последовательно разбирает полученные таким образом вызовы и выполняет каждый из них над состоянием доски через сервис досок, после чего сервис совместного редактирования публикует возникающие изменения всем участникам доски.

На первом этапе пользователь через чат AI-ассистента на интерактивной доске отправляет текстовый запрос. На втором этапе сервис AiService формирует контекст из статической системной инструкции, текущего состояния доски, перечня уже размещенных элементов и истории предыдущих сообщений. На третьем этапе сформированный контекст по протоколу потоковой передачи передается языковой модели DeepSeek через ее программный интерфейс, и модель частями возвращает текстовый ответ и последовательность вызовов инструментов. На четвертом этапе сервис AiService разбирает полученные вызовы и выполняет каждый из них над состоянием доски через сервис досок. На пятом этапе сервис совместного редактирования публикует возникающие события всем участникам доски, а текстовый ответ модели отображается в чате у инициатора запроса.

Таким образом, в разделе 3.4 показано, что модуль искусственного интеллекта реализован как программная обвязка над большой языковой моделью DeepSeek, использующей архитектуру трансформер с механизмом самовнимания и порождающей ответы и вызовы инструментов путем последовательного предсказания токенов. Подбор параметров обращения к модели, разработка собственной системной инструкции и автоматическая подготовка контекста доски на стороне сервиса AiService позволяют использовать готовую языковую модель для автоматического построения визуальных элементов на холсте по текстовому запросу пользователя и согласованного отображения результатов в режиме реального времени у всех участников совместного редактирования.

3.5 Контейнеризация и совместный запуск

Для упрощения развертывания разрабатываемой системы используется контейнерный подход. Контейнеризация позволяет изолировать основные компоненты приложения, обеспечить воспроизводимость среды выполнения и упростить перенос программного решения между различными серверами и рабочими станциями. В рамках проекта такой подход особенно важен, поскольку система включает клиентское приложение, шлюз API с балансировщиком нагрузки, четыре внутренних микросервиса и три отдельных хранилища данных.

Если запускать все эти компоненты вручную на целевой машине, возрастает риск ошибок конфигурации, несовместимости версий и нарушения порядка запуска сервисов.

В данной работе контейнеризация реализована с использованием Docker, а совместный запуск всех взаимосвязанных контейнеров выполняется через Docker Compose. Основная идея заключается в том, что каждый микросервис, балансировщик нагрузки, базы данных и вспомогательные хранилища работают в отдельных контейнерах, взаимодействующих друг с другом через общую внутреннюю сеть. Запуск всей системы из одной команды `docker compose up -d --build` обеспечивает воспроизводимое развертывание, а независимая сборка образов делает возможным обновление и масштабирование отдельных частей системы без остановки остальных.

Общая структура контейнерного развертывания

В состав контейнерной конфигурации входят несколько групп сервисов. К группе сервисов хранения относятся контейнер реляционной СУБД PostgreSQL версии 16, контейнер кэш-хранилища Redis версии 7 и контейнер объектного хранилища MinIO. Контейнер PostgreSQL хранит две логически разделенные базы – `nova_users` и `nova_boards`, контейнер Redis – одноразовые коды подтверждения и состояние совместного редактирования, а контейнер MinIO – файлы изображений, загружаемые пользователями на доски. Для отладочной среды дополнительно развертывается контейнер `mailhog`, перехватывающий исходящие письма с одноразовыми кодами.

К группе серверных сервисов относятся четыре микросервиса приложения: сервис пользователей, сервис аутентификации, сервис досок и сервис совместного редактирования. Каждый микросервис запускается в отдельном контейнере, собранном из общего Dockerfile с параметризованным именем приложения. Дополнительно развертываются три независимых инстанса шлюза API – `api-gateway-1`, `api-gateway-2` и `api-gateway-3`, между которыми распределяется внешний REST-трафик. Внешние запросы принимает контейнер балансировщика нагрузки `nginx`, который выступает единой точкой входа в систему.

Клиентское приложение также упаковывается в отдельный Docker-контейнер на основе оптимизированной production-сборки Next.js. Этот контейнер обслуживает статические ресурсы и серверный рендеринг страниц, а на запросы к серверной части обращается к контейнеру балансировщика nginx через общую внутреннюю сеть. Подобная организация позволяет запускать клиентскую и серверную части системы одной командой и поддерживать единый процесс развертывания.

Логически схема развертывания может быть представлена следующим образом: пользователь обращается к клиентскому приложению, клиентское приложение направляет REST-запросы на балансировщик nginx, который по алгоритму round-robin распределяет их между тремя инстансами шлюза API, а соединения по протоколу WebSocket направляет в контейнер сервиса совместного редактирования с привязкой клиента к одному инстансу по схеме ip_hash. Шлюз API обращается к внутренним микросервисам по протоколу gRPC, а микросервисы – к контейнерам хранилищ по соответствующим протоколам. Все контейнеры находятся в одной внутренней сети Docker Compose, что позволяет им обращаться друг к другу по именам сервисов, а не по внешним адресам.

Для контейнеров хранилищ обязательно подключаются постоянные тома: том postgres_data – для содержимого реляционной базы данных, том minio-data – для файлов объектного хранилища. Это обеспечивает сохранность информации при остановке и пересоздании контейнеров. Таким образом, контейнеризация в проекте решает сразу несколько задач: изоляцию компонентов, единообразие среды выполнения, упрощение развертывания, согласованный порядок старта сервисов и устойчивое хранение данных.

Требования к контейнерам

При проектировании контейнерной схемы были сформулированы требования к каждой группе контейнеров.

Контейнеры микросервисов и шлюза API должны обеспечивать запуск соответствующего NestJS-приложения в полностью подготовленной среде. Для этого используется общий многоступенчатый Dockerfile на базе образа node:20-

alpine, который выполняет установку production-зависимостей, сборку конкретного приложения командой `nest build` с параметром `APP`, копирование готового артефакта и файлов Protobuf-схем в финальный образ и запуск собранного приложения. Каждый контейнер должен поддерживать чтение настроек из переменных окружения, чтобы параметры подключения к базам данных, объектному хранилищу и языковой модели, а также секретные ключи не были жестко зашиты в исходный код.

Контейнер PostgreSQL должен обеспечивать устойчивое хранение информации и автоматический запуск СУБД после старта композиции. Для него подключается постоянный том `postgres_data`, а также файл инициализации `docker/postgres/init.sql`, создающий вторую базу данных `nova_boards` помимо основной `nova_users`, заданной переменной `POSTGRES_DB`. Дополнительно для контейнера базы данных задается механизм проверки готовности `healthcheck` с использованием утилиты `pg_isready`, что предотвращает запуск зависящих от базы сервисов раньше ее полной готовности.

Контейнер Redis должен поддерживать стандартные операции с ключами и предусматривать ту же проверку готовности с помощью команды `redis-cli ping`. Контейнер MinIO должен запускаться в режиме сервера объектного хранилища с прикладным интерфейсом на порту 9000 и административной консолью на порту 9001, а также иметь подключенный постоянный том `minio-data` и проверку готовности через прикладной `endpoint live`-проверки.

Контейнер балансировщика `nginx` должен запускаться с собственным конфигурационным файлом `docker/nginx/nginx.conf`, в котором описаны два `upstream`-блока: `round-robin` между тремя инстансами шлюза API для REST-запросов и `ip_hash` на сервис совместного редактирования для соединений по протоколу `WebSocket`. Дополнительно в конфигурации увеличены значения `proxy_read_timeout` и `proxy_send_timeout`, что предотвращает разрыв продолжительных `WebSocket`-соединений по умолчанию.

Контейнер клиентского приложения должен содержать оптимизированную production-сборку `Next.js` и обслуживать серверный рендеринг страниц на порту

3000. При сборке образа клиентского приложения должна выполняться команда `next build`, а при запуске – команда `next start`.

Отдельным обязательным требованием является поддержка миграций схемы базы данных. Структура баз `nova_users` и `nova_boards` не создается вручную при каждом развертывании, а формируется миграционными скриптами, описанными в каталогах `apps/<service>/src/migrations` соответствующих микросервисов. Миграции запускаются командой `npm run migration:users:run` и `npm run migration:boards:run`, что гарантирует приведение базы данных к актуальной версии схемы.

Структура файлов контейнерного развертывания

Для удобства сопровождения контейнерная конфигурация организована в виде понятной структуры файлов и каталогов проекта. Соответствующая структура представлена на рисунке 3.13.



Рисунок 3.13 – Структура файлов контейнерного развертывания

Файл `Dockerfile` содержит описание многоступенчатой сборки образов микросервисов и шлюза API на базе `node:20-alpine` с использованием параметра `ARG APP`, позволяющего собирать любой из микросервисов из одного общего описания. Файл `docker-compose.yml` содержит описание всех сервисов, сетей и томов. Файл `.env` используется для хранения реальных переменных окружения текущего окружения. Каталог `docker/nginx` хранит конфигурационный файл

nginx.conf для балансировщика нагрузки. Каталог docker/postgres хранит файл init.sql, создающий вторую базу данных при инициализации контейнера PostgreSQL. Каталог proto содержит общие Protobuf-схемы, копируемые в образы микросервисов и шлюза API для межсервисной коммуникации. Такое разбиение упрощает сопровождение проекта, поскольку каждая группа файлов отвечает за свою часть процесса развертывания.

Конфигурация совместного запуска через Docker Compose

Совместный запуск сервисов выполняется через файл docker-compose.yml, фрагмент которого показан в листинге 1. Полная конфигурация содержит описание всех контейнеров системы, общих переменных окружения для трех экземпляров шлюза API, проверок готовности для контейнеров хранилищ и подключений постоянных томов.

```
# 3000 frontend
# 4000 nginx
# 5432 postgres
# 6379 redis
# 9000 minio
# 9001 minio
# 1025 SMTP
# 8025 web-UI mailhog

x-api-gateway-base: &api-gateway-base
  build:
    context: ./backend
    args:
      APP: api-gateway
  restart: unless-stopped
  env_file: ./backend/.env
  environment:
    NODE_ENV: production
    USERS_SERVICE_URL: users-service:5000
    AUTH_SERVICE_URL: auth-service:5001
    BOARDS_SERVICE_URL: boards-service:5002
    COLLAB_INTERNAL_URL: http://collaboration-service:5003/internal
    API_GATEWAY_PORT: 4000
    MINIO_ENDPOINT: http://minio:9000
  depends_on:
    - users-service
    - auth-service
    - boards-service
    - collaboration-service
    - minio

services:
  postgres:
    image: postgres:16-alpine
    restart: unless-stopped
    environment:
```

```

    POSTGRES_USER: nova
    POSTGRES_PASSWORD: nova_secret
    POSTGRES_DB: nova_users
volumes:
  - postgres_data:/var/lib/postgresql/data
  - ./docker/postgres/init.sql:/docker-entrypoint-initdb.d/init.sql
ports:
  - "5432:5432"
healthcheck:
  test: ["CMD-SHELL", "pg_isready -U nova -d nova_users"]
  interval: 5s
  timeout: 5s
  retries: 10

redis:
  image: redis:7-alpine
  restart: unless-stopped
  ports:
    - "6379:6379"
  healthcheck:
    test: ["CMD", "redis-cli", "ping"]
    interval: 5s
    timeout: 5s
    retries: 10

users-service:
  build:
    context: ./backend
    args:
      APP: users-service
  restart: unless-stopped
  env_file: ./backend/.env
  environment:
    USERS_SERVICE_URL: 0.0.0.0:5000
    POSTGRES_HOST: postgres
    POSTGRES_PORT: 5432
    POSTGRES_USER: nova
    POSTGRES_PASSWORD: nova_secret
    POSTGRES_USERS_DB: nova_users
  depends_on:
    postgres:
      condition: service_healthy

auth-service:
  build:
    context: ./backend
    args:
      APP: auth-service
  restart: unless-stopped
  env_file: ./backend/.env
  environment:
    NODE_ENV: production
    AUTH_SERVICE_URL: 0.0.0.0:5001
    REDIS_HOST: redis
    REDIS_PORT: 6379
    USERS_SERVICE_URL: users-service:5000
    SMTP_HOST: mailhog
    SMTP_PORT: 1025
  depends_on:
    redis:
      condition: service_healthy
    users-service:
      condition: service_started
    mailhog:

```

```

        condition: service_started

mailhog:
  image: mailhog/mailhog:latest
  restart: unless-stopped
  ports:
    - "1025:1025"    # SMTP
    - "8025:8025"    # Web UI

boards-service:
  build:
    context: ./backend
    args:
      APP: boards-service
  restart: unless-stopped
  env_file: ./backend/.env
  environment:
    BOARDS_SERVICE_URL: 0.0.0.0:5002
    POSTGRES_HOST: postgres
    POSTGRES_PORT: 5432
    POSTGRES_USER: nova
    POSTGRES_PASSWORD: nova_secret
    POSTGRES_BOARDS_DB: nova_boards
    MINIO_ENDPOINT: http://minio:9000
    MINIO_PUBLIC_ENDPOINT: http://localhost:9000
    MAX_SPACE_IMAGE_BYTES: 524288000
  depends_on:
    postgres:
      condition: service_healthy
    minio:
      condition: service_healthy

collaboration-service:
  build:
    context: ./backend
    args:
      APP: collaboration-service
  restart: unless-stopped
  env_file: ./backend/.env
  environment:
    NODE_ENV: production
    REDIS_HOST: redis
    REDIS_PORT: 6379
    COLLAB_SERVICE_PORT: 5003
    BOARDS_SERVICE_URL: boards-service:5002
    MAX_BOARD_ELEMENTS: 10000
  dns:
    - 127.0.0.11
    - 8.8.8.8
    - 1.1.1.1
  depends_on:
    redis:
      condition: service_healthy
    boards-service:
      condition: service_started

api-gateway-1:
  <<: *api-gateway-base
api-gateway-2:
  <<: *api-gateway-base
api-gateway-3:
  <<: *api-gateway-base

nginx:

```

```

image: nginx:alpine
restart: unless-stopped
ports:
  - "4000:80"
volumes:
  - ./docker/nginx/nginx.conf:/etc/nginx/nginx.conf:ro
depends_on:
  - api-gateway-1
  - api-gateway-2
  - api-gateway-3
  - collaboration-service

minio:
image: minio/minio:latest
command: server /data --console-address ":9001"
environment:
  MINIO_ROOT_USER: ${MINIO_ROOT_USER:-minioadmin}
  MINIO_ROOT_PASSWORD: ${MINIO_ROOT_PASSWORD:-minioadmin12345}
ports:
  - "9000:9000"
  - "9001:9001"
volumes:
  - minio-data:/data
healthcheck:
  test: ["CMD", "curl", "-f", "http://localhost:9000/minio/health/live"]
  interval: 10s
  timeout: 3s
  retries: 5
restart: unless-stopped

frontend:
build:
  context: ./frontend
  args:
    NEXT_PUBLIC_API_URL: http://localhost:4000
    API_URL: http://nginx:80
restart: unless-stopped
environment:
  NODE_ENV: production
  PORT: 3000
  HOSTNAME: 0.0.0.0
  API_URL: http://nginx:80
  NEXT_PUBLIC_API_URL: http://localhost:4000
ports:
  - "3000:3000"
depends_on:
  - nginx

volumes:
  postgres_data:
  minio-data:

```

Листинг 4.1 – Фрагмент конфигурации docker-compose.yml

В блоке x-api-gateway-base описан общий шаблон для трех инстансов шлюза API. Шаблон содержит конфигурацию сборки из общего Dockerfile с параметром APP=api-gateway, политику автоматического перезапуска, подключение файла переменных окружения и блок environment с адресами внутренних микросервисов и параметрами доступа к объектному хранилищу. Три инстанса api-gateway-1, api-

gateway-2 и api-gateway-3 повторно используют этот шаблон через YAML-якорь, что исключает дублирование конфигурации.

Контейнеры хранилищ запускаются на основе официальных образов: postgres:16-alpine для реляционной СУБД, redis:7-alpine для кэш-хранилища и minio/minio:latest для объектного хранилища. Для контейнера PostgreSQL заданы переменные POSTGRES_USER, POSTGRES_PASSWORD и POSTGRES_DB, подключены постоянный том postgres_data и файл инициализации init.sql, создающий вторую логическую базу nova_boards. Для контейнера MinIO подключен том minio-data и задана команда запуска server с указанием порта административной консоли.

Четыре микросервиса – users-service, auth-service, boards-service и collaboration-service – собираются из общего Dockerfile с параметром APP, соответствующим имени микросервиса. Каждый микросервис обращается к необходимым хранилищам данных и доступен остальным сервисам по имени контейнера во внутренней сети.

Контейнер балансировщика nginx запускается на основе образа nginx:alpine, прокидывает порт 4000 наружу и монтирует внешний конфигурационный файл docker/nginx/nginx.conf, в котором описаны правила распределения REST-запросов между тремя инстансами шлюза API и маршрутизация WebSocket-соединений на сервис совместного редактирования.

Контейнер клиентского приложения собирается из каталога frontend, прокидывает порт 3000 наружу и обращается к балансировщику nginx по внутренней сети. Зависимость depends_on на nginx гарантирует, что клиентское приложение запускается только после готовности балансировщика и серверной части системы.

В нижней части конфигурации отдельно описаны постоянные тома postgres_data и minio-data, которые подключаются к соответствующим контейнерам и сохраняются независимо от их перезапуска. Все контейнеры по умолчанию помещаются в одну внутреннюю сеть Docker Compose, что обеспечивает их взаимную доступность по именам сервисов.

Таким образом, контейнеризация разрабатываемой системы реализована на основе Docker Compose и предусматривает запуск четырех микросервисов, трех инстансов шлюза API, балансировщика нагрузки, клиентского приложения, реляционной базы данных, кэш-хранилища и объектного хранилища в отдельных контейнерах. Такая архитектура обеспечивает изоляцию компонентов, согласованный порядок их запуска, сохранность данных за счет использования постоянных томов, а также удобную передачу параметров конфигурации через переменные окружения. Запуск всей системы одной командой `docker compose up -d --build` повышает воспроизводимость развертывания и создает устойчивую основу для дальнейшей эксплуатации и развития программной системы.

4. РЕЗУЛЬТАТЫ ТЕСТИРОВАНИЯ

В рамках проверки разработанной системы было проведено тестирование основных компонентов приложения: серверной части, пользовательского интерфейса, бизнес-процессов и механизмов разграничения прав доступа. Целью тестирования являлась проверка соответствия реализованной системы функциональным требованиям, сформулированным в техническом задании, а также выявление ошибок, которые могли бы повлиять на работоспособность приложения при демонстрации и эксплуатации.

Тестирование проводилось по принципу проверки фактического поведения системы относительно ожидаемого результата. Для каждого тестового сценария фиксировались тип объекта проверки, конкретный объект проверки, содержание теста, ожидаемый результат, фактический результат, выявленные дефекты и выполненные исправления. Такой подход позволяет рассматривать тестирование не только как формальную проверку, но и как протокол подтверждения работоспособности разработанной информационной системы.

Сводная таблица отражает общий результат проверки системы по основным направлениям. В нее включены три ключевые группы тестов: серверная часть, пользовательский интерфейс и бизнес-процессы. Такое разделение позволяет отдельно оценить техническую корректность серверной логики, удобство и работоспособность клиентского приложения, а также выполнение целевых пользовательских сценариев.

Из таблицы 4.1 видно, что часть тестов была пройдена сразу, а часть – после внесения исправлений. Это является нормальным результатом для этапа отладки программного продукта, поскольку в процессе тестирования выявляются не только критические ошибки, но и недочеты в обработке сообщений, отображении интерфейса и последовательности выполнения сценариев. После исправлений все тесты получили положительный статус.

Таблица 4.1 – Сводные результаты тестирования

Категория тестирования	Тип объекта проверки	Что именно проверяется	Количество тестов	Успешно пройдено	Пройдено после исправлений	Не пройдено	Итог
Серверная часть	Автоматизированные наборы юнит-тестов Jest в микросервисах NestJS	Сервис пользователей и журнала действий, сервис аутентификации с одноразовыми кодами по электронной почте и OAuth-входом через Google и GitHub, фильтр исключений и guards шлюза API, сервис совместного редактирования с обработчиком AI-инструментов, ограничителем частоты сообщений и валидацией WebSocket-payload	11	9	2	0	Успешно
Клиентская часть	Автоматизированные наборы юнит-тестов Jest сервисов и утилит веб-приложения	Сервисы аутентификации, профиля пользователя, рабочих пространств, досок, клиента совместного редактирования по WebSocket, AI-ассистента и загрузки изображений, набор административных сервисов, общие модули конфигурации axios и серверного прокси, утилиты движка холста	18	15	3	0	Успешно

Таблица 4.1 – Окончание

Категория тестирования	Тип объекта проверки	Что именно проверяется	Количество тестов	Успешно пройдено	Пройдено после исправлений	Не пройдено	Итог
Бизнес-процессы	Сквозные пользовательские и административные сценарии	Регистрация и аутентификация по одноразовому коду; OAuth-вход через Google и GitHub с привязкой и отвязкой провайдеров; редактирование профиля пользователя; создание и удаление рабочего пространства; приглашение участника по invite-коду; передача владения пространством и выход участника; создание, переименование, удаление досок и добавление в избранное; совместное редактирование интерактивной доски двумя пользователями; загрузка изображений на холст; взаимодействие с AI-ассистентом и выполнение AI-инструментов; разграничение прав доступа между ролями USER, MANAGER, ADMIN и SUPER_ADMIN; административные операции с пользователями, пространствами и просмотр журнала аудита	12	10	2	0	Успешно
Итого	Система в целом	Автоматизированные наборы юнит-тестов микросервисов и веб-клиента, а также сквозные пользовательские и административные сценарии работы с системой	41	34	7	0	Система прошла тестирование

По результатам сводной проверки (таблица 4.1) можно сделать вывод, что система реализует основные функции, необходимые для демонстрации курсового проекта. Серверная часть корректно обрабатывает запросы, пользовательский интерфейс обеспечивает доступ к основным операциям, а бизнес-процессы выполняются в логически завершенной последовательности.

Отдельно следует отметить, что неуспешных тестов после исправления выявленных дефектов не осталось. Это означает, что система может считаться работоспособной в рамках заявленного функционала, однако при дальнейшем развитии проекта целесообразно расширить тестирование за счет нагрузочных тестов, проверки безопасности и автоматизированного тестирования API.

Далее в таблице 4.2 представлены результаты тестирования серверной части приложения. Основное внимание уделялось гвардам и фильтрам шлюза API, сервисам аутентификации, пользователей, журнала действий, совместного редактирования и AI-ассистента. Для каждого теста указан тип объекта проверки: гвард, фильтр исключений, метод сервиса или WebSocket-событие.

Проверка серверной части особенно важна, поскольку именно серверные сервисы отвечают за обработку пользовательских запросов, аутентификацию, разграничение прав, поддержание состояния совместного редактирования и взаимодействие с большой языковой моделью. Даже при корректном пользовательском интерфейсе ошибки на серверной стороне могут привести к невозможности входа в систему, рассинхронизации досок у разных участников или некорректному выполнению инструментов AI-ассистента.

Таблица 4.2 – Протокол тестирования серверной части

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
1	Guard шлюза API	JwtAuthGuard	Проверка подписи и срока действия токена доступа из cookie	Запрос с валидным JWT пропускается, с истёкшим или подделанным – возвращается отказ в доступе	Корректно: проверка подписи и срока работает по обоим сценариям	Дефекты не выявлены	Пройден
2	Guard шлюза API	RolesGuard	Проверка иерархии ролей USER < MANAGER < ADMIN < SUPER_ADMIN против декоратора @Roles на методе контроллера	Запрос отклоняется, если роль пользователя ниже минимально допустимой; в остальных случаях пропускается	Корректно для всех четырёх уровней	Дефекты не выявлены	Пройден

Таблица 4.2 – Продолжение

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
3	Фильтр исключений	AllExceptionHandler	Преобразование исключений в HTTP-ответы с корректными кодами	HttpException возвращает заданный код, gRPC-ошибки транслируются в соответствующие HTTP-коды, неизвестные – в 500	Корректно	Дефекты не выявлены	Пройден
4	Сервисный метод	AuthService.sendAuthCode()	Отправка одноразового кода подтверждения на электронную почту	Код сохраняется в Redis по ключу auth:code:{email} с TTL и отправляется через MailService	Корректно	Дефекты не выявлены	Пройден
5	Сервисный метод	AuthService.verifyAuthCode()	Проверка одноразового кода с истёкшим сроком действия	Возвращается понятное сообщение об истечении кода и предложении запросить новый	Изначально возвращалось обобщённое сообщение «неверный код» без уточнения причины	Уточнено сообщение о причине отказа, добавлена обработка случая истечения TTL отдельно от неправильного значения	Пройден после исправления
6	Сервисный метод	UserService.create()	Назначение роли при создании учётной записи	Первой созданной в системе учётной записи назначается роль SUPER_ADMIN, всем последующим – USER	Корректно	Дефекты не выявлены	Пройден
7	Сервисный метод	UserService.updateRole()	Запрет понижения собственной роли администратором	Запрос отклоняется с ошибкой 403, если actor пытается изменить роль самому себе	Корректно	Дефекты не выявлены	Пройден
8	Сервисный метод	AuditLogService.create()	Запись административного действия в журнал	В таблицу audit_logs сохраняется запись с actorId, actorEmail, action, targetType, targetId и details	Корректно	Дефекты не выявлены	Пройден

Таблица 4.2 – Окончание

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
9	WebSocket-сервис	CollaborationService — событие <code>element:create</code>	Публикация события создания элемента всем участникам доски	Все клиенты, открывшие доску, получают событие, элемент появляется в их локальном состоянии в один и тот же момент	Изначально событие приходило только инициатору; остальные участники получали состояние только после полного переподключения	Исправлен порядок вызовов: сначала сохранение через сервис досок по gRPC, затем broadcast через Redis-адаптер Socket.IO	Пройден после исправления
10	AI-сервис	AIService.executeTool()	Разбор и выполнение последовательности <code>tool_calls</code> языковой модели	Каждый вызов диспетчеризуется в соответствующий обработчик (<code>create_text_element</code> , <code>create_sticky</code> , <code>create_shape</code> , <code>create_connector</code> , <code>update/move/delete_element</code>); при ошибке выполнения публикуется событие <code>tool_failed</code>	Корректно	Дефекты не выявлены	Пройден

По результатам тестирования серверной части установлено, что основные backend-компоненты выполняют свои функции корректно. Гварды и фильтры шлюза API, сервисы аутентификации, пользователей и журнала действий, сервис совместного редактирования и сервис AI-ассистента обеспечивают ожидаемое поведение системы. Также подтверждена корректность взаимодействия между контроллерами, сервисами и репозиториями TypeORM.

Выявленные ошибки не относились к критическим сбоям бизнес-логики. Основные замечания были связаны с понятностью сообщения об истечении срока действия одноразового кода и порядком публикации событий совместного редактирования. После исправления этих недостатков серверная часть может считаться готовой к демонстрации и дальнейшему расширению.

В таблице 4.3 представлены результаты тестирования пользовательского интерфейса. В качестве объектов проверки рассматривались отдельные UI-экраны, формы ввода, кнопки, инструменты холста и плавающие виджеты совместной

работы. Такой подход позволяет проверить не только наличие интерфейсных элементов, но и корректность их поведения при выполнении пользовательских действий.

Пользовательский интерфейс является важной частью системы, поскольку именно через него пользователь взаимодействует с функциями совместной работы. Поэтому проверялись посадочная страница и сценарии аутентификации, личный кабинет и взаимодействие со списком рабочих пространств, страницы досок и панель инструментов холста, а также плавающие виджеты совместного редактирования и чат AI-ассистента.

Таблица 4.3 – Протокол тестирования пользовательского интерфейса

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
1	UI-экран	Посадочная страница приложения	Открытие стартовой страницы и переход к аутентификации	Отображаются компоненты Header, Hero, Templates, SuccessStory и Footer, доступна кнопка входа	Корректно	Дефекты не выявлены	Пройден
2	UI-форма	Компонент EmailStep формы аутентификации	Ввод корректного адреса электронной почты и отправка запроса	Серверу отправляется запрос на отправку одноразового кода, пользователь переходит к компоненту OTPStep	Корректно	Дефекты не выявлены	Пройден
3	UI-форма / валидация	Поле e-mail в компоненте EmailStep	Ввод некорректного формата электронной почты	Поле подсвечивается ошибкой и выводится понятное сообщение о требуемом формате	Изначально сообщение об ошибке появлялось только после потери фокуса, а не при попытке отправки формы	Добавлена валидация при сабмите формы и подсветка проблемного поля	Пройден после исправления
4	UI-форма	Компонент OTPStep с компонентами OTPInputGroup и OTPTimer	Ввод полученного одноразового кода и автоматический переход между ячейками сетки	Курсор автоматически переходит к следующей ячейке, по таймеру доступна повторная отправка кода	Корректно	Дефекты не выявлены	Пройден

Таблица 4.3 – Продолжение

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
5	UI-сценарий	Кнопка социального входа компонента OAuthProviderList	Запуск аутентификации через провайдера Google	Пользователь переходит на страницу провайдера, после подтверждения возвращается в приложение с выпущенным токеном	Корректно	Дефекты не выявлены	Пройден
6	UI-экран	Личный кабинет пользователя	Отображение списка рабочих пространств в компоненте SpacesList после успешного входа	Боковая панель Aside отображает все пространства пользователя, основная область содержит список с возможностью создания нового	Корректно	Дефекты не выявлены	Пройден
7	UI-форма / модалка	Компонент CreateSpaceForm	Создание нового рабочего пространства с пустым именем	Кнопка отправки блокируется или выводится сообщение об обязательности заполнения поля имени	Изначально при пустом имени модалка отправляла запрос на сервер, после чего возвращалась ошибка	Добавлена клиентская валидация поля имени с подсветкой и блокировкой кнопки до его заполнения	Пройден после исправления
8	UI-экран	Страница интерактивной доски	Открытие доски с панелью инструментов в BoardLeftToolBar, верхней панелью BoardTopBar и виджетами совместной работы	Холст отображается, доступны инструменты выделения, карандаша, текста, стикера и фигуры, отображаются виджеты OnlineUsersWidget, ConnectionStatusWidget и ZoomWidget	Корректно	Дефекты не выявлены	Пройден
9	UI-инструменты	Инструменты создания элементов холста	Размещение текстового блока, стикера, графической фигуры и соединительной стрелки в виде кривой Безье	Элементы добавляются на холст в позицию указателя и доступны для последующего выделения и редактирования	Изначально подпанель StickyColorBar при первом открытии сбрасывала выбранный пользователем цвет на значение по умолчанию	Исправлено сохранение последнего выбранного цвета стикера в состоянии инструмента	Пройден после исправления

Таблица 4.3 – Окончание

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
10	UI-виджет	Чат AI-ассистента (AiChatWidget)	Отправка текстового запроса и приём потокового ответа модели вместе с событиями создания элементов на холсте	В чате последовательно отображаются текстовые части ответа, на холсте у всех участников появляются построенные AI-ассистентом элементы	Корректно	Дефекты не выявлены	Пройден

По результатам тестирования пользовательского интерфейса можно сделать вывод, что основные экраны и элементы управления работают корректно. Пользователь может открыть приложение, пройти регистрацию или социальный вход, попасть в личный кабинет, создать рабочее пространство, открыть интерактивную доску, использовать инструменты холста и обратиться к AI-ассистенту.

Обнаруженные недостатки были связаны преимущественно с удобством интерфейса: моментом срабатывания валидации в формах аутентификации и создания пространства, а также сохранением выбранного цвета стикера. После исправлений интерфейс стал более понятным и устойчивым к типовым действиям пользователя. Это позволяет считать клиентскую часть пригодной для демонстрации и практического использования в рамках курсового проекта.

В таблице 4.4 приведены результаты проверки сквозных бизнес-процессов. В отличие от тестирования отдельных серверных методов или интерфейсных экранов, здесь проверяется полный сценарий работы пользователя или администратора: от первого действия до получения конечного результата. Такой подход позволяет убедиться, что отдельные компоненты системы не только работают сами по себе, но и корректно взаимодействуют друг с другом.

Таблица 4.4 – Протокол тестирования сквозных бизнес-процессов

№	Тип объекта проверки	Объект проверки	Содержание проверки	Ожидаемый результат	Фактический результат	Дефекты / исправления	Статус
1	Бизнес-сценарий	Регистрация и аутентификация	Полный сценарий: открытие приложения → ввод адреса электронной почты → получение одноразового кода → подтверждение → вход в систему	Пользователь создаёт учётную запись, серверная часть выпускает подписанный токен, пользователь получает доступ к функциям системы	Сценарий выполняется корректно	Дефекты не выявлены	Пройден
2	Бизнес-сценарий	Создание рабочего пространства и приглашение участника	Вход → создание пространства → копирование кода приглашения → второй пользователь присоединяется к пространству по коду	Второй пользователь появляется в списке участников пространства и получает доступ к его доскам	Сценарий выполняется успешно	Дефекты не выявлены	Пройден
3	Бизнес-сценарий	Совместное редактирование интерактивной доски	Два пользователя одновременно открывают доску → один размещает элементы на холсте → второй наблюдает их появление и сам выполняет действия	Изменения у обоих участников отображаются в режиме реального времени, курсоры и активные участники видны через виджет OnlineUsersWidget	Изначально при кратковременном разрыве соединения состояние доски не синхронизировалось сразу после переподключения	Добавлена автоматическая ресинхронизация состояния доски при восстановлении и WebSocket-соединения	Пройден после исправления
4	Бизнес-сценарий	Взаимодействие с AI-ассистентом	Открытие чата AI-ассистента на доске → отправка текстового запроса → получение ответа модели и автоматическое построение элементов на холсте	В чате отображается потоковый ответ модели, на холсте у всех участников появляются текстовые блоки, стикеры и соединительные стрелки в виде кривых Безье	Изначально при ошибке выполнения отдельного инструмента AI-ассистент не сообщал об этом пользователю	Добавлена публикация события tool_failed и отображение уведомления о неудачном вызове в чате	Пройден после исправления
5	Бизнес-сценарий / безопасность	Разграничение прав доступа	Обычный пользователь пытается обратиться к административным эндпойнтам, администратор изменяет роль и блокирует учётную запись	Запросы обычного пользователя отклоняются с кодом отказа в доступе, действия администратора сохраняются в журнал audit_logs	Корректно	Дефекты не выявлены	Пройден

По результатам проверки сквозных бизнес-процессов установлено, что основные пользовательские и административные сценарии системы выполняются корректно. Пользователи могут проходить аутентификацию, создавать рабочие

пространства, приглашать в них других участников, совместно редактировать интерактивные доски и взаимодействовать с AI-ассистентом. Администраторы могут управлять пользователями и пространствами, а действия административного уровня фиксируются в журнале аудита.

Выявленные ошибки относились к взаимодействию компонентов в пограничных ситуациях – ресинхронизации состояния доски при разрыве соединения и обработке ошибок выполнения отдельных инструментов AI-ассистента. После их исправления сквозные сценарии выполняются устойчиво, что подтверждает работоспособность системы в рамках заявленного функционала.

В таблице 4.5 приведён перечень всех дефектов, выявленных в ходе тестирования серверной части, пользовательского интерфейса и сквозных бизнес-процессов. Для каждого дефекта указаны тип объекта, конкретный объект с обнаруженной ошибкой, описание проявления, выполненное исправление и итоговый результат повторной проверки. Такая таблица обеспечивает прозрачность процесса отладки и позволяет в одном месте увидеть, какие именно правки были выполнены в ходе тестирования.

Большинство выявленных дефектов относилось к некритическим ошибкам: понятности сообщений, моментам срабатывания валидации, сохранению состояния отдельных элементов интерфейса и порядку публикации событий совместного редактирования. Тем не менее такие ошибки важны, поскольку они влияют на удобство работы и согласованность поведения системы в пограничных ситуациях.

Таблица 4.5 – Выявленные дефекты и выполненные исправления

№	Тип объекта проверки	Объект проверки	Описание дефекта	Проявление	Выполненное исправление	Результат
1	Сервисный метод	AuthService.verify AuthCode()	Обобщённое сообщение об ошибке при истечении срока действия одноразового кода	Пользователь не различал ситуации «неверный код» и «срок действия истёк»	Уточнено сообщение с разделением случаев и обработка истечения TTL отдельно от неправильного значения	Исправлено

Таблица 4.5 – Окончание

№	Тип объекта проверки	Объект проверки	Описание дефекта	Проявление	Выполненное исправление	Результат
2	WebSocket-сервис	CollaborationService — событие element:create	Событие создания элемента приходило только инициатору	Другие участники доски не видели появление нового элемента до переподключения	Изменён порядок вызовов: сначала сохранение элемента в сервисе досок по gRPC, затем broadcast через Redis-адаптер Socket.IO	Исправлено
3	UI-форма / валидация	Поле e-mail в компоненте EmailStep	Сообщение об ошибке формата появлялось только после потери фокуса, не при отправке формы	Пользователь нажимал кнопку отправки и не получал понятной обратной связи	Добавлена валидация при сабмите и подсветка проблемного поля	Исправлено
4	UI-форма / модалка	Компонент CreateSpaceForm	Поле имени пространства принимало пустое значение и отправляло запрос на сервер	Сервер возвращал ошибку, пользователь видел общую ошибку без подсказки	Добавлена клиентская валидация поля имени с подсветкой и блокировкой кнопки отправки	Исправлено
5	UI-инструменты	Подпанель StickyColorBar	Выбранный пользователем цвет стикера сбрасывался при следующем открытии инструмента	Каждый новый стикер создавался с цветом по умолчанию вместо ранее выбранного	Добавлено сохранение последнего выбранного цвета в состоянии инструмента	Исправлено
6	Бизнес-сценарий	Совместное редактирование при разрыве WebSocket-соединения	После кратковременного разрыва соединения состояние доски не синхронизировалось с актуальным	Пользователь после переподключения видел устаревшее состояние доски	Добавлена автоматическая ресинхронизация состояния доски при восстановлении соединения	Исправлено
7	Бизнес-сценарий	AI-ассистент — обработка ошибок выполнения инструментов	При ошибке выполнения отдельного инструмента пользователь не получал уведомления	Часть запрошенных элементов не появлялась на холсте без объяснения причины	Добавлена публикация события tool_failed и отображение уведомления о неудачном вызове в чате	Исправлено

По результатам анализа дефектов можно сделать вывод, что в системе не были выявлены ошибки, полностью блокирующие работу приложения. Все обнаруженные проблемы были устранены, после чего соответствующие тесты были выполнены повторно с положительным результатом.

Исправление выявленных недостатков повысило устойчивость и удобство использования системы. Особенно важными являются доработки, связанные с поведением совместного редактирования и обработкой ошибок AI-ассистента,

поскольку они влияют не только на интерфейс, но и на надёжность пользовательского сценария целиком.

Таблица 4.6 содержит итоговую оценку основных критериев работоспособности системы. В ней отражены результаты проверки тех функций, которые являются обязательными для данной курсовой работы: аутентификации, разграничения прав доступа, совместного редактирования досок, взаимодействия с AI-ассистентом, административного сопровождения и контейнеризованного развёртывания.

Итоговая оценка позволяет сделать общий вывод о готовности системы к демонстрации. В отличие от предыдущих таблиц, здесь не фиксируются отдельные тестовые шаги, а обобщается результат проверки системы в целом по ключевым критериям её работоспособности.

Таблица 4.6 – Итоговая оценка результатов тестирования

Критерий	Тип объекта проверки	Что именно проверяется	Результат
Корректность регистрации и аутентификации	Сервис аутентификации, UI-формы EmailStep и OTPStep, Redis	Отправка одноразового кода на электронную почту, его проверка из Redis, выпуск подписанного токена JWT, социальный вход через Google и GitHub	Подтверждена
Корректность работы с рабочими пространствами и досками	Сервис досок, UI-страницы списка пространств и досок, PostgreSQL	Создание рабочих пространств, приглашение участников по коду, создание и редактирование интерактивных досок, добавление в избранное	Подтверждена
Корректность совместного редактирования	Сервис совместного редактирования, Redis-адаптер Socket.IO, клиентский WS-сервис	Синхронизация элементов холста, отображение курсоров и активных участников, автоматическое восстановление состояния после разрыва соединения	Подтверждена
Корректность работы AI-ассистента	Сервис AI-ассистента, языковая модель DeepSeek, набор инструментов	Подготовка контекста доски, потоковое обращение к модели, разбор вызовов инструментов и их выполнение через сервис досок	Подтверждена
Корректность разграничения прав доступа	JwtAuthGuard, RolesGuard, сервисы, audit_logs	Проверка подписи и срока действия токена, иерархия системных ролей USER-MANAGER-ADMIN-SUPER_ADMIN, проверки SpaceRole, фиксация действий администраторов в журнале	Подтверждена
Корректность хранения данных	PostgreSQL, Redis, MinIO, тома Docker	Сохранение учётных записей и журнала действий в базе nova_users, пространств, досок и элементов холста в базе nova_boards, файлов изображений в MinIO, состояния совместного редактирования в Redis	Подтверждена

Таблица 4.6 – Окончание

Критерий	Тип объекта проверки	Что именно проверяется	Результат
Корректность контейнеризованного запуска	Docker Compose, nginx, набор контейнеров системы	Запуск всех микросервисов, балансировщика nginx, контейнеров хранилищ и клиентского приложения одной командой <code>docker compose up -d --build</code>	Подтверждена

По результатам итоговой оценки можно сделать вывод, что разработанная система удовлетворяет всем ключевым критериям работоспособности, сформулированным в техническом задании. Аутентификация и разграничение прав доступа выполняются корректно, совместное редактирование досок и взаимодействие с AI-ассистентом работают согласованно у всех участников, хранение данных обеспечивается тремя выделенными хранилищами, а развёртывание выполняется одной командой Docker Compose.

По результатам проведённого тестирования установлено, что разработанная система совместной работы с интерактивными досками корректно реализует основные функции, предусмотренные техническим заданием. Из 41 проведённого набора тестов 34 были пройдены без замечаний, 7 — после внесения исправлений; неуспешных тестов не осталось. Все семь зафиксированных дефектов относились к некритическим ситуациям и были устранены, после чего повторное выполнение соответствующих тестов прошло без замечаний. Итоговая оценка по ключевым критериям, включающим аутентификацию, работу с пространствами и досками, совместное редактирование, AI-ассистент, разграничение прав, хранение данных и контейнеризацию, подтверждает готовность системы к демонстрации в составе курсового проекта.

5. ДОКУМЕНТАЦИЯ И ВВОД В ЭКСПЛУАТАЦИЮ

Для разработанной системы совместной работы с интерактивными досками подготовлен комплект эксплуатационной документации, включающий текстовое описание системы, руководство пользователя, руководство администратора, описание программного интерфейса REST API и инструкцию по запуску.

Текстовое описание системы предназначено для общего представления о разработанной системе. В данном документе кратко раскрываются назначение системы, роли пользователей, основные бизнес-процессы, модель данных, используемые диаграммы, а также место модуля искусственного интеллекта в общей архитектуре решения.

Руководство пользователя предназначено для описания порядка работы обычного пользователя с системой. В нём отражаются основные пользовательские операции, включая регистрацию по одноразовому коду, вход через OAuth-провайдеров, создание рабочих пространств и интерактивных досок, совместное редактирование, работу с инструментами холста и обращение к AI-ассистенту.

Руководство администратора предназначено для описания функций сопровождения и управления системой. В документе рассматриваются вопросы управления учётными записями, ролями, модерации досок и пространств, просмотра журнала действий, мониторинга активных сессий совместного редактирования, а также резервного копирования и восстановления данных.

Описание API предназначено для представления программного интерфейса системы. В данном документе приводится перечень основных REST-ресурсов, а также примеры запросов и ответов, необходимых для понимания логики взаимодействия клиентской и серверной частей приложения.

Инструкция запуска предназначена для описания порядка развёртывания и первичного запуска системы. В ней отражаются этапы настройки переменных окружения, запуска контейнеров через Docker Compose, подключения баз данных, объектного хранилища, Redis и проверки работоспособности разработанного программного решения.

5.1 Текстовое описание системы

Разработанная информационная система представляет собой клиент-серверное веб-приложение для совместной работы с интерактивными досками. Основная задача системы заключается в том, чтобы предоставить пользователям инструмент для совместного создания и редактирования визуального контента в режиме реального времени, организации досок в рамках рабочих пространств, приглашения участников и автоматизации части работы с помощью встроенного AI-ассистента.

Система построена по микросервисному принципу. На клиентском уровне располагается веб-приложение на платформе Next.js, обеспечивающее серверный рендеринг страниц и взаимодействие с серверной частью по REST API и протоколу WebSocket. Серверный уровень состоит из пяти микросервисов на платформе NestJS: шлюза API `api-gateway`, сервиса аутентификации `auth-service`, сервиса пользователей и журнала действий `users-service`, сервиса досок и рабочих пространств `boards-service`, сервиса совместного редактирования `collaboration-service`. Между микросервисами организовано межсервисное взаимодействие по протоколу gRPC. На уровне хранения данных используется реляционная СУБД PostgreSQL с двумя логически разделёнными базами `nova_users` и `nova_boards`, кэш-хранилище Redis для одноразовых кодов, refresh-токенов и состояния совместного редактирования, а также объектное хранилище MinIO для изображений, размещаемых на досках.

В системе предусмотрено разграничение прав доступа по ролям, реализованное с помощью `JwtAuthGuard` и `RolesGuard` на уровне шлюза API. Неавторизованный пользователь может открыть посадочную страницу системы и перейти к форме аутентификации. Авторизованный пользователь с ролью `USER` может создавать рабочие пространства и доски, приглашать в них других участников, совместно редактировать холст, загружать изображения и взаимодействовать с AI-ассистентом. Расширенная роль `MANAGER` используется как промежуточный уровень иерархии. Пользователь с ролью `ADMIN` получает доступ к административной панели: управлению учётными записями, модерации

досок и пространств, просмотру журнала действий и мониторингу активных сессий. Пользователь с ролью SUPER_ADMIN автоматически назначается первой созданной учётной записи и обладает всеми правами администратора с дополнительной возможностью назначения и снятия роли ADMIN.

Основными бизнес-процессами системы являются аутентификация по одноразовому коду или OAuth-провайдеру, создание и управление рабочими пространствами, приглашение участников по invite-коду, передача владения пространством, создание и совместное редактирование интерактивных досок, загрузка изображений на холст, взаимодействие с AI-ассистентом, а также административные операции по управлению пользователями, модерации содержимого и просмотру журнала. Каждый из этих процессов был описан ранее с помощью диаграмм вариантов использования, диаграмм активности, диаграммы классов и диаграммы развёртывания.

Особое место в системе занимает модуль искусственного интеллекта. Он реализован в составе collaboration-service и обеспечивает обращение к большой языковой модели через единый внешний API. При получении запроса от пользователя сервис AI-ассистента подготавливает динамический контекст из элементов текущей доски, отправляет запрос модели в потоковом режиме и по мере поступления tool-вызовов выполняет операции создания, обновления, перемещения и удаления элементов на холсте через сервис досок. Все изменения, выполненные ассистентом, синхронизируются между участниками доски через WebSocket.

Модель данных системы включает несколько групп сущностей. Первая группа связана с пользователями и журналом действий: таблицы users и audit_logs в базе nova_users. Вторая группа описывает рабочие пространства и участников: таблицы spaces и space_members. Третья группа относится к доскам и их содержимому: таблицы boards, canvas_elements и favorites в базе nova_boards. Четвёртая группа предназначена для работы с изображениями: таблица ассетов и связанные с ней объекты в MinIO. Такая структура позволяет хранить как

метаданные системы, так и пользовательский контент, обеспечивая разделение между учётными записями и содержимым досок.

Таким образом, текстовое описание системы фиксирует её назначение, состав пользователей, основные процессы, архитектуру, модель данных и роль искусственного интеллекта. Этот документ необходим для общего понимания системы перед её использованием, администрированием или развёртыванием.

5.2 Руководство пользователя

Руководство пользователя предназначено для описания действий обычного пользователя, который работает с системой совместной работы с интерактивными досками через веб-интерфейс. Пользователь не должен выполнять настройку серверной части, изменять конфигурацию микросервисов или работать с базой данных напрямую. Его задача заключается в том, чтобы войти в систему, создать или открыть рабочее пространство, организовать совместную работу над доской и при необходимости воспользоваться AI-ассистентом.

Перед началом работы пользователь открывает веб-приложение в браузере. На посадочной странице отображаются разделы Header, Hero, Templates, SuccessStories и форма перехода к аутентификации. Если пользователь не имеет учётной записи, он переходит на страницу аутентификации и в форме EmailStep вводит адрес электронной почты. После отправки формы система генерирует одноразовый код подтверждения, сохраняет его в Redis по ключу `auth:code:{email}` и отправляет на указанный адрес. Пользователь вводит полученный код в форме OTPStep с компонентами OTPInputGroup и OTPTimer.

Альтернативно пользователь может воспользоваться кнопкой социального входа компонента OAuthProviderList и выполнить вход через провайдеров Google или GitHub. В этом случае серверная часть запрашивает у внешнего провайдера данные профиля и автоматически создаёт учётную запись или связывает существующую по адресу электронной почты. Первая учётная запись, созданная в системе, автоматически получает роль SUPER_ADMIN; все последующие — роль USER.

После успешной аутентификации система устанавливает в браузере `httpOnly-cookies accessToken` и `refreshToken`, после чего пользователь переходит в личный кабинет. В личном кабинете отображается боковая панель `Aside` со списком рабочих пространств пользователя. Пользователь может создать новое рабочее пространство через компонент `CreateSpaceForm` или вступить в существующее по `invite`-коду, перейдя по ссылке-приглашению. После создания пространства пользователь становится его владельцем (`OWNER`) и может приглашать в него других участников.

Внутри рабочего пространства пользователь создаёт интерактивную доску с помощью соответствующего действия. После открытия доски пользователь видит холст с панелью инструментов `BoardLeftToolbar`, верхней панелью `BoardTopBar` и панелью присутствия `PresencePanel` со списком активных участников. Доступны инструменты выделения, карандаша, текста, стикеров, графических фигур и загрузки изображений. Каждое размещённое или изменённое пользователем действие отправляется на сервер по `WebSocket`-протоколу и синхронизируется со всеми участниками доски в режиме реального времени.

Для совместной работы пользователь может скопировать `invite`-код пространства и передать его второму пользователю. После вступления второй пользователь получает доступ ко всем доскам пространства, его курсор отображается у других участников, а все изменения становятся видимыми всем участникам в реальном времени. При кратковременном разрыве `WebSocket`-соединения состояние доски автоматически ресинхронизируется при переподключении.

Дополнительно пользователь может воспользоваться `AI`-ассистентом, открыв чат компонента `AiChatWidget`. После отправки текстового запроса ассистент в потоковом режиме генерирует ответ и при необходимости размещает или изменяет элементы на холсте: создаёт стикеры с предложенными идеями, размещает текстовые блоки, перемещает уже существующие элементы. Все действия ассистента отображаются у всех участников доски.

Основной порядок работы пользователя с системой представлен в таблице

5.1.

Таблица 5.1 – Порядок работы пользователя с системой

Шаг	Действие пользователя	Результат
1	Открыть веб-приложение в браузере	Отображается посадочная страница системы
2	Перейти к форме аутентификации и ввести адрес электронной почты	На указанный адрес отправляется одноразовый код подтверждения
3	Ввести полученный код в форме ОТР или воспользоваться кнопкой OAuth-входа через Google или GitHub	Создаётся учётная запись и устанавливаются httpOnly-cookies с токенами доступа
4	Перейти в личный кабинет	Открывается боковая панель со списком рабочих пространств пользователя
5	Создать новое рабочее пространство или вступить в существующее по invite-коду	Пользователь становится участником пространства и получает доступ к его доскам
6	Создать новую интерактивную доску внутри пространства	Открывается холст с панелью инструментов и панелью присутствия участников
7	Использовать инструменты холста: текст, стикеры, фигуры, карандаш, загрузка изображений	Элементы появляются на холсте и синхронизируются со всеми участниками в реальном времени
8	Открыть AI-ассистента и отправить текстовый запрос	Ассистент в потоковом режиме генерирует ответ и размещает элементы на холсте
9	Скопировать invite-код и пригласить второго участника	Второй участник получает доступ к пространству и его доскам

Таким образом, руководство пользователя описывает полный сценарий работы с системой: от аутентификации и создания рабочего пространства до совместного редактирования доски и взаимодействия с AI-ассистентом. Все основные операции выполняются через графический интерфейс и не требуют от пользователя специальных знаний в области программирования, баз данных или работы с микросервисными системами.

5.3 Руководство администратора

Руководство администратора предназначено для пользователей с ролями ADMIN и SUPER_ADMIN, которые отвечают за сопровождение системы, контроль её состояния, управление учётными записями и модерацию пользовательского

контента. В отличие от обычного пользователя, администратор имеет доступ не только к функциям работы с собственными досками, но и к административной панели, доступной по адресу `/app/admin`, с разделами обзора, пользователей, досок, пространств, журнала действий, мониторинга активных сессий и состояния системы.

Одной из основных задач администратора является управление пользователями. В разделе `/app/admin/users` администратор может просматривать список всех учётных записей с фильтрами по роли и статусу, видеть статистику регистраций, изменять роли пользователей, блокировать и разблокировать доступ, удалять учётные записи и просматривать историю действий каждого пользователя. Это необходимо для обеспечения безопасности системы, так как функции административного уровня не должны быть доступны обычным пользователям.

В системе используются следующие основные роли, представленные в таблице 5.2.

Таблица 5.2 – Роли пользователей и их назначение

Роль	Назначение
USER	Авторизованный пользователь, который может создавать рабочие пространства и интерактивные доски, приглашать в них участников и совместно работать с холстом, взаимодействовать с AI-ассистентом, загружать изображения и управлять собственным профилем
MANAGER	Расширенная пользовательская роль; обладает теми же правами, что и USER, и используется как промежуточный уровень в иерархии ролей при настройке доступа к расширенным функциям системы
ADMIN	Администратор системы; помимо пользовательских функций имеет доступ к административной панели, может управлять учётными записями (изменение ролей, блокировка, удаление), модерировать доски и
SUPER_ADMIN	Главный администратор системы; автоматически назначается первой созданной учётной записи; обладает всеми правами роли ADMIN и дополнительно может назначать и снимать роль ADMIN с других пользователей

Важной административной функцией является модерация рабочих пространств и досок. В разделе `/app/admin/spaces` администратор может

просматривать список всех пространств системы с количеством участников и досок, удалять пространства и просматривать состав их участников. В разделе `/app/admin/boards` администратор видит список всех досок системы с информацией о владельце, количестве элементов и пространстве, к которому привязана доска, и при необходимости может удалить доску.

Для контроля действий пользователей и состояния системы используется журнал событий `audit_logs`. В разделе `/app/admin/audit` администратор может просматривать записи журнала с фильтрами по типу действия, идентификатору пользователя и временному промежутку. В журнале фиксируются ключевые операции: вход в систему, изменение роли, блокировка и разблокировка пользователей, удаление учётных записей и иные административные действия. Это позволяет анализировать активность пользователей и контролировать корректность эксплуатации системы.

Раздел `/app/admin/realtime` предоставляет администратору возможность мониторинга активных сессий совместного редактирования. В нём отображается список открытых досок с количеством подключённых пользователей, имена участников и время начала сессии. Администратор может принудительно отключить пользователя от доски или разослать broadcast-уведомление всем подключённым клиентам. Раздел `/app/admin/system` отображает состояние ключевых компонентов системы: доступность PostgreSQL, Redis, MinIO, статус микросервисов и параметры окружения.

Отдельное значение имеет резервное копирование. Администратор должен периодически создавать резервные копии обеих баз данных `nova_users` и `nova_boards`, чтобы при сбоях можно было восстановить информацию. В резервные копии должны попадать сведения об учётных записях, журнале действий, рабочих пространствах, досках, элементах холста и связях избранного. Резервные копии рекомендуется хранить в отдельном каталоге `backup`, расположенном на хост-системе и не входящем в Docker-том контейнера базы данных.

При необходимости восстановления администратор останавливает контейнер `postgres`, выбирает актуальную резервную копию и выполняет

восстановление данных через `psql`. После восстановления необходимо проверить работоспособность системы: выполнить вход, открыть личный кабинет, убедиться в наличии рабочих пространств, открыть доску и проверить состояние журнала действий.

В случае аварийной ситуации администратор должен выполнить действия, представленные в таблице 5.3.

Таблица 5.3 – Действия администратора при типовых ситуациях

Ситуация	Действие администратора
Приложение недоступно в браузере	Проверить состояние контейнеров командой <code>docker compose ps</code> , открыть логи контейнеров <code>nginx</code> и <code>api-gateway</code> , убедиться в доступности портов 3000 и 4000
Нет подключения к базе данных	Проверить контейнер <code>postgres</code> , <code>healthcheck pg_isready</code> , переменные окружения в файле <code>.env</code> , имена баз <code>nova_users</code> и <code>nova_boards</code> , логин и пароль
Пользователь не получает одноразовый код	Проверить контейнер <code>MailHog</code> в режиме разработки или настройки <code>SMTP_HOST</code> и <code>SMTP_PORT</code> в файле <code>.env</code> , открыть логи контейнера <code>auth-service</code>
Пользователь не может войти	Проверить наличие учётной записи в таблице <code>users</code> базы <code>nova_users</code> , статус блокировки <code>isBlocked</code> , корректность OAuth-привязки и записи в журнале <code>audit_logs</code>
Совместное редактирование не работает	Проверить контейнер <code>collaboration-service</code> , контейнер <code>redis</code> , конфигурацию <code>upstream ip_hash</code> для <code>WebSocket</code> в файле <code>docker/nginx/nginx.conf</code>
AI-ассистент не отвечает	Проверить переменные <code>AI_API_KEY</code> , <code>AI_BASE_URL</code> и <code>AI_MODEL</code> в файле <code>.env</code> , доступность внешнего API большой языковой модели и логи <code>collaboration-service</code>
Не загружаются изображения на холст	Проверить контейнер <code>minio</code> , переменные <code>MINIO_ENDPOINT</code> , <code>MINIO_ACCESS_KEY</code> , <code>MINIO_SECRET_KEY</code> и <code>MINIO_BUCKET</code> , состояние bucket <code>nova-board-assets</code> и логи <code>boards-service</code>
Данные повреждены или утеряны	Выполнить восстановление из резервной копии для обеих баз <code>nova_users</code> и <code>nova_boards</code> через <code>psql</code> , после чего перезапустить микросервисы

Таким образом, руководство администратора описывает порядок сопровождения системы после её разработки. Администратор обеспечивает безопасность, управляет учётными записями, модерирует пользовательский контент, контролирует журнал действий, мониторит активные сессии совместного редактирования, выполняет резервное копирование и принимает меры при возникновении нештатных ситуаций.

5.4 Описание API

Описание API предназначено для разработчиков и администраторов, которым необходимо понимать, какие программные интерфейсы предоставляет серверная часть системы. Взаимодействие между клиентской и серверной частями реализуется по принципам REST API через шлюз api-gateway, развёрнутый в трёх инстансах за балансировщиком nginx. Клиент отправляет HTTP-запросы на адреса, начинающиеся с префикса /api, а сервер обрабатывает их и возвращает результат в формате JSON либо в виде потока байт для бинарных ресурсов.

Основные группы API связаны с аутентификацией, управлением профилем пользователя, рабочими пространствами, досками, загрузкой ассетов и административными функциями. Для аутентификации поддерживаются два способа: одноразовый код, отправляемый на электронную почту, и OAuth-вход через Google и GitHub. Аутентификация поддерживается через httpOnly-cookies accessToken и refreshToken. Для обычного пользователя доступны запросы получения и обновления профиля, управления пространствами и досками, загрузки изображений на холст и обращения к AI-ассистенту по WebSocket. Для администратора дополнительно доступны запросы управления пользователями, модерации досок и пространств, просмотра журнала действий и мониторинга системы.

Основные ресурсы API представлены в таблице 5.4.

Таблица 5.4 – Основные ресурсы REST API системы

Метод	Ресурс	Назначение	Доступ
POST	/api/auth/send-code	Отправка одноразового кода подтверждения на электронную почту	Guest
POST	/api/auth/verify-code	Проверка одноразового кода и выпуск пары JWT-токенов в cookies	Guest
POST	/api/auth/refresh	Обновление access-токена по refresh-токену из cookie	USER, ADMIN
POST	/api/auth/logout	Завершение сессии и очистка cookies	USER, ADMIN
GET	/api/auth/google	Запуск OAuth-входа через провайдер Google	Guest

Таблица 5.4 – Окончание

Метод	Ресурс	Назначение	Доступ
GET	/api/auth/github	Запуск OAuth-входа через провайдер GitHub	Guest
DELETE	/api/auth/providers/:provider	Отвязка социального провайдера от учётной записи	USER, ADMIN
GET	/api/users/me	Получение профиля текущего пользователя	USER, ADMIN
PATCH	/api/users/me	Обновление профиля (имя, аватар)	USER, ADMIN
GET	/api/spaces	Получение списка рабочих пространств пользователя	USER, ADMIN
POST	/api/spaces	Создание нового рабочего пространства	USER, ADMIN
POST	/api/spaces/join/:code	Вступление в пространство по invite-коду	USER, ADMIN
GET	/api/spaces/:id/members	Получение списка участников пространства	Участник
POST	/api/spaces/:id/leave	Выход из пространства	Участник
POST	/api/spaces/:id/transfer-ownership	Передача владения пространством	OWNER
GET	/api/boards	Получение списка досок пользователя (опционально по пространству)	USER, ADMIN
POST	/api/boards	Создание новой интерактивной доски	USER, ADMIN
GET	/api/boards/:id	Получение метаданных доски с информацией о создателе	USER, ADMIN
POST	/api/boards/:id/favorite	Добавление или удаление доски из избранного	USER, ADMIN
POST	/api/assets/presign	Получение presigned URL для загрузки изображения в MinIO	USER, ADMIN
GET	/api/admin/users	Получение списка всех пользователей системы	ADMIN, SUPER_ADMIN
PATCH	/api/admin/users/:id/role	Изменение роли пользователя	ADMIN, SUPER_ADMIN
PATCH	/api/admin/users/:id/block	Блокировка или разблокировка пользователя	ADMIN, SUPER_ADMIN
GET	/api/admin/audit	Просмотр журнала действий с фильтрами	ADMIN, SUPER_ADMIN
GET	/api/admin/realtime/boards	Список активных досок и сессий совместного редактирования	ADMIN, SUPER_ADMIN

Пример запроса на отправку одноразового кода представлен в листинге 5.1.

```
POST /api/auth/send-code
Content-Type: application/json
```

```
{
  "email": "user@example.com"
}
```

Листинг 5.1 – Запрос на отправку одноразового кода

Пример ответа сервера при успешной отправке кода представлен в листинге 5.2.

```
{
  "message": "Код отправлен на почту",
  "success": true
}
```

Листинг 5.2 – Ответ сервера при успешной отправке кода

Пример запроса на проверку одноразового кода представлен в листинге 5.3.

```
POST /api/auth/verify-code
Content-Type: application/json

{
  "email": "user@example.com",
  "code": "1234567"
}
```

Листинг 5.3 – Запрос на проверку одноразового кода

Пример ответа сервера при успешной аутентификации представлен в листинге 5.4. JWT-токены устанавливаются в httpOnly-cookies accessToken и refreshToken, в теле ответа возвращаются данные пользователя.

```
{
  "user": {
    "id": "1c2d4e8f-1234-...",
    "email": "user@example.com",
    "name": "Иван Иванов",
    "avatar": null,
    "role": "USER",
    "isBlocked": false
  }
}
```

Листинг 5.4 – Ответ сервера при успешной аутентификации

Пример запроса на создание рабочего пространства представлен в листинге 5.5.

```
POST /api/spaces
Content-Type: application/json
Cookie: accessToken=<jwt>

{
  "name": "Команда дизайна"
}
```

Листинг 5.5 – Запрос на создание рабочего пространства

Пример ответа сервера при успешном создании пространства представлен в листинге 5.6.

```
{
  "id": "ws-7af3-...",
  "name": "Команда дизайна",
  "ownerId": "1c2d4e8f-1234-...",
  "inviteCode": "ABCD-EFGH-IJKL",
  "createdAt": "2026-05-28T10:30:00Z"
}
```

Листинг 5.6 – Ответ сервера при успешном создании пространства

Пример запроса на создание новой доски представлен в листинге 5.7.

```
POST /api/boards
Content-Type: application/json
Cookie: accessToken=<jwt>

{
  "name": "Roadmap Q3",
  "spaceId": "ws-7af3-..."
}
```

Листинг 5.7 – Запрос на создание новой интерактивной доски

Пример ответа сервера с метаданными доски представлен в листинге 5.8.

```
{
  "id": "bd-2911-...",
  "name": "Roadmap Q3",
  "spaceId": "ws-7af3-...",
  "createdByUser": {
    "id": "1c2d4e8f-1234-...",
    "name": "Иван Иванов",
    "avatar": null
  },
  "elementsCount": 0,
  "isFavorite": false,
  "createdAt": "2026-05-28T10:35:00Z"
}
```

Листинг 5.8 – Ответ сервера с метаданными доски

Пример запроса на блокировку пользователя со стороны администратора представлен в листинге 5.9.

```
PATCH /api/admin/users/abc-123/block
Content-Type: application/json
Cookie: accessToken=<admin-jwt>

{
  "blocked": true
}
```

Листинг 5.9 – Запрос на блокировку пользователя

Если при выполнении запроса возникает ошибка, шлюз API возвращает понятное сообщение, не раскрывающее внутренние технические детали реализации. Например, при попытке вступить в пространство по уже использованному invite-коду ответ сервера имеет вид, представленный в листинге 5.10.

```
{
  "statusCode": 409,
  "message": "Вы уже участник этого пространства",
  "spaceId": "ws-7af3-..."
}
```

Листинг 5.10 – Ответ сервера с ошибкой повторного вступления в пространство

Таким образом, приведённые листинги фиксируют основные точки взаимодействия клиентской и серверной частей системы. Они необходимы для тестирования, сопровождения, расширения функциональности и интеграции системы с другими программными компонентами.

5.5 Инструкция запуска

Инструкция запуска предназначена для развёртывания системы на рабочем компьютере или сервере. Разработанное приложение запускается с использованием Docker и Docker Compose. Такой способ позволяет одновременно поднять все микросервисы серверной части, клиентское приложение Next.js, балансировщик nginx, базу данных PostgreSQL, кэш-хранилище Redis, объектное хранилище MinIO и SMTP-приёмник MailHog в отдельных контейнерах, не настраивая каждую службу вручную.

Перед запуском на компьютере должны быть установлены Docker и Docker Compose. Также необходимо иметь исходные файлы проекта, включающие каталоги backend и frontend с собственными Dockerfile, каталог docker с конфигурациями nginx и PostgreSQL, файл docker-compose.yml в корне проекта, файл .env с переменными окружения и файл .env.example в качестве шаблона. Файл .env используется для хранения параметров подключения к базам данных, ключей JWT, параметров OAuth-провайдеров, ключа доступа к большой языковой модели и настроек MinIO.

Структура проекта для запуска совпадает с приведённой ранее на рисунке 3.13 и содержит файлы, представленные в листинге 5.11.

```
nova-app/
├── docker-compose.yml
├── .env
├── .env.example
├── backend/
│   ├── Dockerfile
│   └── apps/
│       └── api-gateway/
```

```

├── auth-service/
├── users-service/
├── boards-service/
├── collaboration-service/
├── libs/
│   ├── common/
│   └── proto/
├── frontend/
│   ├── Dockerfile
│   ├── src/
│   └── public/
├── docker/
│   ├── nginx/
│   │   └── nginx.conf
│   ├── postgres/
│   │   └── init.sql

```

Листинг 5.11 – Структура проекта для запуска

Перед первым запуском необходимо создать файл `.env` на основе примера `.env.example`. В нём задаются параметры баз данных, ключи аутентификации, параметры OAuth-провайдеров, настройки большой языковой модели и параметры объектного хранилища. Пример содержимого ключевых переменных файла `.env` представлен в листинге 5.12.

```

# JWT
JWT_SECRET=replace_me_with_random_string
JWT_ACCESS_EXPIRES=15m
JWT_REFRESH_EXPIRES=7d

# Адрес клиента (используется в OAuth callback)
CLIENT_URL=http://localhost:3000

# PostgreSQL — две базы в одном контейнере
POSTGRES_HOST=postgres
POSTGRES_PORT=5432
POSTGRES_USER=nova
POSTGRES_PASSWORD=nova_secret
POSTGRES_USERS_DB=nova_users
POSTGRES_BOARDS_DB=nova_boards

# Redis
REDIS_HOST=redis
REDIS_PORT=6379

# MinIO (объектное хранилище для ассетов)
MINIO_ENDPOINT=http://minio:9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin12345
MINIO_BUCKET=nova-board-assets

# OAuth-провайдеры
GOOGLE_CLIENT_ID=
GOOGLE_CLIENT_SECRET=
GITHUB_CLIENT_ID=
GITHUB_CLIENT_SECRET=

# AI-ассистент
AI_API_KEY=

```

```
AI_BASE_URL=https://api.artemox.com/v1
AI_MODEL=gpt-5.4
```

Листинг 5.12 – Ключевые переменные файла .env

После настройки переменных окружения необходимо перейти в корневой каталог проекта и выполнить команду запуска, представленную в листинге 5.13.

```
docker compose up -d
```

Листинг 5.13 – Команда запуска контейнеров

Данная команда запускает все сервисы, описанные в файле docker-compose.yml, в фоновом режиме. В результате должны быть созданы и запущены контейнеры postgres, redis, minio, mailhog, четырёх backend-микросервисов (users-service, auth-service, boards-service, collaboration-service), трёх инстансов шлюза api-gateway-1, api-gateway-2, api-gateway-3, балансировщика nginx и клиентского приложения frontend.

Для проверки состояния контейнеров используется команда, представленная в листинге 5.14.

```
docker compose ps
```

Листинг 5.14 – Команда проверки состояния контейнеров

Все контейнеры должны находиться в состоянии running или healthy. Контейнеры postgres, redis и minio имеют настроенные healthcheck-проверки, поэтому остальные сервисы (auth-service, boards-service, collaboration-service, frontend) запускаются только после того, как соответствующие хранилища готовы к работе. Это необходимо для того, чтобы микросервисы не пытались подключиться к базам данных или объектному хранилищу до их полной готовности.

Для просмотра логов конкретного сервиса используется следующая команда (листинг 5.15).

```
docker compose logs -f api-gateway-1
```

Листинг 5.15 – Команда просмотра логов инстанса шлюза API

Для просмотра логов контейнера базы данных используется команда, представленная в листинге 5.16.

```
docker compose logs -f postgres
```

Листинг 5.16 – Команда просмотра логов контейнера базы данных

После успешного запуска клиентское приложение должно быть доступно в браузере по адресу `http://localhost:3000`. Шлюз API доступен по адресу `http://localhost:4000`, административная консоль MinIO — по адресу `http://localhost:9001`, а web-интерфейс MailHog для просмотра отправленных одноразовых кодов в режиме разработки — по адресу `http://localhost:8025`. Если приложение запускается на сервере, вместо `localhost` используется IP-адрес или доменное имя сервера.

Порядок первичной проверки работоспособности системы представлен в таблице 5.5.

Таблица 5.5 – Проверка работоспособности системы после запуска

Шаг	Проверяемое действие	Ожидаемый результат
1	Выполнить <code>docker compose ps</code>	Контейнеры <code>postgres</code> , <code>redis</code> , <code>minio</code> , <code>mailhog</code> , четырёх микросервисов, трёх инстансов <code>api-gateway</code> , <code>nginx</code> и <code>frontend</code> находятся в состоянии <code>running</code> или <code>healthy</code>
2	Открыть приложение в браузере по адресу <code>http://localhost:3000</code>	Отображается посадочная страница системы
3	Перейти к форме аутентификации и запросить одноразовый код	Код принимается через MailHog по адресу <code>http://localhost:8025</code>
4	Ввести полученный код в форме OTP	Создаётся учётная запись и устанавливаются <code>httpOnly-cookies</code> <code>accessToken</code> и <code>refreshToken</code>
5	Создать рабочее пространство и интерактивную доску	Пространство и доска успешно создаются, доска открывается на холсте
6	Открыть доску во второй вкладке браузера от имени второго пользователя	Изменения синхронизируются между двумя клиентами в режиме реального времени, отображаются курсоры участников
7	Загрузить изображение на холст	Файл сохраняется в <code>bucket nova-board-assets</code> контейнера <code>minio</code> и отображается на холсте у всех участников
8	Открыть AI-ассистента и отправить тестовый запрос	Модель возвращает потоковый ответ и при необходимости размещает элементы на холсте
9	Выполнить вход под учётной записью с ролью ADMIN	Доступен раздел <code>/app/admin</code> со списком пользователей, досок, пространств и журналом действий

Если требуется остановить систему, используется команда, представленная в листинге 5.17.

```
docker compose down
```

Листинг 5.17 – Команда остановки контейнеров

Если необходимо остановить контейнеры без удаления данных, достаточно выполнить обычную остановку. Данные PostgreSQL сохраняются в постоянном томе `postgres_data`, ассеты MinIO — в томе `minio-data`, поэтому при повторном запуске они не должны исчезнуть. Если требуется полностью удалить контейнеры и тома, следует использовать расширенную команду с удалением volumes, однако это приведёт к потере всех данных пользователей и содержимого досок (листинг 5.18).

```
docker compose down -v
```

Листинг 5.18 – Команда остановки контейнеров с удалением томов

Перед выполнением такой команды необходимо создать резервные копии обеих баз данных `nova_users` и `nova_boards`. Для создания резервных копий используются команды `pg_dump`, представленные в листинге 5.19.

```
docker exec nova-app-postgres-1 pg_dump -U nova nova_users >
backup/nova_users_backup.sql
docker exec nova-app-postgres-1 pg_dump -U nova nova_boards >
backup/nova_boards_backup.sql
```

Листинг 5.19 – Команды резервного копирования баз данных

Для восстановления данных из резервных копий используются команды, представленные в листинге 5.20.

```
docker exec -i nova-app-postgres-1 psql -U nova nova_users <
backup/nova_users_backup.sql
docker exec -i nova-app-postgres-1 psql -U nova nova_boards <
backup/nova_boards_backup.sql
```

Листинг 5.20 – Команды восстановления баз данных из резервных копий

После восстановления необходимо повторно проверить работоспособность приложения: открыть посадочную страницу, выполнить вход, убедиться в наличии рабочих пространств и досок, открыть интерактивную доску и убедиться в корректности отображения её элементов.

Система считается готовой к эксплуатации, если выполняются следующие условия: все контейнеры успешно запускаются, клиентское приложение доступно через браузер, шлюз API принимает HTTP-запросы, контейнер PostgreSQL обслуживает обе базы `nova_users` и `nova_boards`, Redis принимает подключения, MinIO предоставляет `presigned URL` и хранит ассеты, пользователь может пройти аутентификацию по одноразовому коду или через OAuth-провайдера, создать

рабочее пространство и доску, выполнить совместное редактирование с участием второго пользователя, загрузить изображение на холст, получить ответ от AI-ассистента, а административные функции доступны только пользователю с ролью ADMIN или SUPER_ADMIN.

Таким образом, инструкция запуска описывает порядок подготовки окружения, настройки переменных, запуска контейнеров, проверки состояния приложения и выполнения базовых эксплуатационных действий. Это позволяет развернуть систему в воспроизводимой среде и подготовить её к демонстрации или дальнейшему использованию.

ПРОВЕРКА СООТВЕТСТВИЯ ТРЕБОВАНИЯМ

Для подтверждения соответствия разработанной информационной системы требованиям методических указаний была выполнена проверка структуры отчёта, проектных решений, реализованных функций, результатов тестирования и подготовленной эксплуатационной документации.

В таблице 6.1 приведено соответствие основных требований методических указаний и разделов пояснительной записки, в которых данные требования раскрыты или подтверждены. В качестве подтверждения используются номера разделов, рисунки, таблицы, листинги и адреса REST API.

Таблица 6.1 – Проверка соответствия требованиям методических указаний

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
1	Наличие титульного листа по установленной форме	Стр. 1	Титульный лист курсового проекта по дисциплине «Создание программного обеспечения»	Выполнено
2	Наличие задания на курсовой проект	Стр. 2	Лист задания с темой, исходными данными, перечнем вопросов и сроком представления	Выполнено
3	Наличие реферата с целью, содержанием работы и результатами	Стр. 3	Реферат содержит цель работы, использованные технологии и ключевой результат разработки	Выполнено
4	Наличие перечня сокращений и обозначений	Стр. 4	Приведены сокращения БД, ИИ, API, REST, gRPC, JWT, OAuth, OTP, DTO, ORM, MVC, SSR, JSON, WS, S3, LLM и другие	Выполнено
5	Наличие введения с актуальностью, объектом, предметом, целью и задачами	Стр. 5–6	Описаны актуальность совместной работы с интерактивными досками, объект, предмет, цель и задачи проекта	Выполнено
6	Описание цели разработки	Раздел 1.1	Указано, что целью является создание веб-приложения совместной работы с интерактивными досками с поддержкой AI-ассистента	Выполнено

Таблица 6.1 – Продолжение

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
7	Описание назначения системы	Раздел 1.2	Описаны функции совместного редактирования, организации рабочих пространств, приглашения участников и взаимодействия с AI-ассистентом	Выполнено
8	Описание ролей пользователей и разграничения доступа	Разделы 1.2, 3.2, 5.3	Описаны роли USER, MANAGER, ADMIN, SUPER_ADMIN; приведена таблица 5.2 ролей и схема прав доступа через JwtAuthGuard и RolesGuard	Выполнено
9	Описание профиля предметной области и проблемы «как есть»	Раздел 1.3	Описаны существующие решения Miro, FigJam, Microsoft Whiteboard и проблемы интеграции AI с совместным редактированием в реальном времени	Выполнено
10	Использование каскадной модели жизненного цикла	Раздел 1.4	Описана последовательность этапов разработки; приведена диаграмма Ганта в файле Диаграмма_Ганта.csv	Выполнено
11	Наличие требований к UI/UX и эргономике	Раздел 1.5	Описаны требования к удобству, реактивности, минималистичности интерфейса холста и адаптивности	Выполнено
12	Наличие требований к архитектуре и технологическому стеку	Раздел 1.6	Описаны микросервисная архитектура NestJS, клиент Next.js 16, gRPC между сервисами, PostgreSQL, Redis, MinIO	Выполнено
13	Наличие функциональных требований	Раздел 1.7	Описаны аутентификация, разграничение прав, управление пространствами и досками, совместное редактирование, загрузка изображений, AI-ассистент	Выполнено
14	Наличие требований к модулю искусственного интеллекта	Раздел 1.8	Описаны интеграция с большой языковой моделью, набор tool-вызовов (создание, обновление, перемещение, удаление элементов), потоковая обработка ответа	Выполнено

Таблица 6.1 – Продолжение

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
15	Наличие требований к данным и базе данных	Раздел 1.9	Описаны сущности users, audit_logs, spaces, space_members, boards, canvas_elements, favorites, assets; приведено разделение на базы nova_users и nova_boards	Выполнено
16	Наличие ER-диаграммы базы данных	Раздел 1.9	Приведена схема базы данных в файле schema_dbdiagram.dbml с описанием связей между сущностями	Выполнено
17	Наличие требований по стандартам	Раздел 1.10	Приведены требования по корректности данных, безопасности (JWT в httpOnly-cookies, OAuth-провайдеры), журналированию и устойчивости	Выполнено
18	Наличие требований к тестированию	Раздел 1.11	Описаны автоматизированное юнит-тестирование, ручные функциональные тесты, проверка сквозных бизнес-процессов и прав доступа	Выполнено
19	Наличие требований к документации и вводу в эксплуатацию	Раздел 1.12	Описаны руководство пользователя, руководство администратора, описание API и инструкция запуска	Выполнено
20	Наличие критериев успешности проекта	Раздел 1.13	Описаны критерии полноты реализации, готовности к демонстрации и работоспособности системы	Выполнено
21	Наличие диаграммы вариантов использования	Раздел 2.1	Приведена диаграмма вариантов использования в файле use_case_diagram.puml; показаны актёры USER, ADMIN, SUPER_ADMIN и основные сценарии	Выполнено
22	Наличие диаграмм активности	Раздел 2.2	Приведены диаграммы активности: совместное редактирование, аутентификация, работа AI-ассистента (файлы activity_2_2...2_4.puml)	Выполнено

Таблица 6.1 – Продолжение

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
23	Наличие диаграммы процесса обработки данных	Раздел 2.3	Приведена диаграмма обработки данных в файле activity_2_5_data_processing.puml: ввод элементов, валидация, сохранение, рассылка событий	Выполнено
24	Наличие диаграммы классов	Раздел 2.4	Приведена диаграмма классов в файле class_diagram_2_6.puml: сервисы, контроллеры, сущности TypeORM, DTO	Выполнено
25	Отражение паттернов проектирования	Разделы 2.4, 3.1–3.2	Описаны паттерны Controller–Service–Repository, Dependency Injection, Adapter, микросервисная декомпозиция	Выполнено
26	Наличие диаграммы развёртывания или компонентов	Раздел 2.5	Приведена диаграмма развёртывания в файле deployment_2_7.puml: клиент, nginx, шлюзы API, микросервисы, контейнеры хранилищ	Выполнено
27	Описание структуры проекта и технологического стека	Раздел 3.1	Описаны структура проекта (backend NestJS, frontend Next.js, docker, отчёт), стек технологий, межсервисное взаимодействие по gRPC	Выполнено
28	Реализация серверной части	Раздел 3.2	Описаны микросервисы api-gateway, auth-service, users-service, boards-service, collaboration-service; контроллеры, сервисы, guards, фильтры исключений	Выполнено
29	Реализация REST API	Разделы 3.2 и 5.4	Описаны контроллеры и основные ресурсы API; приведены endpoints /api/auth/send-code, /api/auth/verify-code, /api/spaces, /api/boards, /api/admin/users и другие	Выполнено
30	Реализация пользовательского интерфейса	Раздел 3.3	Приведены экраны посадочной страницы, аутентификации (EmailStep, OTPStep, OAuthProviderList), личного кабинета, доски (BoardLeftToolbar, BoardTopBar, PresencePanel), AI-ассистента (AiChatWidget)	Выполнено

Таблица 6.1 – Продолжение

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
31	Реализация модуля искусственного интеллекта	Раздел 3.4	Описаны AiService, AiGateway, набор tool-вызовов update/move/delete элементов, построение динамического контекста доски, потоковая обработка ответа модели	Выполнено
32	Наличие контейнеризации и совместного запуска	Разделы 3.5 и 5.5	Описаны Docker, Docker Compose, контейнеры PostgreSQL, Redis, MinIO, MailHog, четырёх микросервисов, трёх инстансов api-gateway, балансировщика nginx и фронтенда; приведён листинг 4.1	Выполнено
33	Раздельное развёртывание серверной части и базы данных	Раздел 3.5	Backend-микросервисы и контейнер PostgreSQL описаны как отдельные сервисы в docker-compose.yml; БД хранит данные в постоянном томе postgres_data; healthcheck pg_isready гарантирует запуск микросервисов только после готовности БД	Выполнено
34	Наличие инструкции по запуску	Раздел 5.5	Описаны .env, команды docker compose up -d, ps, logs, down; приведены листинги 5.11–5.18 со структурой проекта, переменными окружения и командами управления	Выполнено
35	Наличие механизма резервного копирования и восстановления	Разделы 3.5 и 5.5	Описаны команды pg_dump и psql для двух баз nova_users и nova_boards; приведены листинги 5.19 и 5.20	Выполнено
36	Наличие тестирования серверной части	Раздел 4, таблицы 4.1 и 4.2	Проведено 11 наборов автоматизированных юнит-тестов Jest в микросервисах NestJS и ручные сценарии серверной части	Выполнено
37	Наличие тестирования пользовательского интерфейса	Раздел 4, таблицы 4.1 и 4.3	Проведено 18 наборов автоматизированных юнит-тестов Jest сервисов и утилит фронтенда и ручные сценарии UI-компонентов и форм	Выполнено

Таблица 6.1 – Продолжение

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
38	Наличие тестирования бизнес-процессов	Раздел 4, таблица 4.4	Проверены аутентификация, OAuth-вход, создание пространств и приглашение участников, совместное редактирование, AI-ассистент, разграничение прав	Выполнено
39	Наличие таблицы дефектов и исправлений	Раздел 4, таблица 4.5	Зафиксированы дефекты, их проявление, выполненные исправления и итоговый результат повторной проверки	Выполнено
40	Наличие итоговой оценки результатов тестирования	Раздел 4, таблица 4.6	Подтверждены аутентификация, разграничение прав доступа, совместное редактирование, AI-ассистент, хранение данных и контейнеризация	Выполнено
41	Подготовка комплекта эксплуатационной документации	Раздел 5	Подготовлены текстовое описание системы, руководство пользователя, руководство администратора, описание API и инструкция запуска	Выполнено
42	Наличие текстового описания системы	Раздел 5.1	Описаны назначение системы, микросервисная архитектура, роли, бизнес-процессы, модель данных и роль модуля искусственного интеллекта	Выполнено
43	Наличие руководства пользователя	Раздел 5.2	Таблица 5.1 описывает порядок работы пользователя: открытие приложения, аутентификация, создание пространства, работа с доской, AI-ассистент	Выполнено
44	Наличие руководства администратора	Раздел 5.3	Таблицы 5.2 и 5.3 описывают роли пользователей и действия администратора при типовых ситуациях (БД, Redis, MinIO, MailHog, AI)	Выполнено
45	Наличие описания API с примерами запросов и ответов	Раздел 5.4	Таблица 5.4 содержит 25 REST-ресурсов; листинги 5.1–5.10 содержат примеры запросов и ответов системы	Выполнено

Таблица 6.1 – Окончание

№	Требование методических указаний	Где выполнено в отчёте	Подтверждение выполнения	Статус
46	Наличие инструкции запуска через Docker Compose	Раздел 5.5	Листинги 5.13–5.16 содержат команды docker compose up -d, ps, logs; приведены параметры запуска всех контейнеров системы	Выполнено
47	Проверка работоспособности после запуска	Раздел 5.5, таблица 5.5	Описаны шаги проверки: состояние контейнеров, посадочная страница, OTP через MailHog, создание пространства и доски, совместное редактирование, загрузка изображения, AI, доступ ADMIN	Выполнено
48	Наличие вывода о готовности системы к демонстрации	Раздел 4, стр. в конце; раздел 5.5	Указано, что система прошла тестирование, контейнеры запускаются одной командой, выполняются аутентификация, совместное редактирование, работа AI-ассистента и административные функции	Выполнено

По результатам проверки можно сделать вывод, что отчёт в целом соответствует структуре и обязательным требованиям методических указаний. В работе представлены техническое задание, проектирование информационной системы, реализация серверной части в виде микросервисов NestJS и клиентской части на Next.js, модуль искусственного интеллекта с набором tool-вызовов, контейнеризация через Docker Compose, результаты автоматизированного и ручного тестирования, эксплуатационная документация и инструкция запуска. Все основные требования методических указаний имеют отражение в соответствующих разделах отчёта.

ЗАКЛЮЧЕНИЕ

В результате выполнения курсовой работы было разработано клиент-серверное веб-приложение совместной работы с интерактивными досками с поддержкой искусственного интеллекта на платформах NestJS и Next.js. Основной целью проекта было предоставить пользователям инструмент для совместного создания и редактирования визуального контента в режиме реального времени с возможностью автоматизации части работы через встроенного AI-ассистента.

Для достижения этой цели были выполнены следующие задачи: разработка серверной части в виде микросервисов `api-gateway`, `auth-service`, `users-service`, `boards-service` и `collaboration-service` с межсервисным взаимодействием по протоколу `gRPC`; разработка клиентской части на основе Next.js 16 с серверным рендерингом и совместным редактированием по `WebSocket`; реализация модуля искусственного интеллекта с интеграцией большой языковой модели и набором `tool`-вызовов для управления элементами холста; контейнеризация системы через `Docker Compose` с поднятием всех зависимостей в отдельных контейнерах.

В результате работы получена работоспособная и удобная система, обеспечивающая совместное создание интерактивных досок несколькими пользователями в режиме реального времени, организацию рабочих пространств с приглашением участников, загрузку изображений на холст и взаимодействие с AI-ассистентом. Разработанное приложение может быть использовано проектными командами, образовательными учреждениями и распределёнными коллективами для визуальной совместной работы с поддержкой искусственного интеллекта.

Также в ходе выполнения курсовой работы были получены навыки разработки микросервисных систем на платформе NestJS, построения клиент-серверных приложений на основе Next.js, организации межсервисного взаимодействия по `gRPC`, реализации совместного редактирования в реальном времени через `WebSocket` с использованием Redis-адаптера, интеграции с большой языковой моделью и контейнеризации системы через `Docker Compose`. Эти навыки могут быть применены при разработке современных веб-приложений, требующих

масштабируемой архитектуры, совместной работы пользователей и интеграции с моделями искусственного интеллекта.

Список использованных источников

1. Sailer M., Schultz-Pernice F., Fischer F. Contextual facilitators for learning activities involving technology in higher education: The Cb -model // Computers in Human Behavior. – 2021. – Vol. 121. – Article 106794. – DOI: 10.1016/j.chb.2021.106794.
2. Yang X., Tang H., Xiu Z., Liu J. Collaborative whiteboard systems for distributed teams: a systematic review // International Journal of Human-Computer Studies. – 2023. – Vol. 178. – Article 103079. – DOI: 10.1016/j.ijhcs.2023.103079.
3. Miro. The Visual Workspace for Innovation [Электронный ресурс] : Official documentation. – URL: <https://miro.com/online-whiteboard/> (дата обращения: 28.05.2026).
4. Figma, Inc. FigJam — Online Whiteboard for Teams [Электронный ресурс]. – URL: <https://www.figma.com/figjam/> (дата обращения: 28.05.2026).
5. Unidraw. Open-source collaborative whiteboard [Электронный ресурс]. – URL: <https://github.com/unidraw/unidraw> (дата обращения: 28.05.2026).
6. Microsoft Corporation. Microsoft Whiteboard documentation [Электронный ресурс]. – URL: <https://learn.microsoft.com/en-us/microsoft-365-apps/whiteboard/> (дата обращения: 28.05.2026).
7. OpenAI. GPT-5 Technical Report [Электронный ресурс]. – 2025. – URL: <https://openai.com/research/gpt-5> (дата обращения: 28.05.2026).
8. Bommasani R. et al. On the Opportunities and Risks of Foundation Models // arXiv preprint. – 2024. – arXiv:2108.07258v3. – URL: <https://arxiv.org/abs/2108.07258> (дата обращения: 28.05.2026).
9. NestJS — A progressive Node.js framework [Электронный ресурс] : Official documentation. – URL: <https://docs.nestjs.com/> (дата обращения: 28.05.2026).
10. Vercel Inc. Next.js Documentation [Электронный ресурс]. – URL: <https://nextjs.org/docs> (дата обращения: 28.05.2026).
11. Meta Platforms, Inc. React Documentation [Электронный ресурс]. – URL: <https://react.dev/learn> (дата обращения: 28.05.2026).

12. Microsoft Corporation. TypeScript Handbook [Электронный ресурс]. – URL: <https://www.typescriptlang.org/docs/handbook/> (дата обращения: 28.05.2026).
13. OpenJS Foundation. Node.js Documentation [Электронный ресурс]. – URL: <https://nodejs.org/en/docs> (дата обращения: 28.05.2026).
14. Socket.IO. Real-time bidirectional event-based communication [Электронный ресурс]. – URL: <https://socket.io/docs/v4/> (дата обращения: 28.05.2026).
15. gRPC Authors. gRPC — A high performance, open source universal RPC framework [Электронный ресурс]. – URL: <https://grpc.io/docs/> (дата обращения: 28.05.2026).
16. PostgreSQL Global Development Group. PostgreSQL 16 Documentation [Электронный ресурс]. – URL: <https://www.postgresql.org/docs/16/> (дата обращения: 28.05.2026).
17. TypeORM. Documentation for the TypeORM library [Электронный ресурс]. – URL: <https://typeorm.io/> (дата обращения: 28.05.2026).
18. Redis Ltd. Redis Documentation [Электронный ресурс]. – URL: <https://redis.io/docs/> (дата обращения: 28.05.2026).
19. MinIO, Inc. MinIO Object Storage Documentation [Электронный ресурс]. – URL: <https://min.io/docs/minio/linux/index.html> (дата обращения: 28.05.2026).
20. Docker Inc. Docker Compose Specification [Электронный ресурс]. – URL: <https://docs.docker.com/compose/> (дата обращения: 28.05.2026).
21. Jones M., Bradley J., Sakimura N. JSON Web Token (JWT) // RFC 7519. – Internet Engineering Task Force, 2015. – URL: <https://datatracker.ietf.org/doc/html/rfc7519> (дата обращения: 28.05.2026).
22. Hardt D. The OAuth 2.0 Authorization Framework // RFC 6749. – Internet Engineering Task Force, 2012. – URL: <https://datatracker.ietf.org/doc/html/rfc6749> (дата обращения: 28.05.2026).
23. Fielding R. T. Architectural Styles and the Design of Network-based Software Architectures : Doctoral dissertation. – University of California, Irvine, 2000. – 162 p.
24. Newman S. Building Microservices: Designing Fine-Grained Systems. – 2nd ed. – O'Reilly Media, 2021. – 612 p. – ISBN 978-1-4920-3402-5.

25. Shapiro M., Preguiça N., Baquero C., Zawirski M. Conflict-Free Replicated Data Types // Stabilization, Safety, and Security of Distributed Systems (SSS 2011). – Lecture Notes in Computer Science. – Springer, 2011. – Vol. 6976. – P. 386–400. – DOI: 10.1007/978-3-642-24550-3_29.